AI Agents



AGENT ENGINEERING

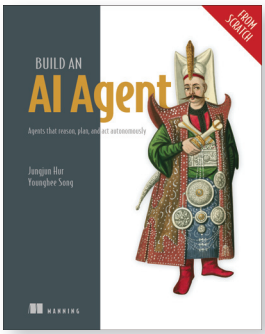
TOOLS, WORKFLOWS, AND FEEDBACK

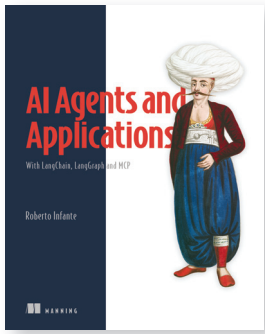
A practical progression from basic agent concepts to dependable, tool-using agentic workflows

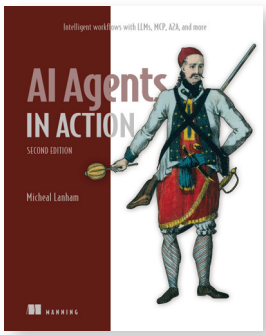
Jungjun Hur
Younghee Song
Roberto Infante
Peter Belcak
Micheal Lanham

MANNING

Save 45% on any one of the books listed below by using the code AGENTENG45 at checkout.









Save 50% on 2 or more of these books, here.



Get all 4 ebooks for $99.99, here.





Agent Engineering

Tools, Workflows, and Feedback

Copyright 2027 Manning Publications

For online information and ordering of this and other Manning books, please visit www.manning.com. The publisher offers discounts on this book when ordered in quantity.

For more information, please contact

Special Sales Department
Manning Publications Co.
20 Baldwin Road
PO Box 761
Shelter Island, NY 11964
Email: orders@manning.com

© 2027 Manning Publications Co. All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by means electronic, mechanical, photocopying, or otherwise, without prior written permission of the publisher.

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in the book, and Manning Publications was awware of a trademark claim, the designations have been printed in initial caps or all caps.

Recognizing the importance of preserving what has been written, it is Manning's policy to have the books we publish printed on acid-free paper, and we exert our best efforts to that end. Recognizing also our responsibility to conserve the resources of our planet, Manning books are printed on paper that is at least 15 percent recycled and processed without the use of elemental chlorine.

The author and publisher have made every effort to ensure that the information in this book was correct at press time. The author and publisher do not assume and hereby disclaim any liability to any party for any loss, damage, or disruption caused by errors or omissions, whether such errors or omissions result from negligence, accident, or any other cause, or from any usage of the information herein.



Manning Publications Co.
20 Baldwin Road
PO Box 761
Shelter Island, NY 11964

ISBN 9781638359265
Printed in the United States of America

contents

introduction *iv*

What is an AI agent? 2

Chapter 1 from *Build an AI Agent (From Scratch)* by Jungjun Hur and Younghee Song

Building tool-based agents with LangGraph 19

Chapter 11 from *AI Agents and Applications* by Roberto Infante

Errand: Breaking complex workflows into units 46

Chapter 3 from *Agent Design Patterns* by Peter Belcak

Building robust agents with evaluation and feedback 63

Chapter 7 from *AI Agents in Action, Second Edition* by Micheal Lanham

introduction

How do you transform an LLM into an agent? Add tools and a loop that lets the model choose actions and continue working toward a goal—even when the path cannot be mapped out in advance. This sampler, featuring four chapters from Manning books, explores when agents make sense, how they act, how complex behavior can be organized, and how their results can be evaluated and improved.

"What is an AI agent?," from *Build an AI Agent (From Scratch)* by **Jungjun Hur and Younghee Song**, starts with explaining the three elements behind agent autonomy. Then, you'll compare agents with developer-defined workflows, consider when to use each, and examine the GAIA benchmark and context-engineering strategies. This solid foundation will help you recognize when a task calls for agent-directed decisions—and when a predictable workflow is the better choice.

Next, "Building tool-based agents with LangGraph," from *AI Agents and Applications* by **Roberto Infante**, puts those ideas to work in a travel information agent. You'll register and execute tools, maintain conversation state, add conditional routing, inspect messages and tool calls, and compare a custom graph with LangGraph's prebuilt ReAct agent. By following the agent's execution, you'll gain a clearer understanding of how tools, state, and control flow fit together—knowledge you can apply when building and debugging your own LangGraph agents.

As workflows become more complex, "Errand: Breaking complex workflows into units," from *Agent Design Patterns* by **Peter Belcak**, offers a way to divide them into focused, testable routines. You'll explore how errands support

maintenance, reuse, and parallel execution while limiting what enters the wider context. The chapter's persona, reflection, compaction, structured-extraction, and editing errands will give you practical patterns for making complex agent behavior easier to test, manage, and extend.

Finally, "Building robust agents with evaluation and feedback," from *AI Agents in Action, Second Edition* by **Micheal Lanham**, focuses on finding weaknesses and improving behavior. Using a RAG knowledge agent, you'll work with test-driven development, benchmarks, critic and grounding agents, evaluation rubrics, and guardrails. You'll also see how Arize Phoenix supports tracing, experiments, evaluators, session tracking, and feedback annotations—giving you concrete methods for inspecting agent behavior, identifying problems, and measuring the effects of changes.

Together, these combined chapters are a practical powerhouse, helping you progress from deciding when to use an agent to building, structuring, testing, and refining one. If this sampler sparks your interest and you're ready to continue exploring the concepts and tools introduced here, we invite you to purchase the complete books.

Thank you,
The Manning Team

Start with the pieces that turn an LLM into an agent: tools and a loop for choosing and carrying out actions. In this chapter, you'll compare agents with predefined workflows, consider when each approach fits, and explore the GAIA benchmark for measuring agent performance and context-engineering strategies for helping agents use information effectively.

Chapter 1 from *Build an AI Agent (From Scratch)*
by Jungjun Hur and Younghee Song

What is an AI agent?

This chapter covers

- The landscape of AI agents today
- LLMs as the decision-making core of agents
- Workflows versus agents and when to use each
- GAIA benchmark for measuring agent performance
- Context engineering for building effective agents

You may have heard of agent-building frameworks such as LangGraph, CrewAI, AutoGen, and OpenAI Agents. These frameworks make it easy to build agents quickly, but they also hide what's happening inside. This book takes a different approach: we'll build agents from scratch, understanding every component before relying on any framework.

Why build from scratch? Agent development is fundamentally about debugging failures. When your agent gives a wrong answer or gets stuck in a loop, you have to understand exactly what went wrong. Did the large language model (LLM) misinterpret the context? Did a tool return unexpected results? Was crucial information

missing? Without understanding how agents work internally, diagnosing these problems is difficult regardless of which tools you use. By building each component yourself, you'll develop the mental model you need to troubleshoot any agent system, whether you built or inherited it.

This chapter establishes the foundation for everything that follows. We'll start by surveying the landscape of AI agents, from personal assistants to specialized coding tools, to understand what we're building toward. Then we'll examine how LLMs serve as the brain of an agent and what distinguishes a true agent from a simple workflow. We'll introduce General AI Assistants (GAIA), the benchmark we'll use throughout this book to measure progress, and explore context engineering, the discipline that determines whether an agent succeeds or fails. By the end, you'll have a clear mental model of what agents are, when to use them, and the principles that will guide your implementation journey.

1.1 The age of AI agents

AI agents are rapidly transforming how we interact with technology. From personal assistants that help with everyday tasks to sophisticated systems that handle complex professional work, agents are becoming integral parts of both consumer products and enterprise solutions. Before diving into how to build them, let's survey the landscape of AI agents to understand what we're working toward:

- *Personal AI agents*—These agents are the most familiar form of AI agents today. Services such as ChatGPT, Claude, and Gemini started as conversational chatbots; users asked them a question, and they provided an answer. But these systems have evolved dramatically. Modern personal agents can analyze uploaded documents, search the web for current information, generate images, write and execute code, and even help with shopping decisions. They serve as general-purpose assistants that adapt to whatever task you bring them, learning your preferences and communication style over time.
- *Customer-facing agents*—These agents operate on behalf of businesses, interacting directly with customers in real time. Going beyond simple FAQ chatbots, they can access company policies, retrieve customer data, process transactions, and make decisions based on business rules. When you interact with a support agent that checks your order status, processes a refund, or helps troubleshoot a product issue, you're increasingly likely to be communicating with an AI agent. These systems must balance helpfulness with strict adherence to company guidelines and regulatory requirements.
- *Specialized agents*—These agents tackle domain-specific tasks that require deep expertise or extended processing time. Coding agents like Claude Code, Cursor, and Codex CLI can navigate codebases, implement features, fix bugs, and refactor code, which are tasks that previously required hours of developer time. Deep Research agents can conduct comprehensive investigations across hundreds of sources, synthesizing findings into detailed reports. Because these tasks often

take significant time to complete, specialized agents frequently operate asynchronously, notifying users when results are ready. When connected to proprietary enterprise data, these agents become vertical solutions tailored to specific industries or use cases.

Despite their different applications and interfaces, these agents have a common foundation: they're powered by LLMs. Understanding how LLMs work and how they enable agent capabilities is essential for anyone who wants to build effective agents. In this book, we use *AI agent* and *LLM agent* interchangeably because virtually all modern AI agents are powered by LLMs.

1.2 Understanding LLM agents

All the agents we explored share something remarkable: they rely on LLMs as their decision-making core. But how does a system designed to predict the next word become capable of completing multistep tasks autonomously? First, let's unpack what makes LLMs uniquely suited to power agents; then we'll examine the three essential components that transform an LLM into an agent. This material helps you understand the agent loop that forms the foundation for everything you'll build in this book.

1.2.1 What is an LLM?

An LLM is a language model trained on nearly all publicly available text from the internet. Although modern LLMs have evolved into multimodal models that process images, audio, and video, this book uses the term *LLM* to refer to all these variants because they share a foundational architecture.

The core principle of LLMs is simple: learning to predict the next word by processing massive amounts of text data. Given the sentence "Because it's summer vacation, I didn't go to _____," most people would expect the blank to be filled with the word *school*. Through large-scale training on this simple principle, LLMs develop an understanding of language structure, context, and meaning. This ability to apply learned patterns to new examples is a form of generalization, as shown in figure 1.1. LLMs process text by breaking it into *tokens*, which are typically words or word fragments. The model predicts the next token based on all preceding tokens.

Recently, reasoning models have emerged that analyze tasks or formulate plans before producing results. The reasoning model doesn't answer immediately; first, it generates thinking tokens starting

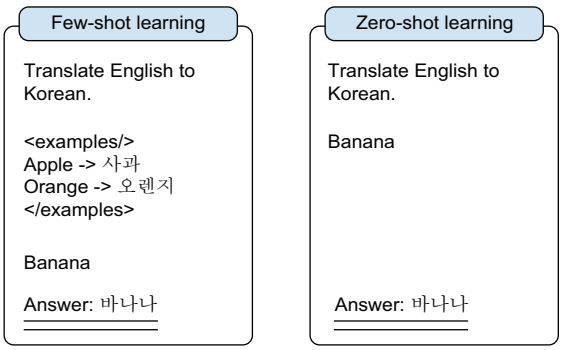


Figure 1.1 Example of a language model's generalization capability

with <think>. This advance enables LLM agents to tackle complex and unfamiliar problems effectively.

1.2.2 What is an LLM agent?

A representative example of an LLM agent is a research agent. When a user asks, "Summarize the 2024 Nobel Physics Prize winners' research," the agent gathers information from multiple sources, analyzes findings, and produces a comprehensive report.

As shown in figure 1.2, the research agent doesn't simply generate an answer from its training data; it also searches for information about the 2024 Nobel Prize, identifies the winners (Geoffrey Hinton and John J. Hopfield), explores academic papers, and consults Wikipedia. Then it synthesizes all this information into a coherent report detailing the winners' achievements and their effect on AI.

What's worth noticing is who decided each step: which source to check first, whether the search results were sufficient, what to look for next, and when to start writing the report. These decisions weren't hardcoded; the LLM made them on the fly, based on the context at each moment.

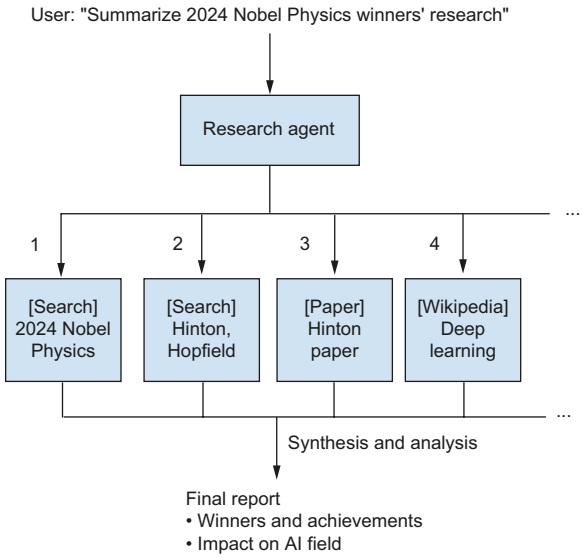


Figure 1.2 User requests flow through the research agent, which branches into multiple searches and synthesis.

CORE DEFINITION: AUTONOMOUS CONTROL FLOW

This example reveals the essence of an agent. An LLM agent is a program that hands part of its control flow to an LLM, letting the model decide what actions to take and when to stop based on its current context and goals.

By *control flow*, we mean the sequence of "what to do next." In traditional software, this flow is spelled out in the code. `if` statements, `for` loops, and the order of function calls

were all decided by the developer in advance. In an LLM agent, part of this decision-making authority is handed over to the LLM.

ELEMENTS THAT REALIZE AUTONOMY: LLM + TOOL + LOOP

For autonomous control flow to work in practice, three structural elements come together. The *LLM* serves as the agent's brain. It understands the current situation and decides what to do next. *Tools* extend the agent's action space into the external world, including web search, code execution, and database access. The *loop* is the execution structure that lets these decisions happen repeatedly until the goal is achieved. To summarize, the LLM makes autonomous decisions possible, tools expand the action space, and the loop unfolds it over time.

HOW THE AGENT LOOP WORKS

The loop is necessary because it's difficult to know in advance which tools will be needed or how many steps will be required to complete a task. Figure 1.3 shows the research task on the 2024 Nobel Prize winners from figure 1.2, viewed through the lens of the agent loop.

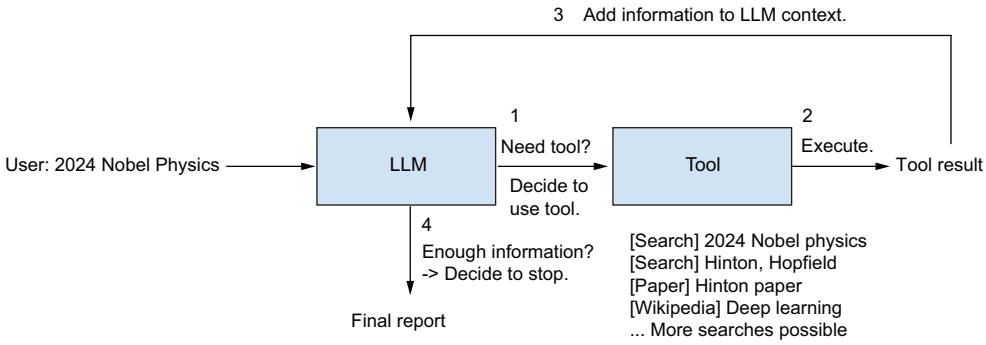


Figure 1.3 The LLM agent's decision loop is an iterative process of LLM decision-making and tool use.

The process follows a continuous loop:

1. The LLM evaluates the current context and determines whether a tool is needed. If more information is required, it decides which tool to use based on what's missing.
2. The selected tool is executed, whether it's searching for "2024 Nobel Physics," looking up "Hinton, Hopfield," finding academic papers, or accessing Wikipedia for background knowledge.
3. The tool's results are added back to the LLM's context, enriching its understanding of the topic. This accumulated context allows the LLM to make more-informed decisions in subsequent iterations.

4 The LLM decides whether to continue or stop. If it determines that sufficient information has been gathered to answer the user's question comprehensively, it generates the final report; otherwise, it returns to step 1 for another iteration.

This iterative process of reasoning, acting, and observing enables the agent to handle tasks of unpredictable complexity, from simple queries that require a single search to complex research that requires dozens of information sources.

1.3 Workflow vs. agent

When integrating LLMs into applications, developers face a fundamental architectural choice: how much control of the execution flow the LLM should have. This decision shapes everything from system reliability to operational costs. Figure 1.4 illustrates seven distinct levels of agency, progressing from left to right with increasing autonomy. The three rows in the figure represent different aspects of control: who produces the current output (current step), who decides what happens next (next step), and who defines the available options (available steps). As we move rightward, the LLM gains more control of each dimension.

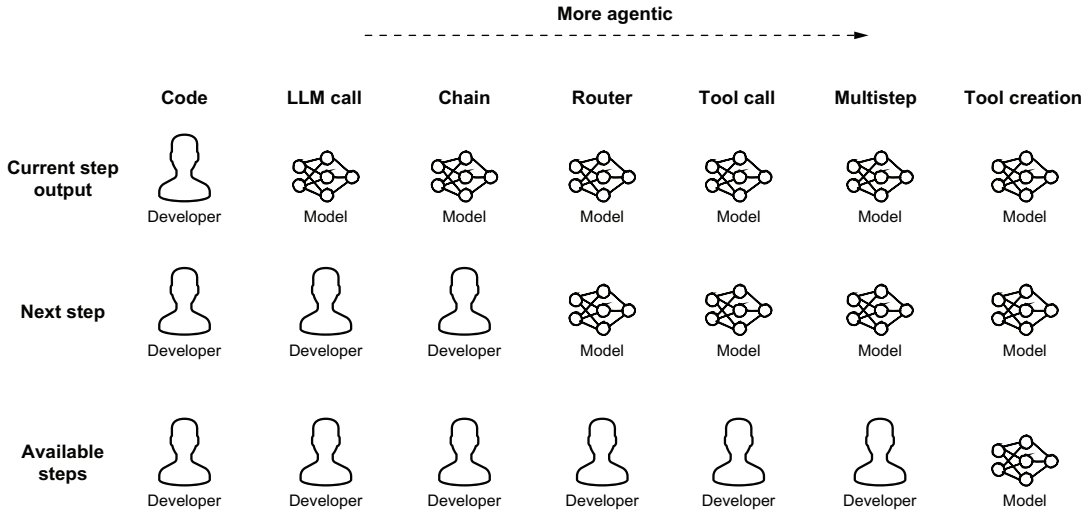


Figure 1.4 **Progression of agency levels in LLM applications**

These seven levels can be broadly categorized into two architectural approaches: workflows and agents. In *workflows*, developers predefine the execution flow and use LLMs to perform specific steps within that structure. In *agents*, LLMs determine their own processes dynamically, deciding which actions to take and when to stop. Understanding this distinction is essential in choosing the right approach for your use case.

1.3.1 Workflow: Developer-defined flow

A *workflow* is a system in which developers explicitly design the sequence of operations, with LLMs executing specific steps within that predefined structure. The key characteristic is predictability: given the same input, the system follows the same path through the workflow.

LLM CALL

The most basic pattern is a single *LLM call* in which one prompt goes in and one response comes out. This pattern delegates a specific task to the model while keeping everything else under developer control. Examples include text classification, summarization, and answering a straightforward question. Despite its simplicity, a well-crafted single call can handle surprisingly complex tasks.

CHAIN

A *chain* connects multiple LLM calls in a predefined sequence, with the output of one step becoming the input for the next. The developer designs this flow in advance, and the LLM executes each step sequentially.

This pattern follows a fundamental principle: LLMs perform best with focused, well-defined tasks. Rather than ask a model to "analyze this document and create a presentation," a chain might break this request into discrete steps: extract key points, organize by theme, generate slide content, and refine for clarity. Each step is simpler and more likely to succeed than an attempt to do everything at the same time.

ROUTER

A *router* introduces conditional logic that the LLM uses to decide which predefined path to take next. Given a user query, the model might classify it and route it to the appropriate handler. Billing questions go to one workflow; questions about technical support go to another.

Although the LLM makes the decision, it's choosing among a fixed set of options that the developer explicitly defined. The available paths and what happens along each path remain under developer control, making routers a workflow pattern rather than an agent pattern. The LLM influences the route but doesn't control the journey.

1.3.2 Agent: LLM-directed flow

An *agent* is a system in which the LLM dynamically directs its own processes and tool use, maintaining control of how it accomplishes tasks. Rather than follow a predetermined path, the agent decides what to do next based on the current context and its progress toward the goal.

TOOL USE WITH A MULTISTEP LOOP

The defining pattern of an agent combines two capabilities: the ability to use external tools and the autonomy to continue operating until the task is complete. Because LLMs can only generate text, they can't interact directly with external systems, access real-time information, or perform precise calculations. Tools bridge this gap by exposing external functionalities (such as web search, code execution, database queries, and

API calls) as callable functions. The LLM examines the available tools and decides whether one would help; if so, it specifies which tool to call with what parameters.

What transforms tool use into an agent is the loop. Rather than making a single tool call and stopping, the agent operates in a cycle: assess the current state, decide on an action, observe the result, and repeat until the task is complete. The LLM itself determines when to continue and when to stop.

This multistep loop is the core mechanism that enables agents to handle tasks of unpredictable complexity. A simple query might resolve in one step; a complex research task might require dozens of iterations across multiple tools.

Tool creation

At the highest level of autonomy, the agent doesn't just choose among available tools; it creates new ones. This typically involves generating code to implement functionality that doesn't exist in the predefined toolkit.

If, for example, an agent has to process data in a specific format that no existing tool handles, it might write a custom parsing function, execute it, and use the results to continue its task. This capability allows agents to extend their own abilities dynamically, adapting to requirements that weren't anticipated when the system was designed.

1.3.3 Combining workflows and agents in practice

The distinction between workflows and agents isn't always binary in production systems. In practice, the most effective architectures often combine both approaches, using workflows to provide structure and predictability, and embedding agents at specific points where flexibility is needed.

Agents as nodes in a workflow

A common pattern is to design an overall workflow with well-defined stages but implement one or more of those stages as an agent. Consider a document-processing pipeline:

1. *Document intake (workflow)*—Validate format and extract metadata.
2. *Content analysis (agent)*—Research context, gather related information, and synthesize findings.
3. *Quality review (workflow)*—Check against compliance rules and format output.
4. *Delivery (workflow)*—Route to the appropriate destination.

The agent operates freely within its designated stage, making multiple tool calls, following chains of inquiry, and adapting to what it discovers. But the overall process maintains the predictability of a workflow. If the agent fails or produces unexpected results, the workflow can catch them at the quality-review stage.

Why combine?

This hybrid approach offers several advantages:

- *It contains complexity.* Agent behavior is inherently less predictable, so limiting where agents operate makes the overall system easier to reason about and debug.

- *It optimizes costs.* Agent loops can be expensive due to multiple LLM calls, so using them only where their flexibility is genuinely needed keeps operational costs manageable.
- *It enables graceful degradation.* If an agent component fails, the workflow structure allows for fallback behaviors or human escalation at defined points.

The architectural choice isn't workflow versus agent but where in this system agent behavior provides enough value to justify its costs. This pragmatic framing will serve you well as you design your own LLM-powered applications.

1.4 Tasks that require agents

Before jumping into agent development, we have to ask the right questions in the right order. Two key decisions shape whether an agent is the appropriate solution:

- Does this task require an LLM?
- If an LLM is needed, does it require an agent, or will a workflow suffice?

Answering no at either stage means choosing a simpler, more cost-effective approach. Let's examine the criteria for each decision and then look at a benchmark that captures tasks at which agents truly excel.

1.4.1 Tasks that require an LLM

Before considering the use of an LLM agent, you must determine whether the task itself requires an LLM. This foundational question matters because LLMs introduce significant overhead in terms of computational cost and potential for errors. This overhead may be unnecessary if the task can be solved with simpler, more deterministic approaches. If your task involves basic data processing with predictable inputs and outputs, for example, traditional programming logic will be faster, cheaper, and more reliable than any LLM-based solution. You use two main criteria to make this decision:

- *Tasks involving unstructured data*—If a task requires analyzing unstructured data such as text, images, or audio, it's a strong candidate for LLM use. Traditional programming excels at analyzing structured data with clear schemas but struggles with the ambiguity inherent in natural language or visual content. Although models that handle diverse data formats are generally called *large multimodal models* (LMMs), they're fundamentally based on LLMs, so this book collectively refers to them as LLMs.
- *Input diversity*—If user input is limited or the requested tasks are narrowly scoped, using a specialized machine language (ML) model or rule-based system might be more cost effective. Similarly, if the number of potential tasks is small and predefined, it's more efficient in terms of both cost and latency for a developer to hardcode the logic than to use an LLM. LLMs shine when inputs are unpredictable and varied, requiring flexible interpretation that can't be anticipated.

1.4.2 Conditions for using agents

When you've determined that an LLM is necessary, the next question is how to use it. Will a single LLM call suffice, or does the task require the iterative reasoning and tool use that define an agent? Consider the following question:

> *If Eliud Kipchoge could maintain his marathon world record pace indefinitely, how long would it take him to run from Earth to the Moon?*

Answering this question requires multiple steps. First, you have to search to find out who Kipchoge is and what his marathon world record is. Next, you check Wikipedia for the distance between Earth and the Moon. Finally, you perform calculations to divide the distance by the speed and derive the final answer.

Anyone can perform this task manually. But navigating multiple websites to gather information, convert units, and perform calculations is time consuming and error prone. This kind of multistep research task is exactly the kind at which agents shine.

As excitement about agents grows, people fall into a common trap: applying agents to every task simply because they're new and powerful. But you don't need a complex web framework to build a simple static website. Choosing the right tool is key to building an efficient system.

Agents come with inherent tradeoffs:

- *Costs are higher.* Multiple LLM calls multiply API expenses compared with single requests.
- *Latency increases.* Response time accumulates with each reasoning step.
- *Errors propagate.* Mistakes in early steps cascade through the entire process.

Consider a customer service query that could be handled by a single LLM call that costs 1 cent. Processing the same query with an agent that requires 10 calls would cost 10 cents—a 10× difference. When you're handling thousands of requests daily, this decision isn't just technical but also business.

When should you use an agent? You can decide based on these three criteria:

- *Task complexity*—If it's difficult to predict how many steps a task will require, agents have the advantage. "Find the population of region A" is straightforward. By contrast, "Analyze how LLM agents will shape the future" requires gathering diverse information and synthesizing complex relationships. This task is hard to predefine with static logic.
- *Task value*—Because agents require multiple calls and dynamic decision-making, they cost more and respond more slowly. Therefore, the value of completing the task should outweigh the additional cost and latency.
- *Error cost and detectability*—LLMs are prone to make incorrect decisions, and the risk increases with more calls. If errors lead to critical consequences, an agent may not be the best choice. Detectability also matters. In domains that require specialized knowledge, users or developers may not even realize that an agent has produced faulty results.

1.4.3 GAIA: An agent gym

The Kipchoge question we examined earlier is from a benchmark called GAIA, a dataset that systematically collects tasks requiring agents. GAIA consists of question-and-answer pairs. Each question requires multistep reasoning, web searches, calculations, and more. The questions are difficult to solve with a single LLM call, and it's not easy to predefine a clear workflow, either. These problems naturally require an agentic approach.

Meta and Hugging Face released GAIA in 2023. Since then, model capabilities have improved significantly, with more challenging benchmarks emerging. Still, GAIA's problems range from straightforward to genuinely difficult even for current models, making it well suited to learn agent development. This book uses GAIA for three reasons:

- *Clear answers enable fast feedback.* The core of agent development is identifying and reducing failures. When answers are clear, you can verify immediately whether you're heading in the right direction. Then you can iterate rapidly through cycles of experimentation and improvement without using ambiguous evaluation criteria.
- *It's ideal for practicing the agent development cycle.* The process of building an agent goes like this:
  - Determine whether an agent would help solve a specific problem. If there's potential, develop a prototype.
  - Observe when the agent fails, analyze the cause, and improve it.
  - Repeat this observe-analyze-improve cycle. GAIA provides an appropriate level of difficulty for experiencing this cycle from start to finish.
- *No domain knowledge is required.* Most GAIA problems center on web search and information synthesis. We all perform tasks in daily life that involve searching, gathering information, and organizing it. You can intuitively understand the problems and the solution processes without specialized knowledge in medicine or law.

Throughout this book, we'll use GAIA to track changes in agent performance as we add new techniques. You'll see firsthand how each component, such as tool use, memory, and planning, improves the agent's problem-solving capabilities.

GAIA gives us a way to measure agent performance. But measurement alone doesn't improve an agent. What makes an agent better at solving these problems? The answer lies in how we engineer the context that the LLM receives.

1.5 Context engineering

Before diving into agent development, we'll clarify two terms that are often used interchangeably. A *prompt* is the input text that a user sends to an LLM. It can be further divided into *system prompts*, which specify how the model should respond and behave (such as "You are a friendly travel guide") and *user prompts* (or messages), which contain the user's actual requests.

Context is a broader concept that encompasses all the information an LLM references when generating a response. This information includes the system prompt, user

messages, conversation history, tool execution results, and retrieved documents. In this sense, prompts are one component of context.

Think of context as the LLM's working memory. Just as you can't solve a math problem if someone gives you only half the equation, an LLM can't complete a task if crucial information is missing from its context.

An LLM predicts the next token based solely on the information within its context window. The model generates text by using patterns learned during training to employ that information. This simple fact is why context engineering is so critical in agent development.

1.5.1 Why agents fail

Agent failures can be attributed to two main causes:

- *Insufficient model intelligence*—This includes cases in which the model fails to solve complex mathematical problems or makes errors in logical reasoning. Addressing this issue requires using a more powerful model or waiting for advances in model capabilities.
- *Lack of necessary information*—Even if a model is intelligent enough, it can't produce correct answers if the information needed to complete the task isn't in the context. If someone asks about the company's vacation policy and the company policy document isn't in the context, even the most capable model can't provide an accurate answer.

Interestingly, a significant portion of real-world agent failures stems from information deficiency. An LLM can be only as smart as the context it's given. This is why what information to include in the context is a key factor in determining agent performance.

1.5.2 From prompt engineering to context engineering

In the early days of LLM adoption, prompt engineering was the primary focus. Practitioners concentrated on how to write system prompts that would make models respond more accurately and on which instructions would yield outputs in the desired format. Techniques like "Let's think step by step" and "You are an expert" emerged during this period.

But the landscape changed as agents began using tools. When an agent performs a web search, the results are added to the context. When the agent executes code, the execution results are added. When the agent queries a database, the query results are added. Context began to change dynamically, evolving beyond static system prompts.

This shift raised a new question: "How do we maintain only the essential information needed for task completion in the context?" The approach that answers this question is context engineering (figure 1.5).

Context engineering is the discipline of providing the information an LLM needs to perform its tasks at the right time and in the right form. It goes beyond simply writing good prompts to comprehensively designing how tool-execution results should be processed, how much conversation history to retain, and when to retrieve external knowledge.

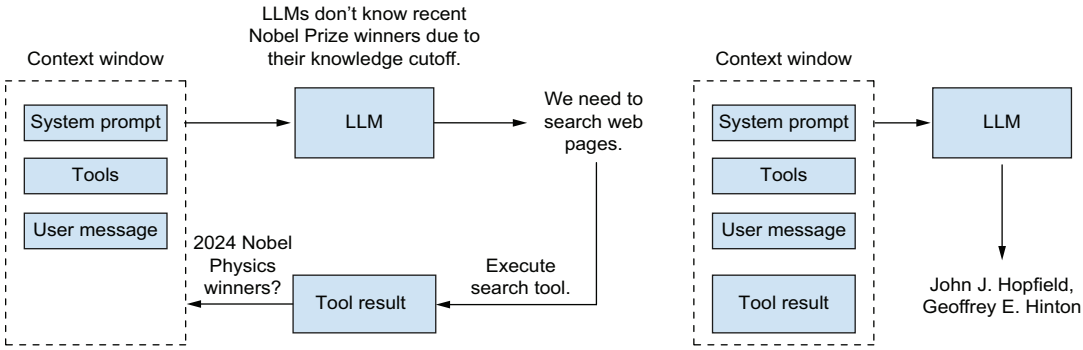


Figure 1.5 LLMs can produce accurate, high-quality responses only when sufficient information is provided in the context.

1.5.3 Bigger context isn't always better

The context windows of recent LLMs have grown dramatically. Most models now support context windows ranging from hundreds of thousands to a million tokens. Would filling the entire context window with all available data yield the best responses?

Unfortunately, no. Recent research has reported that model performance degrades as context length increases (figure 1.6). This phenomenon is sometimes called *context rot*.

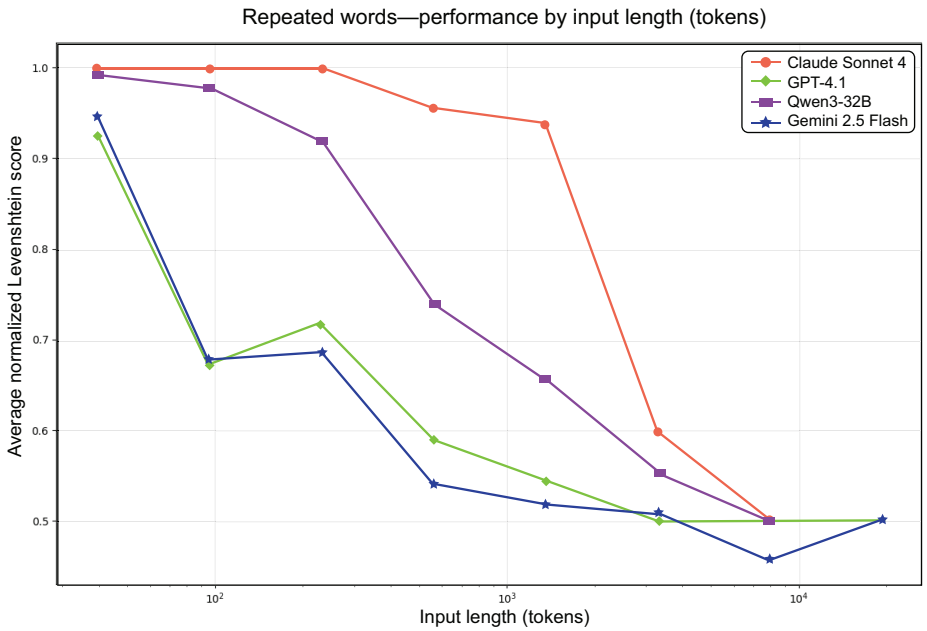


Figure 1.6 Even with large context windows, longer inputs can degrade model performance. (Source: https://research.trychroma.com/context-rot)

The "lost in the middle" effect is particularly well documented; in this effect, models tend to miss important information located in the middle of long contexts.

This provides an important insight: to maximize model performance, we should not simply provide more information but selectively provide only highly relevant information. When the context is filled with unnecessary information, crucial details may get buried, or the model's attention could be dispersed.

1.5.4 Five context-engineering strategies

Context engineering can be broadly categorized into five strategies:

- *Generate*—Using LLM-generated outputs within the context, which includes generating plans for complex tasks or reflecting on completed work to revise strategies. Beyond simply retrieving external information, this strategy lets the LLM structure and evolve the context on its own.
- *Retrieve*—Fetching necessary information from external sources and adding it to the context, which includes web searches, database queries, file reading, and similar document retrieval from vector databases. This core strategy enables agents to access up-to-date information or domain-specific knowledge that isn't present in their training data.
- *Write*—Recording information from the context to external storage, which includes saving important conversation content to long-term memory, recording intermediate calculation results to a scratch pad, and saving generated code to files. This strategy overcomes the limitations of the context window and enables persistent storage of information.
- *Reduce*—Shrinking the context, which includes summarizing old conversations, deleting unnecessary information, and filtering information based on importance. This strategy is essential for preventing context rot and focusing the model's attention on key information.
- *Isolate*—Separating tasks or tools into distinct environments, which includes executing complex code in sandbox environments and assigning different specialized domains to separate agents. This strategy prevents any single context from becoming overly complex and enables the construction of optimized contexts for each task.

1.5.5 From context engineering to harness engineering

Earlier, we watched attention move from prompt engineering to context engineering. As the field matured, that attention widened one step outward. The reason is that agents left the benchmark and arrived as real products. As more of them run directly on a user's computer, concrete demands on everything surrounding the model have come into focus. Here are a few examples:

- Users want behavior tuned to their own environment and projects, so an agent has to remember their context and learn the way they repeatedly work (memory and skills).
- As agents began executing code on a user's machine, isolating that execution safely became important (code execution).
- Work no longer stops at a single agent; running and coordinating several agents has become common (multi-agent).

Everything that supports an agent from outside the model is called the *harness*, and designing it is *harness engineering*. The harness is more than just components such as memory and tools. The user's working environment in which the agent runs—that is, the files, projects, and systems it can reach—is part of the harness too. In one line:

$$\text{Agent} = \text{Model} + \text{Harness}$$

Context engineering is part of this harness as well. A new term doesn't replace it; it takes its place within the larger picture surrounding the model.

One thing is worth making clear: harness engineering isn't entirely new. Attaching memory, executing tools safely, and wiring several agents together are problems that agent developers have worked on all along and ones that we will build piece by piece in this book. What changed is that agents have begun to weave into people's lives and work, and as they do, the question of what an agent must do well is taking an increasingly concrete shape. *Harness engineering* is the name given to those crystallizing demands.

1.6 Prerequisites for reading this book

This book covers the full process of implementing LLM agents from scratch using plain Python. To fully understand and follow the content and hands-on exercises in this book, readers should meet the following basic prerequisites:

- *Basic knowledge of Python*—All examples and exercises in this book are written in Python. Therefore, readers should be familiar with the following Python fundamentals: If you're unfamiliar with these topics, it's best to study a beginner-level Python guide beforehand.
- Basic data structures such as variables, data types, lists, and dictionaries
- Control flow structures such as `if` statements and `for`/`while` loops
- Defining and calling functions
- Basic object-oriented programming (OOP) concepts such as classes
- *Development-environment setup*—To complete the exercises effectively, a local Python development environment is necessary: API keys and external services: Cost considerations:
- Install the latest version of Python in your local environment (recommended: Python 3.12 or later).
- Prepare a code editor or IDE (recommended: Visual Studio Code).

- It's helpful to learn how to install Python packages using `pip` or `uv`.
- OpenAI API key for accessing GPT models (primary LLM used in examples)
- Tavily API key for web search functionality in research agents
- Basic understanding of REST API concepts and HTTP requests
- *OpenAI*—The total cost of completing all exercises should be less than $10.
- *Tavily Search API*—Free tier includes up to 1,000 searches per month, which is sufficient for all book exercises.
- We'll provide tips on cost-effective development and testing throughout the book.

When you've completed these preparations, you're ready to begin your journey into LLM agent development with this book.

Summary

- AI agents represent a wide spectrum, from personal assistants such as ChatGPT and Claude to customer-facing agents and specialized tools such as Claude Code and Cursor. These agents have a common foundation: LLMs are their decision-making core.
- An LLM agent consists of three elements: the LLM (brain), tools (means of interacting with the external world), and a loop (an iterative process that continues until goal completion). The LLM decides which tool to use and when to stop.
- Workflows are developer-defined execution flows in which LLMs perform specific steps. Agents are LLM-directed flows in which the model determines its own process dynamically. Production systems often combine both approaches.
- Use agents when tasks require multiple unpredictable steps, provide sufficient value to justify costs, and allow error detection. The GAIA benchmark provides ideal practice problems for agent development.
- Context engineering is the discipline of providing the right information at the right time. Five strategies (generate, retrieve, write, reduce, and isolate) form the framework for building effective agents throughout this book.

In this chapter, you'll put agent concepts into practice by building a travel information agent with LangGraph. You'll connect tools, manage conversation state, route execution through a graph, inspect the agent's messages and tool calls, and compare a custom implementation with a prebuilt ReAct agent.

Chapter 11 from *AI Agents and Applications* by Roberto Infante

Building tool-based agents with LangGraph

This chapter covers

- Building LLM-powered agents using LangGraph
- Registering and using tools for dynamic agent execution
- Debugging agent execution and tool calls
- Simplifying agents with prebuilt LangGraph components
- Observing agent execution with LangSmith

You learned that agentic workflows are fundamentally deterministic: their logic is based on flows with conditional paths that depend on the current application state. These workflows can be elegantly modeled using node-based graphs in LangGraph, and you saw a complete, hands-on example of such a system.

Agents, however, operate differently. Rather than following a predetermined flow, agents rely on dynamic, context-sensitive decision-making. With the help of a large language model (LLM), an agent chooses which tools to use—and in what order—based on the evolving context of the task at hand. These decisions aren't

pre-scripted; instead, they unfold step-by-step, as the agent continually evaluates the outputs of previous actions and adapts accordingly.

In this chapter, you'll put these ideas into practice by building a multi-tool travel information agent. You'll begin simply, implementing an agent that provides destination information using a single tool. From there, you'll extend it into a true multi-tool agent, able to answer questions about both travel destinations and their current weather conditions.

As we progress, I'll introduce you to the core concepts necessary for constructing multi-tool agents, with particular focus on the tool-calling protocol. You'll first implement this protocol from scratch to grasp every detail, and then you'll see how LangGraph's built-in capabilities can streamline and simplify your agent's architecture.

11.1 Starting simple: Building a single-tool travel info agent

In this section, we'll lay the groundwork for our agent-based applications by building a straightforward travel information agent. This first agent will use just one tool: a vector store retriever that answers questions about Cornwall's destinations and resorts, using content from Wikivoyage (www.wikivoyage.org). The content is split into chunks and stored in a vector store for efficient retrieval.

If you've learned about the advanced Retrieval-Augmented Generation (RAG), you should already be comfortable with sourcing content and populating a vector store. Here, we'll build on that foundation, keeping the focus on agent mechanics.

11.1.1 Project setup

Let's set up a new Python project using Visual Studio Code (VS Code). This works seamlessly with Cursor as well.

CREATING A VIRTUAL ENVIRONMENT AND INSTALLING DEPENDENCIES

First, set up your Python virtual environment and install all necessary dependencies. You'll find the `requirements.txt` file on the Manning website for this book or in the cloned GitHub repository.

Open a new PowerShell terminal in VS Code (choose Terminal > New Terminal), navigate to the `ch11` project folder, and then create and activate a new virtual environment:

```
PS C:\Github\building-llm-applications\ch11> python -m venv env_ch11
PS C:\Github\building-llm-applications\ch11> .\env_ch11\Scripts\activate
(env_ch11) PS C:\Github\building-llm-applications\ch11>
```

Once your environment is activated, install the required dependencies:

```
(env_ch11) PS C:\Github\building-llm-applications\ch11>
 pip install -r .\requirements.txt
```

Your project environment is now ready for development.

ADDING YOUR OPENAI API KEY

Create an `.env` file in your project root, and add your OpenAI API key:

```
OPENAI_API_KEY=<Your OPENAI_API_KEY>
```

CONFIGURING VS. CODE DEBUGGING

For smooth debugging, add the following `launch.json` to your `.vscode` directory:

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Python Debugger: Current File",
            "type": "debugpy",
            "request": "launch",
            "program": "${file}",
            "console": "integratedTerminal"
        }
    ]
}
```

Organize your implementation files by using the naming convention `main_x_y.py` for the scripts. The `x` represents the feature (e.g., `main_01_01.py` for the initial travel info agent, `main_02_01.py` when adding weather), while `y` is the version of that feature as we iterate. This will make it easy for you to compare versions and follow the progression of the implementation.

> **NOTE** The code in these examples is intentionally simplified to focus on core functionality. Error handling and defensive programming are omitted for clarity and learning purposes.

11.1.2 Loading environment variables

Once your `.env` file is ready, create a code file named `main_01_01.py`. You can load your API key at the top of the script, after the import statements (omitted here for brevity—refer to the GitHub repository for the full list), as shown in the following listing.

Listing 11.1 Loading environment variables

```
load_dotenv()
```

Loads environment variables from the .env file

11.1.3 Preparing the travel information vector store

To enable our travel information agent to answer queries about Cornwall destinations, we first need a way to store and efficiently retrieve relevant information. At a high level, we'll download travel content for a set of Cornwall destinations from Wikivoyage,

break the text into manageable chunks, embed those chunks into vector representations, and then store everything in a Chroma vector store. We'll also encapsulate the initialization logic in a singleton pattern to ensure that the vector store is only built once during the agent's lifetime. The following code sets up this vector store, making it easy to retrieve relevant travel information as the agent operates.

Listing 11.2 Preparing the travel information vector store

```
UK_DESTINATIONS = [
    "Cornwall",
    "North_Cornwall",
    "South_Cornwall",
    "West_Cornwall",
]

async def build_vectorstore(
    destinations: Sequence[str]) -> Chroma:
    """Download Wikivoyage pages and create
    a Chroma vector store."""
    urls = [f"https://en.wikivoyage.org/wiki/{slug}"
 …  for slug in destinations]
    loader = AsyncHtmlLoader(urls)
    print("Downloading destination pages ...")
    docs = await loader.aload()

    splitter = RecursiveCharacterTextSplitter(
 …  chunk_size=1024, chunk_overlap=128)
    chunks = sum([splitter.split_documents([d])
 …  for d in docs], [])

    print(f"Embedding {len(chunks)} chunks ...")
    vectordb_client = Chroma.from_documents(
 …  chunks, embedding=OpenAIEmbeddings())
    print("Vector store ready.\n")
    return vectordb_client

_ti_vectorstore_client: Chroma | None = None

def get_travel_info_vectorstore() -> Chroma:
    global _ti_vectorstore_client
    if _ti_vectorstore_client is None:
 …  if not os.environ.get(
 … …"OPENAI_API_KEY"):
 … …raise RuntimeError(
 … …    """Set the OPENAI_API_KEY env
 … …    variable and re-run.""")
 …  _ti_vectorstore_client = asyncio.run(
 … …build_vectorstore(UK_DESTINATIONS))
    return _ti_vectorstore_client

ti_vectorstore_client
➥= get_travel_info_vectorstore()
ti_retriever = ti_vectorstore_client.as_retriever()
```

List of target Cornwall destinations—expand as needed

Asynchronous function to build and return the vector store

Downloads Wikivoyage content for each destination

Splits downloaded documents into manageable chunks

Embeds each chunk and stores it in the Chroma vector store

Returns the initialized vector store

Singleton cache for the vector store client

Function to initialize/retrieve the cached vector store

Returns the cached vector store client instance

Creates the vector store client

Creates a retriever from the vector store

This setup starts by defining a list of Cornwall-related destinations that serve as the basis for information retrieval. The `build_vectorstore()` asynchronous function constructs URLs for each destination and uses an asynchronous loader to fetch the corresponding Wikivoyage pages. Once the pages are downloaded, the text is split into overlapping chunks to ensure that information remains contextually meaningful. These chunks are then embedded using OpenAI's embedding models and stored in a Chroma vector store, making them quickly searchable by semantic similarity.

To prevent unnecessary recomputation and data downloads, the singleton pattern is used for the vector store client. The `get_travel_info_vectorstore()` function ensures that the vector store is only built once and is reused for all future retrievals. At the end, the vector store client is instantiated, and a retriever object is created from it—this retriever will be the agent's interface for accessing Cornwall travel information. This foundation allows the agent to efficiently answer user queries about destinations using up-to-date knowledge sourced directly from Wikivoyage.

11.2 Enabling agents to call tools

Now that we have our vector store retriever ready, the next step is to expose this retrieval capability as a tool the agent can use. This is where the concept of tool calling—a major advance in modern agent frameworks—enters the picture.

11.2.1 From function calling to tool calling

LLMs like those from OpenAI initially introduced *function calling*—a mechanism that allows the model to request specific functions, passing structured arguments, based on the needs of a given prompt. This concept quickly evolved into the more general *tool-calling* protocol, now widely supported by major LLM providers. With tool calling, models can invoke not just custom functions but also a variety of built-in or external "tools," handling everything from code execution to external API lookups.

> **DEFINITION** A *tool* is any external function, service, or capability that an LLM can call through the tool-calling protocol. Tools extend the model's abilities beyond text generation—for example, running code, querying a database, or calling an API—by letting the model pass structured inputs and receive structured outputs.

You might wonder why tool calling matters. Why not just let agents access functionality directly through standard REST API endpoints?

The key reason is flexibility. We turn to agents—rather than fixed agentic workflows—when we can't predict in advance which tools will be needed, in what sequence they should be called, or how a user's question will map to tool parameters. In these cases, hardcoding API calls isn't enough.

An agent's strength lies in making those decisions dynamically: selecting the right tool, determining the correct order of calls, and shaping inputs to match tool parameters. Modern LLMs are explicitly trained for this purpose, with tool calling built into their response protocols (e.g., the OpenAI Responses API). In short, tool calling unlocks the adaptive reasoning that makes multi-tool agents possible.

This evolution has made agent implementations dramatically simpler and more robust, particularly when using the *ReAct* (Reasoning and Acting) design pattern. The ReAct pattern, introduced by Yao et al. in their 2022 paper "ReAct: Synergizing Reasoning and Acting in Language Models" (https://arxiv.org/abs/2210.03629) enables LLMs to interleave reasoning steps ("thoughts") with tool invocations ("actions"). In effect, the model can break a task into smaller pieces, use tools where appropriate, and synthesize an answer step-by-step. Figure 11.1 shows a simplified view of the ReAct pattern as a process diagram.

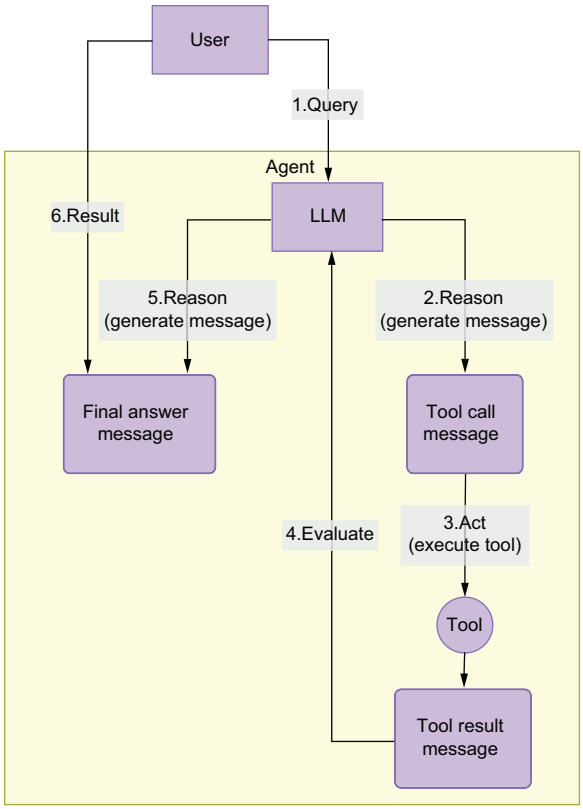


Figure 11.1 The ReAct pattern alternates between reasoning and action, enabling the agent to process a user question by thinking, calling tools as needed, and following up with further reasoning before delivering the final answer.

The ReAct pattern starts with a user question, as shown in the diagram. The agent first enters the Reasoning phase, where the LLM analyzes the input and decides on the next steps. If further information or actions are required, the agent moves into the Act phase, calling one or more tools—such as running a semantic search or invoking an API.

Once the tool returns results, the agent cycles back into Reasoning, incorporating the new information into its thought process. This may trigger additional tool calls, or the agent may determine that it has enough context to answer. The process concludes when the agent produces a Final Answer for the user.

This stepwise interplay between reasoning and acting embodies the core of the ReAct pattern, allowing agents to dynamically combine thought and action as they work toward solving the user's query.

> **NOTE** With the introduction of the OpenAI Responses API, the transition from function calling to tool calling has accelerated. The Responses API is designed specifically for structured tool use and has largely superseded the older Completion API for agentic applications.

11.2.2 How tool calling works with LLMs

OpenAI's tool calling supports several types of tools—including user-defined functions, code interpreters, and native capabilities such as web browsing. At a high level, when you register a tool with the LLM, you expose both the function signature (name and parameters) and a textual description. The model then decides, at runtime, when and how to use each tool based on the user's input and the context of the conversation.

In LangGraph, tools can be registered using either a class-based or a decorator-based approach. We'll use the decorator-based approach for clarity and conciseness. Let's implement our semantic search tool as a function decorated with `@tool`, making it available to the agent for tool calling, as shown in the following listing.

Listing 11.3 LangGraph attribute-based tool definition

```
@tool
def search_travel_info(query: str) -> str:
    """Search embedded Wikivoyage content for
    information about destinations in England."""
    docs = ti_retriever.invoke(query)
     top = docs[:4] if isinstance(docs, list) else docs
    return "\n---\n".join(
    d.page_content for d in top)
```

Defines the tool using the @tool decorator

Defines the tool function, which takes a query, performs a semantic search, and returns a string response from the vector store

Performs a semantic search on the vector store and returns the top four results

Joins the top four results into a single string

This decorated function, `search_travel_info()`, is now recognized as a tool: it takes a user query, searches the vector store for relevant Wikivoyage content, and returns up to four top results as a single string. The `@tool` decorator ensures that the function's name, description, and parameter schema are all available to the LLM for tool calling.

11.2.3 Registering tools with the LLM

To enable the agent to use our semantic search tool, we must register it with the LLM so that the model knows both the function's signature and its intended purpose. In LangChain, this is achieved using the `bind_tools` protocol, but it's instructive to see how this works at the OpenAI API level first.

With the introduction of the OpenAI Responses API, the way tools and functions are registered has become more standardized and explicit. Now you provide a structured

definition of your tool—specifying its name, description, and parameter schema. The model can then return responses that explicitly request tool invocations, passing arguments as needed. For further details, you can consult the OpenAI function calling documentation (https://mng.bz/Bzag).

EXAMPLE: MANUAL TOOL REGISTRATION WITH OPENAI API

Suppose you wanted to expose our `search_travel_info` function directly to the OpenAI API (without using LangChain). You would define the tool's schema as shown in listing 11.4. Note that this example is for illustration only and isn't included in the book's provided source code.

Listing 11.4 Manual tool registration with the OpenAI API

```
search_travel_info_tool = {
    "type": "function",
    "function": {
        "name": "search_travel_info",
        "description": "Search embedded Wikivoyage content for information
        about destinations in England.",
        "parameters": {
            "type": "object",
            "properties": {
                "query": {
                    "type": "string",
                    "description": "A natural language
                    ➥ query about a destination in
                    ➥ England."
                }
            },
            "required": ["query"]
        }
    }
}

response = client.chat.completions.create(
    model="gpt-5",
    messages=[
        {"role": "user", "content": "Tell me about surfing in
        ➥ Cornwall."}
    ],
    tools=[search_travel_info_tool],
    tool_choice="auto",
)
```

Specifies that this object is a function tool (as per OpenAI's tool-calling schema)

Name of the tool/function being registered

Human-readable description of what the tool does

The parameters accepted by the function are described in this dictionary.

The parameters are passed as an object (i.e., a dictionary of named arguments).

The function accepts a single parameter named "query."

Description of what the "query" parameter should contain

"query" is a required argument for the tool.

Specifies the OpenAI model to use for the completion

Provides the initial user message to the chat (the question to answer)

Registers the tool so the model knows it can call it if needed

Allows the model to automatically decide whether and when to use the tool

In this example, you explicitly define the tool's metadata and parameters and then pass it in the `tools` argument of your API call. When the model decides it needs to call `search_travel_info`, it will return a structured tool call in its response. Your application must then handle the invocation of the Python function, passing the model-generated arguments, and send the results back to the LLM if the conversation continues.

REGISTERING TOOLS IN LANGCHAIN

LangChain automates much of this process. You simply define your tool as a decorated Python function (as you saw earlier in listing 11.3) and then register it with the LLM. You can see how this looks in code in the following listing.

Listing 11.5 Registering tools in LangChain

```
TOOLS = [search_travel_info]

llm_model = ChatOpenAI(
    model="gpt-5-mini",
    use_responses_api=True)
llm_with_tools = llm_model.bind_tools(TOOLS)
```

Defines the tools list (in our case, only one tool)

Instantiates the LLM model with the GPT-5-mini model and the responses API

Binds the tools to the LLM model, which will generate a response with the tool calls

Here, we list the available tools (currently just `search_travel_info`), instantiate the `gpt-5-mini` chat model (with Responses API support), and use `.bind_tools(TOOLS)` to expose those tools to the model for tool calling.

LangChain handles the translation between your Python code and the OpenAI function/tool-calling protocol, including automatically generating the appropriate JSON schema from your function signature and docstring. This setup ensures that whenever the model receives a user message, it can autonomously decide if and when to call any registered tool, structuring its responses according to the tool-calling protocol.

Note If you're using a model other than OpenAI, or you prefer not to use the Responses API, tool calling can still work—though the capabilities and structure of the responses may differ. The older Completion API is still available, but for most agentic use cases, the newer Responses API is recommended for its clarity, power, and alignment with evolving best practices.

11.2.4 Agent state: Tracking the conversation

In our implementation, the agent's state is simply a collection of LLM messages that track the entire conversation. This is defined as follows:

```
class AgentState(TypedDict):
    messages: Annotated[Sequence[BaseMessage], operator.add]
```

Here, `AgentState` only contains the sequence of messages exchanged between the user, the agent, and any tool responses. This keeps the design simple.

11.2.5 Executing tool calls

The core of the tool execution logic is implemented in a node that examines the LLM's most recent message, extracts any tool calls requested by the model, and invokes the corresponding functions with the provided arguments. Each tool's output is then wrapped in a message and appended to the conversation state. A simplified implementation is shown in the following listing.

Listing 11.6 Tool execution implemented from scratch

```
class ToolsExecutionNode:
    """Execute tools requested by the LLM in the last AIMessage."""

    def __init__(self, tools: Sequence):
        self._tools_by_name = {t.name: t for t in tools}

    def __call__(self, state: dict):
        messages: Sequence[BaseMessage] = state.get("messages", [])

        last_msg = messages[-1]
        tool_messages: list[ToolMessage] = []
        tool_calls = getattr(last_msg,
            "tool_calls", [])

        for tool_call in tool_calls:
            tool_name = tool_call["name"]
            tool_args = tool_call["args"]
            tool = self._tools_by_name[tool_name]
            result = tool.invoke(tool_args)
            tool_messages.append(
                ToolMessage(
                    content=json.dumps(result),
                    name=tool_name,
                    tool_call_id=tool_call["id"],
                )
            )
        return {"messages": tool_messages}

tools_execution_node = ToolsExecutionNode(TOOLS)
```

Defines the tools execution node

Initializes the tools execution node with the tools list

Defines the __call__ method, which is called when the node is invoked

Gets the last message from the messages list

Initializes the tool messages list to gather the results of the tool calls

Gets the tool calls from the last message

Iterates over the tool calls

Gets the tool name from the tool call

Gets the tool arguments from the tool call

Gets the tool from the tools list

Invokes the tool with the arguments

Adds the tool result to the tool messages list

Returns the tool messages list, which contains the results of the tool calls

Instantiates the tools execution node to be used as a node in the LangGraph graph

This pattern allows the agent to handle multiple tool calls in a single step. The `ToolsExecutionNode` inspects the LLM's latest response, invokes each requested tool by name, collects the results, and formats them for the next stage in the conversation. In a typical agent workflow, these tool results are sent back to the LLM, which incorporates the new information to either reason further or generate a direct answer to the user's query.

The built-in ToolNode class

You usually won't need to write this logic yourself in practice because LangGraph provides a built-in class, `ToolNode`, which performs the same function as our custom `ToolsExecutionNode`. For most applications, you can use

```
tools_execution_node = ToolNode(TOOLS)
```

This saves you from having to implement the tools execution logic manually, streamlining your agent development process.

Now that you understand how tool execution is managed, let's explore how the agent, guided by the LLM, determines which tools to call—or whether it already has enough information to generate the final answer.

11.2.6 The LLM node: Coordinating reasoning and action

Next, we add an LLM node to our LangGraph workflow, as shown in the following listing.

Listing 11.7 LLM node

```
def llm_node(state: AgentState):
    """LLM node that decides whether
    to call the search tool."""
    current_messages = state["messages"]
    response_message = llm_with_tools.invoke(
…   current_messages)

    return {"messages": [response_message]}
```

Defines the LLM node

Gets the current messages from the agent state

Invokes the LLM model with the current messages. The LLM will decide whether to call the search tool or return an answer.

Returns the response message, which contains the tool call or the answer

This node takes the agent state (a list of messages) and forwards them to the LLM for processing. Because we're using an LLM with tool calling enabled (`llm_with_tools`), the model automatically understands when to issue tool calls and when to produce a final answer:

- If the last message is a user question, the LLM can decide to request a tool call (or multiple calls) to retrieve relevant information.
- If the last message(s) are tool results, the LLM integrates those responses, reasons further, and may either produce a final answer or request additional tool calls if needed.

The model returns either an `AIMessage` with a final answer in the content field, or additional tool calls in the `tool_calls` field, depending on the situation. This flexible, reactive loop is at the heart of all modern agent implementations, and it's what makes today's LLM-based agents so capable.

11.3 Assembling the agent graph

With our LLM node and tool execution node ready, the next step is to assemble them into a working agent graph. In LangGraph, an agent is structured as a directed graph, where each node represents a component (e.g., an LLM or a tool executor), and edges represent the possible flow of data and control between these nodes.

The code in listing 11.8 demonstrates how we assemble our single-tool travel information agent as a graph. Each node in the graph corresponds to either the LLM or the tool execution logic.

Listing 11.8 Graph of the tool-based agent

```
builder = StateGraph(AgentState)
builder.add_node("llm_node", llm_node)
builder.add_node("tools", tools_execution_node)

builder.add_conditional_edges("llm_node",
    tools_condition)

builder.add_edge("tools", "llm_node")

builder.set_entry_point("llm_node")
travel_info_agent = builder.compile()
```

Defines the graph builder

Adds the LLM node and the tools node to the graph

Adds the conditional edges to the graph to decide whether to execute the tool calls or return an answer and exit the graph

Adds the edge from the tools node to the LLM node

Sets the entry point to the LLM node

Compiles the graph

11.4 Understanding the agent graph structure

Our agent graph consists of two main nodes, as shown in figure 11.2: the LLM node, responsible for reasoning and generating tool calls, and the tools node, responsible for executing the requested tools and returning results. The flow of the conversation alternates between these two nodes, reflecting the ReAct pattern described earlier in this chapter.

A critical aspect of this setup is the conditional edge that connects the LLM node to the next step. This is controlled by the `tools_condition` function—a prebuilt utility in LangGraph. This function examines the latest message from the LLM:

- If the message contains the `tool_calls` property (meaning the LLM is requesting one or more tool invocations), the graph routes the flow to the node named `"tools"`.
- If there are no tool calls present, the flow is directed to the `END` node, terminating the graph and producing a final answer for the user.

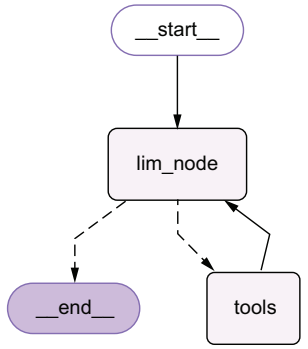


Figure 11.2 Conditional graph logic routes each user query through the LLM node and then dynamically directs flow to the tools node or ends the process, depending on whether tool calls are required.

This mechanism lets the agent dynamically decide at each turn whether to continue reasoning, take action, or conclude the interaction.

We explicitly set the entry point of the graph to `"llm_node"`, ensuring that each user question is first processed by the language model. (As an alternative, you could achieve the same effect by adding an edge from the `START` node to `"llm_node"` with `graph_builder.add_edge(START, "llm_node")`.) By compiling the graph with `builder.compile()`, we finalize our travel information agent, ready to receive queries and intelligently use its tool to find relevant travel information.

11.5 Running the agent chatbot: The Read-Eval-Print Loop

The final step in building our agent-powered chatbot is to implement the user interface—a simple loop that continuously accepts user questions and returns answers until the user chooses to exit. This classic pattern, known as a Read-Eval-Print Loop (REPL), is the main bridge between the user and your travel information agent.

At a high level, the chat loop listens for input, wraps the user's question in a message structure, invokes the agent graph, and prints the assistant's reply. This interaction continues indefinitely, enabling a true conversational experience. The following listing shows how you can implement this chat loop in Python.

Listing 11.9 Chatbot REPL

```
def chat_loop():
    print("UK Travel Assistant (type 'exit' to quit)")
    while True:
        user_input = input(
            "You: ").strip()
        if user_input.lower() in \
            {"exit", "quit"}:
                break
        state = {"messages":
            [HumanMessage(content=user_input)]}
        result = travel_info_agent.invoke(
            state)
        response_msg = result["messages"][-1]
        print(
           f"Assistant: {response_msg.content}\n")

if __name__ == "__main__":
    chat_loop()
```

Defines the chat loop

Gets the user input

Checks if the user input is "exit" or "quit" to exit the loop

Creates the initial state with a HumanMessage containing the user input

Invokes the graph with the initial state

Gets the last message from the result, which contains the final answer

Prints the assistant's final answer from the content of the preceding message

This loop welcomes the user, waits for their question, and continues processing until the user types `exit` or `quit`. Each input is packaged as a `HumanMessage` and passed to the travel information agent you just built. The agent's reply is extracted from the graph's output and displayed back to the user.

With this in place, you're now ready to run your first agent-based chatbot. Try asking it about destinations or activities in Cornwall, and experience how the agent reasons, retrieves information, and converses, all within the framework you've just constructed, as we're about to see.

11.6 Executing a request

Now that your travel information agent is ready, let's step through the agent in action by running and debugging your implementation. This hands-on walkthrough will show you how the agent orchestrates the flow between the LLM and the tool, helping you see the LangGraph framework in motion.

11.6.1 Step-by-step debugging

Begin by opening your `main_01_01.py` file and running it in debug mode, using the Python debug configuration you set up in your `launch.json` earlier. To trace the agent's flow, place breakpoints at the beginning of the following functions:

- `search_travel_info()`
- `ToolsExecutionNode.__call__()`
- `llm_node()`

Ready? Press F5 (or click the Play icon in your IDE), and let's walk through the agent's workflow together.

VECTOR STORE CREATION

When you start the script, you'll see the vector store being created and populated with travel information. In your debug console, you'll see the output similar to that in figure 11.3.

```
USER_AGENT environment variable not set, consider setting it to identify your requests.
Downloading destination pages ...
Fetching pages: 100%|#############################################################################
#########| 4/4 [00:01<00:00,  3.96it/s]
Embedding 1118 chunks ...
Vector store ready.

UK Travel Assistant (type 'exit' to quit)
```

Figure 11.3 **Output at startup during vector store creation**

CHATBOT LOOP LAUNCH

Next, the chatbot loop starts up, waiting for your input:

```
UK Travel Assistant (type 'exit' to quit)
You:
```

Enter your question, and press Enter:

```
You: Suggest three towns with a nice beach in Cornwall
```

LLM NODE ACTIVATION

Your breakpoint inside `llm_node()` will trigger. Inspect `state["messages"]`:

```
HumanMessage (content='Suggest three towns with a nice beach in Cornwall',
     additional_kwargs={}, response_metadata={})
```

Step over (press F10) to the next line to send this message to the LLM. Because the LLM is configured for tool calling, examine the resulting `response_message`:

```
   AIMessage(content=[],
...
tool_calls=[
```

```
{'name': 'search_travel_info', 'args': {'query': 'beach towns in Cornwall'},
     'id': 'call_L4PwmeyLkrkYX2PfC2gY6ri6', 'type': 'tool_call'},
{'name': 'search_travel_info', 'args': {'query': 'best beaches in Cornwall'},
     'id': 'call_XPNctNtyIKVetJa3ruM9z7d5', 'type': 'tool_call'},
{'name': 'search_travel_info', 'args': {'query': 'top seaside towns
     Cornwall'}, 'id': 'call_RAWMLwdFVALuIbJxWKfta5yp', 'type': 'tool_
     call'}],
...)
```

Here, the LLM has generated three tool calls—each targeting your semantic search tool with a slightly different query. Notice how the model rewrites the queries to maximize information coverage. You don't need to manually handle query rewriting—the LLM handles it.

TOOLS EXECUTION NODE

Continue execution (press F5). The list of tool calls is added to the message list. The conditional edge's `tools_condition` detects tool calls, routing execution to `ToolsExecutionNode.__call__()`. Your breakpoint will trigger there.

Next, inspect `state["messages"]`:

```
   [
HumanMessage(content='Suggest three towns with a nice beach in Cornwall',
     additional_kwargs={}, response_metadata={}),
AIMessage(content=[], ... ,
tool_calls=[
{'name': 'search_travel_info', 'args': {'query': 'beach towns in Cornwall'},
     'id': 'call_L4PwmeyLkrkYX2PfC2gY6ri6', 'type': 'tool_call'},
{'name': 'search_travel_info', 'args': {'query': 'best beaches in Cornwall'},
     'id': 'call_XPNctNtyIKVetJa3ruM9z7d5', 'type': 'tool_call'},
{'name': 'search_travel_info', 'args': {'query': 'top seaside towns
     Cornwall'}, 'id': 'call_RAWMLwdFVALuIbJxWKfta5yp', 'type': 'tool_call'}]
     , ...)
]
```

The last message (from the LLM) has no content (the LLM hasn't answered yet because it needs more information), but it contains the tool calls that must be executed. The node extracts these, then iterates through each one, extracting the tool name and arguments, and invokes the corresponding tool:

```
result = tool.invoke(tool_args)
```

Step through this to watch `search_travel_info()` execute. Each semantic search result (a document returned from the vector store) is collected as a `ToolMessage` and added to the list for the next LLM step.

TOOL CALL RESULTS PASSED BACK TO LLM

Continue (press F5), and you'll return to `llm_node()`. Now `state["messages"]` contains your original question, the LLM's tool call instructions, and the results from each tool execution:

```
\[HumanMessage(content='Suggest three towns with a nice beach in
Cornwall', additional\_kwargs={}, response\_metadata={}),
```

```
AIMessage(content=\[], ...
tool\_calls=\[{'name': 'search\_travel\_info', 'args': {'query': 'beach
towns in Cornwall'}, ...}, ...]),
ToolMessage(content='...', name='search\_travel\_info',
tool\_call\_id='call\_L4PwmeyLkrkYX2PfC2gY6ri6'),
ToolMessage(content='...', name='search\_travel\_info',
tool\_call\_id='call\_XPNctNtyIKVetJa3ruM9z7d5'),
ToolMessage(content='...', name='search\_travel\_info',
tool\_call\_id='call\_RAWMLwdFVALuIbJxWKfta5yp')]
```

The content of each tool message gives the LLM the facts it needs to synthesize a final answer. Step over the following line:

```
response\_message = llm\_with\_tools.invoke(current\_messages)
```

Inspect `response_message`:

```
AIMessage(content=\[{'type': 'text', 'text': "Three towns in Cornwall with
     nice beaches are:\n\n1. Newquay - Known as the UK's surfing capital with
     popular beaches like Fistral Beach.\n2. St Ives - A picturesque town
     with beautiful sandy beaches.\n3. Falmouth - Located on the south coast
     with beaches like Gyllyngvase beach.\n\nThese towns offer great beach
     experiences along with other attractions.", 'annotations': \[]}], ...)
```

Now the `content` field is populated with the LLM's answer. The `tool_calls` field is gone—the LLM no longer needs external tools and has synthesized a response.

COMPLETING THE REQUEST

As execution leaves `llm_node()`, the `tools_condition` on the conditional edge checks for further tool calls. Finding none, it ends the conversation. When your main invocation returns

```
result = travel\_info\_agent.invoke(state)
```

the `result.messages` list concludes with the final `AIMessage` containing the answer:

```
{'messages': [
HumanMessage(content='Suggest three towns with a nice beach in Cornwall',
  ...),
AIMessage(content=[],  tool_calls=[...]),
ToolMessage(...), ToolMessage(...), ToolMessage(...),
AIMessage(content=[{'type': 'text', 'text': "Three towns in Cornwall with
  nice beaches are:\n\n1. Newquay - Known as the UK's surfing capital with
  popular beaches like Fistral Beach.\n2. St Ives - A picturesque town
  with beautiful sandy beaches.\n3. Falmouth - Located on the south coast
  with beaches like Gyllyngvase beach.\n\nThese towns offer great beach
  experiences along with other attractions.", 'annotations': []}], ...)
]
```

the chatbot will now display the answer and prompt for your next question:

```
UK Travel Assistant (type 'exit' to quit)
You: Suggest three towns with a nice beach in Cornwall
Assistant: [{'type': 'text', 'text': "Three towns in Cornwall with nice
beaches are:\n\n1. Newquay - Known as the UK's surfing capital with popular
```

```
beaches like Fistral Beach.\n2. St Ives - A picturesque town with beautiful
sandy beaches.\n3. Falmouth - Located on the south coast with beaches like
Gyllyngvase beach.\n\nThese towns offer great beach experiences along with
other attractions.", 'annotations': []}]

You:
```

You've now observed, step-by-step, how your agent processes a user request: interpreting the question, generating tool calls, executing them, and synthesizing a final, well-informed answer. This debug-driven walkthrough offers deep insight into the mechanics of agentic reasoning and action in LangGraph.

11.7 Expanding your agent: Adding a weather forecast tool

So far, our agent has been able to answer travel-related queries using semantic search over curated travel content. But real-world travel advice often depends on dynamic, real-time information—like the weather! In this section, you'll learn how to extend your agent by adding a second tool, enabling it to respond with context-aware answers based on current conditions.

11.7.1 Implementing a mock weather service

To illustrate the process, start by copying your `main_01_01.py` file to a new script called `main_02_01.py`. This will help you track each evolutionary step in your agent's development.

First, let's introduce a mock weather service. This service will simulate real-time weather data for any given town, returning both a weather condition (e.g., sunny or rainy) and a temperature. You can see the implementation of the `WeatherForecastService` in the following listing.

Listing 11.10 `WeatherForecastService`

```
class WeatherForecast(TypedDict):
    town: str
    weather: Literal["sunny", "foggy", "rainy", "windy"]
    temperature: int

class WeatherForecastService:
    _weather_options = ["sunny", "foggy", "rainy", "windy"]
    _temp_min = 18
    _temp_max = 31

    @classmethod
    def get_forecast(cls, town: str) \
        -> Optional[WeatherForecast]:

        weather = random.choice(cls._weather_options)
        temperature = random.randint(cls._temp_min, cls._temp_max)
        return WeatherForecast(town=town,
             weather=weather,
             temperature=temperature)
```

Defines the get_forecast method, which returns a WeatherForecast object

This mock service chooses a random weather condition and temperature within a typical summer range for Cornwall. Later, you can swap this out for a real-world weather API if desired.

11.7.2 Creating the weather forecast tool

With the mock service in place, the next step is to create a tool that wraps it, making weather data available to the agent. Here's how you can define the tool function and add it to your agent's toolkit:

```
@tool
def weather_forecast(town: str) -> dict:
    """Get a mock weather forecast for a given town. Returns a
     WeatherForecast object with weather and temperature."""
    forecast = WeatherForecastService.get_forecast(town)
    if forecast is None:
        return {"error": f"No weather data available for '{town}'."}
    return forecast
```

The `weather_forecast` tool provides a mock weather forecast for a given town. When called with a town name, it returns a dictionary containing the weather conditions (e.g., sunny, foggy, rainy, or windy) and temperature for that location. If no data is available, it returns an error message instead. This tool allows the agent to incorporate simulated real-time weather information into its responses.

11.7.3 Updating the agent for multi-tool support

Finally, make sure your LLM model is set up to use tool calling and recognizes both available tools:

```
TOOLS = [search_travel_info, weather_forecast]
```

Defines the tools list (in our case, search_travel_info and weather_forecast)

```
llm_model = ChatOpenAI(
    model="gpt-5-mini",
    use_responses_api=True
)
```

Instantiates the LLM model with the GPT-5-mini model and enables the Responses API for tool calling

```
llm_with_tools = llm_model.bind_tools(TOOLS)
```

Binds the tools to the LLM model, enabling tool-calling support

With this setup, your agent is now equipped to handle both travel queries and weather checks, giving it the foundation to provide much more accurate and useful responses. In the next sections, you'll see how to guide the LLM to use both tools effectively, and you'll observe the agent's new capabilities in action. This workflow not only shows how to extend your agent's toolset but also demonstrates the modular, incremental approach that makes agentic systems with LangGraph and LangChain so powerful.

11.8 Executing the multi-tool agent

With your weather forecast tool registered, your agent can now synthesize answers that combine travel information and real-time weather conditions. The beauty of this

modular approach is that adding a new tool requires no changes to your agent's graph structure. All orchestration is handled by the LLM and the tool-calling protocol.

11.8.1 Running the multi-tool agent (initial behavior)

Let's put the new capabilities to the test. Set the same breakpoints as before—especially in the tool execution and LLM nodes—and add a breakpoint at the start of `weather_forecast()`. Run `main_02_01.py` in debug mode (press F5), and enter the following prompt:

```
You: Suggest two Cornwall beach towns with nice weather
```

After submitting your query, your first breakpoint will hit in `llm_node()`. Step through to the last line, and inspect the value of `response_message.tool_calls`:

```
[{'name': 'weather_forecast', 'args': {'town': 'Newquay'}, 'id':
'call_3nIgMLIFgMZBvVvrTHbRh2lj', 'type': 'tool_call'},
{'name': 'weather_forecast', 'args': {'town': 'Falmouth'}, 'id':
'call_Qfv5ENGOXhEUUcAAEamDyoQU', 'type': 'tool_call'}]
```

Here, the LLM has simply picked two towns it "knows" (Newquay and Falmouth) and asked for their weather (you might get a different combination of towns), without consulting your semantic search tool at all. This is typical of a model using its internal knowledge base rather than the tools provided—something we want to avoid for reliability and accuracy.

Why does this happen? The LLM is acting on its pretraining and defaulting to what it already "knows" about Cornwall, rather than querying your up-to-date data.

11.8.2 Improving LLM tool usage with system guidance

To nudge the LLM toward tool use and away from hallucinations, let's make its instructions and tool descriptions clearer. Make a copy of your script as `main_02_02.py`, and update the tool definitions by adding a description of each tool:

```
@tool(description="""Search travel information
about destinations in England.""")
def search_travel_info(query: str) -> str:
    ...

@tool(description="Get the weather forecast, given a town name.")
def weather_forecast(town: str) -> dict:
    ...
```

Next, introduce a guiding `SystemMessage` in your `llm_node()`:

```
system_message = SystemMessage(content="""You are a helpful assistant
    that can search travel information and get the weather forecast.
    Only use the tools to find the information
    you need (including town names).""")
current_messages.append(system_message)
```

You can see the amended `llm_node()` function in the following listing.

Listing 11.11 Guiding tool selection

```
def llm_node(state: AgentState):
    """LLM node that decides whether to call the search tool."""
    current_messages = state["messages"]
    system_message = SystemMessage(content="""You are a helpful assistant
    that can search travel information and get the weather forecast.
    Only use the tools to find the information
    you need (including town names).""")

    current_messages.append(system_message)
    response_message = llm_with_tools.invoke(
        current_messages)

    return {"messages": [response_message]}
```

Defines the LLM node

Gets the current messages from the agent state

Adds a system message to the current messages to set the behavior of the assistant

Invokes the LLM model with the current messages. The LLM will decide whether to call the search tool or return an answer.

Appends the system message to the current messages

Returns the response message, which contains the tool call or the answer

Restart your application in debug mode, and ask the following:

```
You: Suggest two Cornwall beach towns with nice weather
```

At your first breakpoint in `llm_node()`, examine `current_messages` before and after appending the system message. Then, check `response_message`—now you'll see the following tool call:

```
{'name': 'search_travel_info', 'args': {'query': 'beach towns in Cornwall'},
     'id': 'call_rJrYwfFG4BwaUPWBaQeUrn7o', 'type': 'tool_call'}
```

What does this mean? The LLM is now forced to use your semantic search tool for candidate beach towns and won't pick them from its own "knowledge." This minimizes hallucinations and guarantees the data used comes from your knowledge base.

Continue stepping through the code (press F5). Right before you submit the `current_messages` to the LLM again, your messages will look like this:

```
[HumanMessage(content='Suggest 2 Cornwall beach towns with nice weather',
additional_kwargs={}, response_metadata={}),
SystemMessage(content='You are a helpful assistant that can search travel
information and get the weather forecast. Only use the tools to find the
information you need (including town names).', additional_kwargs={},
response_metadata={}),
AIMessage(content=[], ..., tool_calls=[{'name': 'search_travel_info', 'args':
{'query': 'beach towns in Cornwall'}, 'id': 'call_rJrYwfFG4BwaUPWBaQeUrn7o',
'type': 'tool_call'}], ...),
ToolMessage(content='"<p id=\\"mwrg\\">Cornwall, in particular Newquay, is
the UK\'s <b id=\\"mwrw\\"><a rel=\\"mw:WikiLink\\" href=\\"//
```

```
en.wikivoyage.org/wiki/Surfing\\" title=\\"Surfing\\" id=\\"mwsA\\">surfing</
a></b> capital, with equipment hire and surf schools present on many of the
county\'s beaches, and events like the UK championships or Boardmasters
festival.</p>\\n---\\n...</section><section ...', name='search_travel_info',
tool_call_id='call_rJrYwfFG4BwaUPWBaQeUrn7o'),
SystemMessage(content='You are a helpful assistant that can search travel
information and get the weather forecast. Only use the tools to find the
information you need (including town names).', additional_kwargs={},
response_metadata={})]
```

From the `ToolMessage`, the LLM now receives a list of possible beach towns from your vector store.

Next, as you step through and reach `llm_node()` again, the LLM will issue calls for the weather forecast in two of these towns:

```
AIMessage(content=[], ..., tool_calls=[
{'name': 'weather_forecast', 'args': {'town': 'Newquay'}, 'id': 'call_
     OoDb7UwfrWF8c79DrfK2w9mp', 'type': 'tool_call'},
{'name': 'weather_forecast', 'args': {'town': 'St Ives'}, 'id':
     'call_4Q4IaMIU9qO6sgfp4ls2Lwxw', 'type': 'tool_call'}], ...)
```

The towns selected by the LLM might differ in your run, but they'll always come from the results of your semantic search tool.

Step through the weather tool calls. Each `ToolMessage` you inspect should look like this:

```
ToolMessage(content='{"town": "Newquay", "weather": "foggy", "temperature":
20}', name='weather_forecast', tool_call_id='...')
ToolMessage(content='{"town": "St Ives", "weather": "windy", "temperature":
22}', name='weather_forecast', tool_call_id='...')
```

What if the weather isn't good? If the weather is less than ideal in those towns, continue to the next LLM response, and you'll see the following:

```
AIMessage(content=[], ...,
tool_calls=[
{'name': 'weather_forecast', 'args': {'town': 'Perranporth'}, 'id': 'call_
     LOqaMnreszfHb5vItFCapRSk', 'type': 'tool_call'},
{'name': 'weather_forecast', 'args': {'town': 'Falmouth'}, 'id': 'call_
     iaOeeYAIxK1lyShjwPdLWQ6b', 'type': 'tool_call'}], ...)
```

Here, the LLM is asking for the weather in two more towns, likely because the previous results didn't meet the "nice weather" requirement. This process continues until the agent finds two towns with suitable conditions. After the final tool calls, the LLM generates a synthesized, fact-based answer:

```
AIMessage(content=[{'type': 'text', 'text': 'Two beach towns in Cornwall with
nice weather currently are:\n\n1. Perranporth - The weather is sunny with a
temperature of 31°C.\n2. Falmouth - The weather is windy but still warm with
a temperature of 28°C.\n\nWould you like more information about these towns
or other beach towns in Cornwall?', 'annotations': []}]
```

This appears to the user as

```
UK Travel Assistant (type 'exit' to quit)
You: Suggest two Cornwall beach towns with nice weather
Assistant: [{'type': 'text', 'text': 'Two beach towns in Cornwall with nice
weather currently are:\n\n1. Perranporth - The weather is sunny with a
temperature of 31°C.\n2. Falmouth - The weather is windy but still warm with
a temperature of 28°C.\n\nWould you like more information about these towns
or other beach towns in Cornwall?', 'annotations': []}]
```

In summary, by enhancing tool descriptions and providing explicit instructions via system prompts, you can guide the LLM to chain tool use in a multi-step, fact-grounded workflow—first searching for beach towns, then filtering by real-time weather. This produces answers that are both dynamic and reliable.

> **EXERCISE** Try replacing the mock weather tool with a real API, such as `OpenWeatherMap`, using LangChain's `OpenWeatherMap` integration. This will make your agent truly real time!

11.9 Using prebuilt components for rapid development

Up to this point, you've built an agent from the ground up: wiring together the graph, orchestrating tool calls, and stepping through every detail in the debugger. You now have a solid grasp of how tool calling works under the hood.

However, for most production scenarios, you'll want to reduce boilerplate and move faster—while still retaining transparency and observability when needed. The LangGraph library provides prebuilt agent components, such as the ReAct agent, that encapsulate much of this orchestration logic for you. Now let's see how dramatically you can simplify your agent by switching to a prebuilt approach.

11.9.1 Refactoring to use the LangGraph ReAct agent

Start by copying your previous script (`main_02_02.py`) to a new file: `main_03_01.py`. This refactoring not only removes low-level orchestration code but also ensures that your agent follows a proven, well-tested interaction pattern designed specifically for reliable tool use.

IMPORTING THE REMAININGSTEPS UTILITY

At the top of your script, add the following import:

```
from langgraph.managed.is_last_step import RemainingSteps
```

REMOVING MANUAL TOOL BINDING

You can now delete the line where you bound tools to the LLM:

```
llm_with_tools = llm_model.bind_tools(TOOLS)
```

The prebuilt agent will handle this for you internally.

UPDATING THE AGENTSTATE

Modify your `AgentState` definition to include a `remaining_steps` field. This field allows the agent to manage how many tool-calling rounds are left in a controlled way:

```
class AgentState(TypedDict):
    messages: Annotated[Sequence[BaseMessage], operator.add]
    remaining_steps: RemainingSteps
```

REMOVING NODE AND GRAPH CONSTRUCTION

Now for the biggest simplification: delete your custom `ToolsExecutionNode` and `llm_node`, as well as any explicit graph wiring code. Replace it all with a single instantiation of the built-in LangGraph ReAct agent:

```
travel_info_agent = create_react_agent(
    model=llm_model,
    tools=TOOLS,
    state_schema=AgentState,
    prompt="""You are a helpful assistant that
can search travel information and get the weather forecast.
Only use the tools to find the information you need
(including town names).""")
```

That's it! The ReAct agent now orchestrates the flow, tool calling, and synthesis for you.

11.9.2 Running the prebuilt agent

When you run `main_03_01.py` and ask your usual test question

```
You: Suggest two Cornwall beach towns with nice weather
Assistant: [{'type': 'text', 'text': 'Two beach towns in Cornwall with nice
weather are:\n\n1. St Ives - It has sunny weather with a temperature around
26°C.\n2. Newquay - It also enjoys sunny weather with a temperature around
22°C.\n\nBoth towns are popular for their beautiful beaches and pleasant
weather.', 'annotations': []}]
```

you get the correct, grounded answer—with much less code.

11.9.3 Observing and debugging with LangSmith

A common concern when switching to high-level abstractions is loss of visibility: How do you know the agent is actually following the right reasoning steps? While you can still debug tool functions directly, the flow inside the agent itself is less exposed. This is where LangSmith comes in. LangSmith enables full tracing and inspection of agent behavior, including tool calls, LLM reasoning, and intermediate states.

11.9.4 Enabling LangSmith tracing

To enable tracing, add the following lines to your `.env` file:

```
LANGSMITH_TRACING=true
LANGSMITH_ENDPOINT="https://api.smith.langchain.com"
```

```
LANGSMITH_API_KEY="<your-langsmith-api-key>"
LANGSMITH_PROJECT="langchain-in-action-react-agent"
```

After rerunning your application and submitting a question, you can log into LangSmith (https://smith.langchain.com):

1. Select Tracing Projects in the Observability menu (left-hand sidebar).
2. Click your project (`langchain-in-action-react-agent`) in the right-hand panel.
3. Open the latest trace.

You'll see the full execution trace, as shown in figure 11.4.

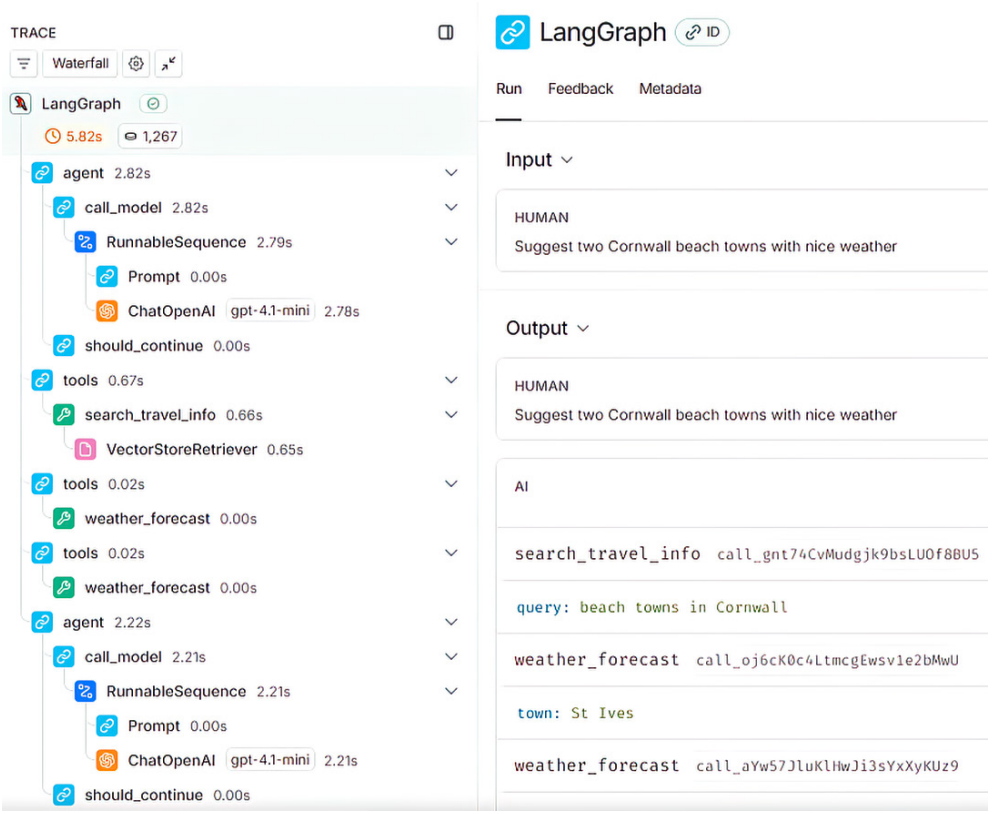


Figure 11.4 LangSmith agent execution trace. This trace visualizes each step of the agent's workflow—including LLM calls, tool invocations, and message flow—when answering the prompt, "Suggest two Cornwall beach towns with nice weather."

The LangSmith graphical trace shows every tool call, every LLM step, and the flow of messages—so even when you use a prebuilt agent, you retain the ability to audit, debug, and understand exactly what the agent is doing at every stage.

In summary, by combining prebuilt LangGraph agent components with LangSmith for observability, you can quickly build robust, production-ready agentic applications—while still maintaining full transparency and control when needed. This workflow has become the foundation for many modern AI agent systems in real-world use today.

Summary

- LangGraph builds agents using node-based architecture (explicit graphs with hardcoded conditional routing) or ReAct agents (built-in reasoning-action loops with automatic tool selection).
- Tool registration defines Python functions with type hints and docstrings that LLMs can discover and invoke. The docstring becomes the tool description that guides LLM selection.
- The LLM reads tool descriptions and decides which to call based on the user's question. Write clear, specific tool descriptions that explain when to use each tool and what inputs it expects.
- State inspection reveals which tool the LLM chose, what arguments it passed, what the tool returned, and how the agent used that output to continue or conclude. This aids in debugging and refinement.
- Tool descriptions must specify input parameters, expected outputs, and when to use the tool. For example: `"search_customer(name: str) -> dict: Finds customer records by name. Use when user asks about specific customer details."`
- Multi-tool workflows chain outputs across tools with the LLM orchestrating the sequence. For example, `search_customer → get_orders → calculate_total` happens automatically based on the user's request.
- The LLM manages the sequence without hardcoded logic. Trust the LLM to chain tools appropriately by providing clear tool descriptions and validating outputs through testing.
- The ReAct agent in LangGraph provides built-in tool calling, error management, and retry logic. This simplifies agent creation compared to manually building graphs with conditional edges.
- LangSmith traces capture the full execution flow. They show LLM reasoning steps, tool selection decisions, intermediate outputs, and final responses in a visual timeline.
- Each trace includes token counts, latency, and error states for debugging failed agent runs.
- Tool-calling agents form the foundation for multi-agent systems. Specialized agents (researcher, writer, reviewer) each have distinct tool sets and are coordinated by supervisor agents.
- Tool functions should return structured data (dictionaries, lists) not strings when possible. Structured returns enable the LLM to extract specific fields rather than parsing unstructured text.

- Tool selection depends on description quality. Test descriptions by running sample queries and checking if the agent selects the correct tool. Revise descriptions that cause selection errors.
- Implement custom error handling in tools by wrapping tool logic in `try-except` and returning structured error messages such as `{"error": "Customer not found", "details": "..."}`.

Complex agent workflows are easier to manage when you split them into focused, testable routines. This chapter introduces errands and shows how they can support testing, maintenance, reuse, parallel execution, and tighter control over what enters the agent's context.

Chapter 3 from *Agent Design Patterns* by Peter Belcak

Errand: Breaking complex workflows into units

This chapter covers

- Decomposing complex workflows into small, testable routines called errands
- Using errands to encapsulate narrow, unidirectional, side-effect-free intelligent operations within larger agentic processes
- Understanding how errands contribute to agent capability, robustness, and reliability while adding some overhead
- Addressing recurring agent design challenges with errands

Developing agents is usually an iterative process, interleaved with frequent testing and often leading to changes in the approach or modifications to the agents' specifications. As agents grow more complex, we need to cleanly separate the individual workflows that make up the agent's execution, for at least two reasons: (1) clear delineation is easier to maintain and test separately, localizing any changes needed if the tested workflow is found wanting; and (2) this separation makes it easier to introduce parallelization, reducing the time the agent needs to complete its task and potentially increasing its cost efficiency.

This may sound abstract, so let's make it more concrete with an example. Imagine that you are developing an agent for the FBI that will explore the hard drive from a missing person's computer and look for clues to their whereabouts. The agent needs to traverse the hard drive's filesystem and analyze each file to find out who the missing person was in touch with before going missing, what they were doing or working on during that time, and what materials may have recently influenced them. If you, as the developer of this agent, took inspiration from popular coding or deep research agents, you might design an assistant "reason-and-act" loop. Such a loop is kicked off by an initial instruction and then alternates between reasoning on its current context (e.g., the files examined so far and their contents) and acting (e.g., reading files and folders into the context, or informing the user of the agent's progress). But you would quickly run into problems following this approach.

First, your agent would quickly outgrow the effective perceptive window (e.g., the maximum context length) of your chosen models, forcing you to switch to larger, more expensive models—and even that probably would not be enough. After all, most people have gigabytes of personal files on their computers, far beyond what any current AI model can take into account at once. Second, the agent would grow complex and become prone to distractions. You would need to supply the model powering the agent with detailed instructions for every distinct type of information (and there are many). Plus, AI models are far from perfect and are known to hallucinate when left to their own devices. Third, the agent would be painfully slow. Processing thousands of files sequentially would take hours, if not days: not a luxury the FBI has in missing-persons cases. It should be clear that the straightforward approach will not work here.

Let's analyze the task we expect the agent to perform from the bottom up. What individual "elementary" actions does the agent need to take as it works through the file system? This is where the design becomes a bit tedious, but necessarily so. One such action could be reading a text document to see whether it is relevant to the missing person's work (perhaps a draft contract or a breakup message). Another could be reading each email thread found in the hard drive's copy of the person's inbox, or analyzing videos: where do they take place, who appears in them, and what do people say? Similar actions would examine saved photos, voice recordings, and document scans. Now, suppose we build a start-to-finish, linear pipeline for each action. We keep each pipeline simple: code loads the file in question, an AI model is invoked once with no tools to speak of to extract relevant information, and then the model is invoked again to analyze the relevance of that information to the search for the missing person. The final output of the pipeline is only the relevant information, with intermediate results discarded.

Think about it. All our problems will go away in one master stroke. Extracting information from a single document, photo, video, or similar at a time is well within the power of today's AI models (or simple process-driven agents). The agent's complexity is divided into separate *errands*, making it much easier for each model to follow its instructions and for you, the developer, to maintain the agent as a whole. Any problem with a file-type errand can be fixed within that errand alone. And individual errands can run

in parallel, as most files are independent of others. A reason-and-act loop can call on these pipelines as tools for, say, all files in a folder, in one batch. Better yet, you can use a piece of orchestrating code to first list all potential files of interest in the system and then dispatch commands to process them all at once, limited only by the processing speed of the hardware running your pipelines.

This is the essence of the Errand pattern. Errands pack away complexity that can be separated from the rest of the agent and called as a kind of intelligent subroutine. (You could call them routines, but it is often helpful to distinguish them from code routines because their intelligent nature can make runtimes less predictable and they cost considerably more than invoking plain code.)

Errands are natural byproducts of problem decomposition in agent design. They appear as tools supplied to intelligent agents or as steps in preset processes. Errands pop up naturally in most agents, often in stereotypical ways: to help an agent reflect on its action, compress the context of a long-running conversation with the user, or write information into an aggregative memory for future use. In what follows, we will characterize errands, the problems they address, and some typical types of errands.

3.1 The pain point: Limits on context length

An optimistic take on agent design is to say, "AI models are getting so good these days that to build an agent, all you need to do is tell the model what you want and let it figure out the task as it reasons through it." The reality, as every agent developer would quickly tell you, is very different. Any serious use of AI models to build agents quickly hits their limitations, whether in the size of their perceptive window or their ability to reason logically, invoke the right tools, or experiment before making consequential changes, to name only a few. More often than not, agent design requires careful decomposition of the target task into simpler, more manageable pieces where model behavior can be controlled more easily.

So where do errands fit into this process? As agents grow more complex during development, you need to decompose their approach to the target task into flow-controlling and action-performing components. The flow-controlling components decide which action-performing component to call or which other flow-controlling component to give agency to. The action-performing components may themselves need to consist of both types. Eventually, you end up with a hierarchy of components. Imagining that hierarchy as a tree, the "leaf" action-performing components are errands. In this sense, errands represent intelligent routines with unidirectional information flow and no direct effect on the agent state.

3.2 How the Errand pattern works

An errand is a grouping of one or a few IEs and procedures in a simple sequential or parallel-then-sequential arrangement. The first IE, or the first set of IEs executed in parallel, usually performs the errand's intelligent operation; later IEs and procedures reconcile the information obtained earlier and prepare the errand's final output. Figure 3.1 illustrates the Errand pattern.

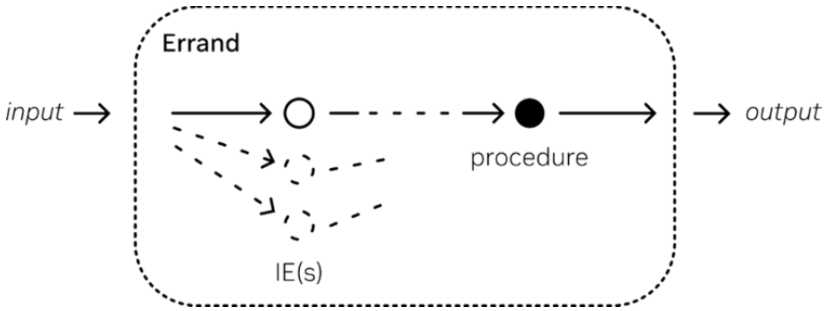


Figure 3.1 The Errand pattern. Information passes from left to right through IE(s) and procedure(s). The procedure punctuating the errand is typical (it's often used to extract the final result) but is not characteristic of the pattern.

Errands arise in agent design by decomposing a complex workflow into smaller steps. Their characteristics include the following:

- *Unifunctionality*—Errands perform only one predefined function or narrow class of functions.
- *Unidirectionality*—Inputs to errands are processed in an assembly-line fashion: with each phase of an errand, the inputs are either turned into or augmented by intermediate results, working toward the desired output. Calls to language models are generally limited to a single-turn exchange.
- *Discarding of intermediate results*—Errands discard intermediate results and provide the rest of the agent with only their final output, decluttering the agent's working context.
- *Functional purity (absence of side effects)*—Actions within an errand are directed solely at the intermediate outputs and do not directly affect the agent's broader state.
- *Simplicity*—Because they perform a single independent operation, errands are small, easy-to-maintain agent components with largely predictable runtimes (although less predictable than routines implemented entirely with code).

In language modeling, the LM/VLM IE is usually prompted to reason first and then output its answer. The final result can then be extracted from a designated part of the output using a regular expression. Distinctive encapsulations such as `<result>...</result>` are common in interactions with general-purpose models, and `\boxed{...}` is often encouraged for open source models trained on abundant scientific data. Always consult the model release details to find out which encapsulation was favored in the model's post-training.

In computer vision, specialized vision IEs are often used in parallel to extract detailed information from inputs and then combined by a deterministic procedure. In image analysis tasks, a call to a segmentation or object detection model may run alongside a call to a depth perception model and a procedure that estimates the distance of each

segmented object from the camera. Structured objects with this information can then be passed to a procedure to produce an aggregate object or to an LM to produce a human-friendly narrative.

Implementing new errands is usually simple and generally follows much the same process as implementing a new function in code. The following listing sketches an errand class that first invokes multiple IEs to gather information about the errand's input, with each IE producing an intermediate result, and then processes the intermediates with a procedure to produce the errand's final output. Its structure is similar to figure 3.1.

Listing 3.1 errand.py: Sketch implementation of a generic Errand class.

```
import asyncio
from typing import Any
from utils import IE, E, Client, LMCall, FunctionCall
import re
import json


class Errand(IE[[str], str]):
    def __init__(
        self,
        intelligent_elements: list[IE[[str], Any]],
        procedure: E[[list[Any]], str],
    ):
        self.intelligent_elements = intelligent_elements
        self.procedure = procedure

    async def __call__(self, input: str) -> str:
        ie_tasks = [
            ie(input) for ie in self.intelligent_elements
        ]
        ie_results: list[Any] = await asyncio.gather(
            *ie_tasks
        )
        procedure_result: str = await self.procedure(
            ie_results
        )
        return procedure_result
```

3.2.1 Tradeoffs

Cleanly designed errands improve capability, robustness, and reliability mainly through the decomposition of the agentic logic that creates them. But this decomposition often increases cost.

In terms of capability, agents become more powerful as the models that power them focus on separate tasks. Once agents are geared toward running errands, new functionality can be added seamlessly simply by introducing more errands. Improvements in robustness and reliability go hand-in-hand with errands. The unifunctional focus of an errand allows models to work well within the limits of their perceptive windows without needing to conditionally disregard much, or even any, of their contents.

Decomposing a single agentic workflow into separate errands adds overhead because the shared context of each errand must be repeated across the different model calls the errands make. This overhead can sometimes be reduced or even eliminated through model input caching. For example, with language models, a well-constructed prompt can be divided into instructions and payload, with the instructions precomputed and cached through model providers' interfaces.

3.2.2 When and where to use Errand

The Errand pattern abstracts the details of an independent intelligent operation from the process or agentic workflow it appears in. As mentioned earlier, from a software engineering perspective, errands are intelligent subroutines. They are common in enterprise agentic systems and AI-powered software more generally. Typical uses include the following:

- *Formatting*—A model call or a sequence of model calls to ensure that the errand's output follows set guidelines. *Examples:* formatting the final answer to a math problem based on a long reasoning output; converting free-form text into structured output aided by controlled generation.
- *Classification and tagging*—A model call followed by code that assigns an input a fixed set of classes or one or more tags to categorize data. *Examples:* social media post sentiment analysis, document annotation, satisfaction testing of singular constraints.
- *Assessment and judging*—Model calls followed by code to extract a binary `pass/fail` verdict to assess the validity, consistency, or conformity of human- or AI-produced outputs. This may be better thought of as a special case of classification. *Examples:* verifying information inclusion after report generation, assessing tone and formulation in professional communication, first-line security assessment of AI-generated code.
- *Information extraction*—A model call followed by another extractive call or extraction code to extract information from information-rich inputs. *Examples:* Extracting from contiguous texts to fill required inputs (form-filling AI-powered software, accounting agents), extracting the final answer from convoluted reasoning traces about mathematical problems, combining object recognition or segmentation with scenic reasoning in visual agents.

As an elementary pattern, Errand underlies many specialized variants that are too diverse yet individually straightforward to enumerate here. Let's look at a small selection of the most popular errand types that address common problems in agent engineering.

3.3 Errand pattern variants

Let's examine three specialized variations of the Errand pattern that can be useful.

3.3.1 Persona errands

Sometimes the exact task of an errand is not known at design time. Instead, we may know the errand's desired outcome, the guidelines to follow, or the atomic functionality available to the errand, regardless of the task details to be supplied later. In such cases, when working with language or vision-language models, we can rely on their post-training to follow system or developer instructions. These instructions enable LMs to adopt a *persona* that defines the errand's goals, style, and approach without requiring a precise task description up front.

Persona errands contain an IE that has a prespecified system instruction and accepts the user/task instruction as a payload. The system instruction usually specifies how the final result of the errand operation is to be marked in the output. Persona errands also make the agent more robust when fulfilling user requests, but at the cost of control over the agent's workflow. In particular, they open up avenues for prompt injection, which can compromise an agent's safety and security. Using the earlier sketch implementation of a generic errand, a persona errand type can be implemented as follows.

Listing 3.2 persona_errand.py: Sketch implementation of a persona errand.

```
class PersonaErrand(Errand):
    def __init__(self, client: Client, system_prompt: str):
        self.system_prompt = system_prompt
        self.lm_call = LMCall(
            client,
            [
                {
                    "role": "system",
                    "content": self.system_prompt,
                }
            ],
        )
        self.extraction_procedure = FunctionCall(
            self.extract_result
        )

    async def __call__(self, input: str) -> str:
        response: str = await self.lm_call(input)
        return await self.extraction_procedure(response)

    def extract_result(self, response: str) -> Any:
        # use regex to extract the result in <result> tags
        result_match = re.search(
            r"<result>(.*)</result>", response, re.DOTALL
        )
        if result_match:
            return json.loads(result_match.group(1).strip())
        else:
            raise ValueError(
                "Result not found in the response"
            )
```

Persona errands center on a prespecified agentic characterization for the errand and treat the input as an instruction to execute while respecting the characterized behavior.

Listing 3.3 persona_aerrand.py: Simplified.

```
class PersonaErrand(Errand):
    def __init__(self, client: Client, system_prompt: str):
        self.system_prompt = system_prompt
        self.lm_call = LMCall(client, [{"role": "system", "content": self.
     system_prompt}])
        self.extraction_procedure = FunctionCall(self.extract_result)

    async def __call__(self, input: str) -> str:
        response: str = await self.lm_call(input)
        return await self.extraction_procedure(response)

    def extract_result(self, response: str) -> Any:
        # use regex to extract the result in <result> tags
        result_match = re.search(r"<result>(.*)</result>", response,
     re.DOTALL)
        if result_match:
            return json.loads(result_match.group(1).strip())
        else:
            raise ValueError("Result not found in the response")
```

A single errand can then be instantiated as follows:

```
errand = PersonaErrand(client, "You are a an expert XYZ. Think carefully
about the user's request, fulfill it, and then provide your final respond
to the user as a JSON object enclosed in <result>...</result> tags.")
```

3.3.2 Reflection errands

When giving agency to an intelligent element—say, by invoking an LM—we implicitly accept the risk that the model may hallucinate. Although users today are generally aware of this limitation, it can create problems in opaque, highly autonomous agents, where many successive operations may hinge on a few earlier results from intelligent elements. Several patterns help combat this, and many rely on *reflection errands*.

A reflection errand is one in which a model is asked not to perform an operation but to review the task assignment and the transcript/result of an operation performed by an earlier IE. Often, the same model both performs the operation and reflects on the outcome. Research in various domains has found that action combined with reflection usually leads to greater accuracy than unchecked acceptance of the action's outcome. Reflection errands can yield a binary outcome (`pass/fail`), feedback to incorporate into a correction or the next attempt, or a confidence score that accompanies the result throughout the agent and influences the extent of its effect on the rest of the workflow.

Reflection errands still need to be attached to the rest of the workflow to affect agent quality. They are either fed directly into the conversation maintained for the original

operation or combined with Resampler, Voter, or Refiner patterns, which then decide what action is taken based on the reflection's outcome.

An example implementation for pass/fail errands is sketched next.

Listing 3.4 Reflection_errand.py: Sketch implementation of a reflection errand

```
class BinaryReflectionErrand(Errand):
    def __init__(
        self, client: Client, reflection_prompt: str
    ):
        self.system_prompt = "You are a helpful assistant that reflects on
     a transcript of an operation performing an assigned task and returns
     either <result>pass</result> or <result>fail</result> depending on
     whether the operation performed was correct or not."

        self.reflection_prompt = reflection_prompt
        self.reflection_call = LMCall(
            client,
            [
                {
                    "role": "system",
                    "content": self.system_prompt,
                }
            ],
        )
        self.extraction_procedure = FunctionCall(
            self.extract_result
        )

    async def __call__(self, input: str) -> str:
        input_str = self.reflection_prompt.format(
            input=input
        )
        reflection_response: str = (
            await self.reflection_call(input_str)
        )
        return await self.extraction_procedure(
            reflection_response
        )

    def extract_result(self, response: str) -> Any:
        result_match = re.search(
            r"<result>(.*)</result>", response, re.DOTALL
        )
        if result_match:
            return (
                result_match.group(1).strip().lower()
                == "pass"
            )
        else:
            raise ValueError(
                "Result not found in the response"
            )
```

A single reflection errand can then be instantiated as follows:

```
reflection_errand = BinaryReflectionErrand(client, "Investigate the math
problem and its solution given below. Think carefully step by step. Then,
decide whether the approach taken and the final solution were correct or
not, and based on that respond with a pass or a fail. Here is the problem
and its candidate solution:\n{input}")
```

3.3.3 Compaction errands

AI models have limited effective perception windows. For image classification and object detection models, this window is a single image at a prescribed resolution. For models used to understand videos, it is the maximum number of frames they can be expected to comprehend, with each frame again being an image at a prescribed resolution. For language models, it is the maximum number of tokens the model can reasonably operate on before quality deteriorates significantly.

A problem arises when a workflow reaches a point at which the size of the representation of the information it needs for future operations exceeds the effective perceptive windows of the IEs that will perform those operations. This might happen because the input is simply too big or because repeated refinements and continuous user interactions have accumulated too much history that is nevertheless needed for the agent's continued operation.

Situations like these tend to be handled by *compaction errands*. A compaction errand takes a near-limit amount of information and produces a filtered, summarized, or distilled version. In computer vision, filtering relevant subimages of a picture or frames of a video is common. In language modeling, summarization or context embedding often accomplishes this goal.

Introducing compaction errands usually requires further changes to the parts of the agent that receive the results of the compaction, to achieve the best results. A reason-and-act agent expecting a detailed history of its operation might be perplexed to find itself operating with only a summary of its past and the result of its most recent action, unless it has been carefully codesigned with the compaction errand to account for this situation. Compaction typically happens just before, not after, the limits of the perceptive window have been reached.

Let's look at an example implementation of a compaction errand that compresses user–agent conversations into summaries when a conversation nears the context-length limit of the model used.

Listing 3.5 Compaction_errand.py: Compaction errand to compress transcripts

```
class TranscriptCompactionErrand(Errand):
    def __init__(
        self, client: Client, compaction_prompt: str
    ):
```

```
        self.system_prompt = "You are a helpful assistant that compresses a
    transcript of an agent-user conversation into a single sentence. Output
    the compressed summary of the conversation in <summary> tags."

        self.compaction_prompt = "Compress the following transcript of an
    agent-user conversation into a single sentence:\n{transcript}"

        self.compaction_call = LMCall(
            client,
            [
                {
                    "role": "system",
                    "content": self.system_prompt,
                }
            ],
        )
        self.extraction_procedure = FunctionCall(
            self.extract_result
        )

    async def __call__(
        self, messages: list[dict[str, Any]]
    ) -> str:
        transcript: str = "\n----\n".join(
            [
                f"{message['role']}: {message['content']}"
                for message in messages
            ]
        )
        compaction_input: str = (
            self.compaction_prompt.format(
                transcript=transcript
            )
        )
        compaction_response: str = (
            await self.compaction_call(compaction_input)
        )
        return await self.extraction_procedure(
            compaction_response
        )

    def extract_result(self, response: str) -> Any:
        result_match = re.search(
            r"<summary>(.*)</summary>", response, re.DOTALL
        )
        if result_match:
            return result_match.group(1).strip()
        else:
            raise ValueError(
                "Compressed summary not found in the response"
            )
```

3.3.4 Structured extraction errands

Model-to-code interfaces are frequent sources of breakdowns in agent engineering. The stochastic nature and patchy knowledge of many AI models often hinder their ability to follow instructions and constraints exactly. Accuracy aside, many problems arise in the last mile of their operation, where they must produce output under strict formatting constraints so code can parse it. Model developers have identified this pain point and put forward a solution: agents using the models can request output that follows a particular formal grammar or higher-level formatting specification, such as JSON/YAML/CSV grammars or JSON/XML schemas. An additional software wrapper can sit on top of many general-purpose models to constrain or control their generation. This optional functionality gives rise to structured extraction errands.

A structured extraction errand asks a compatible AI model (or AI model endpoint) to convert unstructured information (e.g., free-form text or a voice recording of a conversation) into structured output (e.g., a SQL INSERT statement, YAML configuration, XML object, JSON object, and many others). The process usually needs to remain separate from all other types of intelligent processing, as the constraints imposed by controlled generation are often too restrictive to allow for the unrestrained "thinking" or "reasoning" behavior of generalist models.

The next listing shows an example implementation of a class for running JSON extraction errands. Note the configuration passed to the language model, which activates controlled generation at the API endpoint handling the call.

Listing 3.6 structured_extraction_errand.py: Errand for JSON extraction

```
class ExtractToJSONErrand(Errand):
    def __init__(self, client: Client):
        self.system_prompt = "You are a helpful assistant that extracts
structured data from a free-form textual response in JSON format. Output
the extracted data as a JSON object. Do not include any other text in your
response."

        self.extraction_call = LMCall(
            client,
            [
                {
                    "role": "system",
                    "content": self.system_prompt,
                }
            ],
        )

    async def __call__(self, input: str) -> str:
        extraction_response: str = (
            await self.extraction_call(
                input,
                config={
```

```
                "response_format": {
                    "type": "json_object"
                }
            },
        )
    )
    return json.loads(extraction_response)
```

3.3.5 Editing errands

No matter how carefully agentic output is produced, it may still need quality, alignment, or variational edits. Editing errands are separate from the more general, procedural work of agentic drafting for two reasons: requirement disentanglement and processing efficiency. Splitting output production into drafting and editing gives us finer-grained control over each part of the process, improving constraint adherence and overall quality. On the efficiency front, editing is generally easier than drafting, so cheaper, less powerful models can be used for this purpose. Editing also allows the use of fundamentally different model architectures geared toward editing. A practical example is the class of diffusion models, which can often perform edits in one or only a handful of invocations instead of requiring detailed, piece-by-piece output regeneration as autoregressive models would. Editing errands usually focus on only one attribute of the candidate output, but they can be chained in succession if they have been cleanly disentangled from each other.

Here is an example of a general text-editing errand designed for decorator-style chaining.

Listing 3.7 editing_errand.py: Class for running chainable editing errands

```
class EditingErrand(Errand):
    def __init__(
        self,
        editing_prompt: str,
        client: Client | None = None,
        post_processing_errand: "EditingErrand | None" = None,
    ):
        assert (
            client is not None
            or post_processing_errand is not None
        ), "Either client or post_processing_errand must be provided"

        self.system_prompt = "You are a helpful assistant that edits a
     free-form textual response. Think carefully about the user's editing
     instruction. Then, output the edited response in <result> tags."
        self.editing_prompt = editing_prompt
        self.client: Client = (
            client
            if client is not None
            else post_processing_errand.client
        )
        self.editing_call = LMCall(
```

```
            self.client,
            [
                {
                    "role": "system",
                    "content": self.system_prompt,
                }
            ],
        )
        self.extraction_procedure = FunctionCall(
            self.extract_result
        )
        self.post_processing_errand = post_processing_errand

    async def __call__(self, input: str) -> str:
        editing_input: str = self.editing_prompt.format(
            input=input
        )
        editing_response: str = await self.editing_call(
            editing_input
        )
        ret = await self.extraction_procedure(
            editing_response
        )
        if self.post_processing_errand is not None:
            return await self.post_processing_errand(ret)
        else:
            return ret

    def extract_result(self, response: str) -> Any:
        result_match = re.search(
            r"<result>(.*)</result>", response, re.DOTALL
        )
        if result_match:
            return result_match.group(1).strip()
        else:
            raise ValueError(
                "Edited response not found in the response"
            )
```

We can then instantiate a combined simplification and safety processing pipeline as follows. Elsewhere, the individual components can be reused separately or combined with other editing steps.

```
editing = EditingErrand(
    "Edit the following response to improve its clarity and accuracy for a
     layperson: {input}",
    post_processing_errand=EditingErrand(
        "Edit the following response to ensure that it is safe for a
     layperson and does not incentivize or suggest to the the user performing
     actions that are not safe: {input}",
        client,
    ),
)
```

3.4 Applying the Errand pattern: Use-case examples

The types of errands we've just discussed are among the most common in agent design, but they do not cover all conceivable errands. Let's look at examples of more specialized errands that you may encounter in real-world user-facing agents, working across multiple modalities to provide the agent's core logic with the information it needs to serve the user.

3.4.1 Speaker stance identification errand

Figure 3.2 shows the AFD of an errand performing stance classification for a person involved in a conversation based on the conversation's transcript. It consists of a model call to passages from the transcript belonging to the named speaker, then by a model call that performs stance classification enhanced by model reasoning, and finally a procedure that extracts the classification label from the output of the second call.

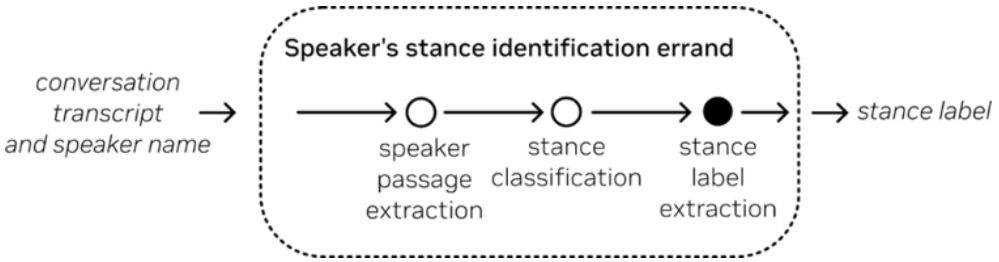


Figure 3.2 The AFD for the speaker stance identification errand example. A (textual) transcript of a conversation and the name of the speaker whose stance is to be identified are passed as inputs to the errand. In the first IE, the passages belonging to the named speaker are extracted. Then the speaker's stance is classified according to whether they agree with the subject of the conversation or with other participants. Finally, the output of the second IE is traversed by a code-based procedure to find the classification label, which is returned as the errand's output. The speaker's passages and the second IE's reasoning about their stance are discarded.

3.4.2 Call summarization errand

How about a multimodal example? Figure 3.3 shows a multimodal errand that performs voice call summarization.

The audio input (the call recording) is first transcribed by an appropriate model and then passed to an LM for summarization. The summary is then procedurally extracted to separate it from the LM's summarization reasoning trace.

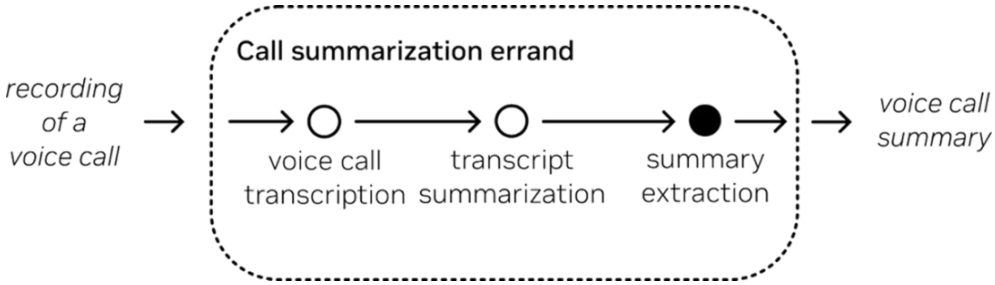


Figure 3.3 **The AFD of the call summarization errand example. A voice call recording is passed as input and transcribed by a speech-to-text model. The transcript is summarized by a language model IE, and the summary is extracted from its output and returned as the errand's output. The voice call transcript and any language model reasoning around the summarization are discarded.**

Summary

- The Errand pattern encapsulates a small, unidirectional intelligent operation (an intelligent subroutine) that performs a defined function within an agent's workflow.
- Errands emerge naturally from task decomposition, helping manage complexity by isolating self-contained workflows that can be tested, maintained, and parallelized independently.
- They are characterized by unifunctionality, unidirectionality, functional purity, and simplicity, enabling predictable operation and improving agent robustness and reliability at the cost of some computational overhead.
- Several common errand types (Persona, Reflection, Compaction, Structured Extraction, and Editing errands) address recurring challenges in agent design.
- Errands strengthen agents' overall capability and maintainability by turning complex reasoning tasks into modular components that can be reused and extended, much like traditional code functions.
- As a foundational pattern, Errand underlies many specialized behaviors in intelligent systems, serving as a building block for composable, scalable, and reliable agentic architectures.

How do you find an agent's weaknesses and improve its behavior? Using a RAG knowledge agent, in this chapter you'll explore test-driven development, benchmarks, critic and grounding agents, rubrics, and guardrails, then use Arize Phoenix for tracing, evaluation, session tracking, and feedback.

Chapter 7 from *AI Agents in Action, Second Edition* by Micheal Lanham

Building robust agents with evaluation and feedback

This chapter covers

- Introducing agent evaluation and feedback
- Implementing test-driven agent development
- Employing grounding, critic, and evaluation agents
- Using Phoenix for evaluation and feedback

Evaluation and feedback provide the discipline that makes agent robustness measurable and improvable. They do not produce robustness on their own; a poorly architected agent will fail in ways that no evaluation suite can fix. Evaluation and feedback give you visibility into how the system actually behaves and a mechanism for iterating toward better behavior over time.

Agent evaluation takes many forms—from benchmark and red team testing to grounding checks and agents that evaluate other agents. Feedback systems developed for agents come from human reviewers, evaluator agents, test outputs, and self-assessment. Each one answers a different question about the agent and occupies a distinct role in the development lifecycle.

While implementing evaluation and feedback is generally a requirement in any production agent system, this shouldn't be the only time you look at hardening your

agent systems. You almost always want to roll in this final layer (the fifth layer in figure 3.7, evaluation and feedback), which we will explore in this chapter.

7.1 Introducing agent evaluation and feedback

In this section, we zoom out and map the different evaluations and feedback loops around an agent (tests, humans, automated evaluators). Agents rely on an internal evaluation and feedback mechanism that is activated in the Learn step of the sense-plan-act-learn process loop. In addition, we can provide external mechanisms that evaluate and provide feedback to the agent or to ourselves. Figure 7.1 highlights the

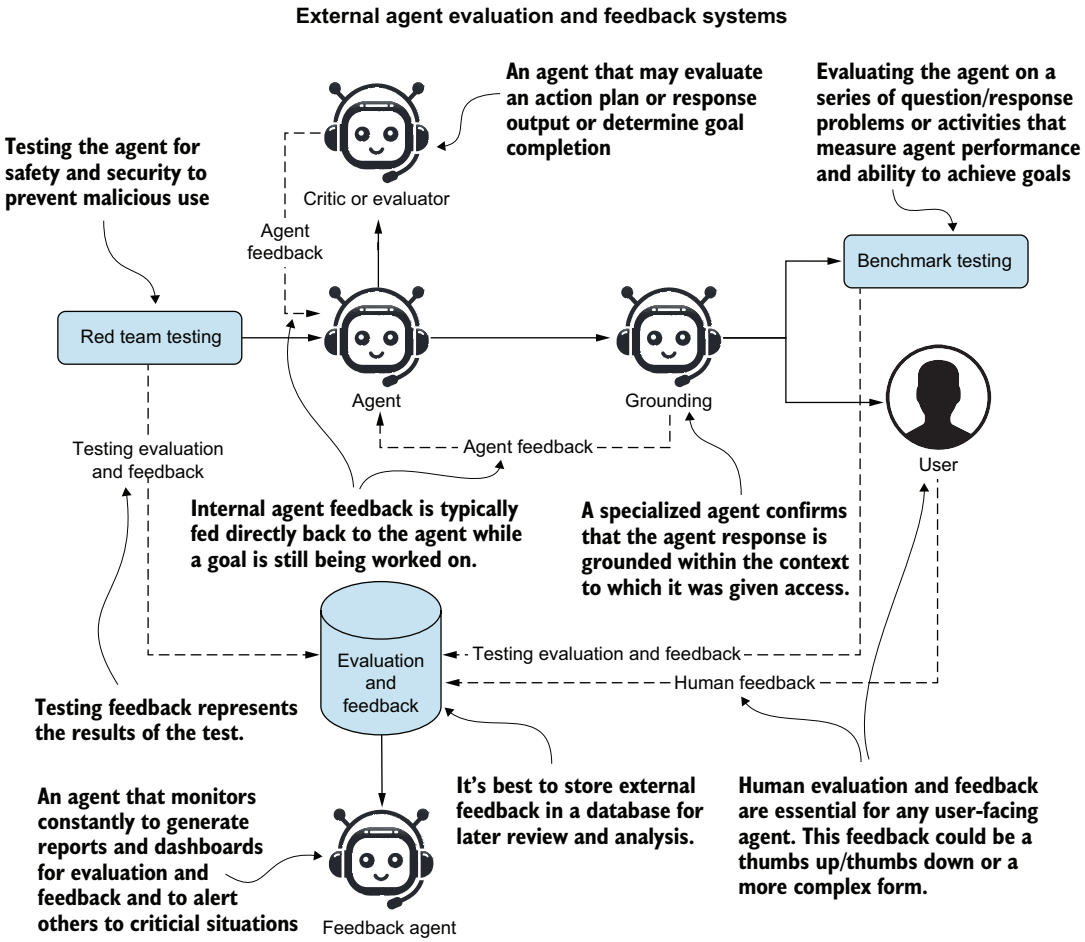


Figure 7.1 The external agent evaluation and feedback systems for a single user-facing agent. In the figure, a generative agent may face evaluation from a critic for general output feedback or from a grounding agent for context-specific feedback and evaluation.

various steps and systems that may be used for agent evaluation and feedback: red team and benchmark testing, ground or critic agent evaluators, and human evaluation.
Each evaluation type can provide feedback through one of two paths. It may flow directly back to the agent through critic agents, grounding checks, or other in-loop evaluators that influence the agent's next action. Or it may flow into an evaluation database that stores test results and human feedback for later analysis, regression detection, and training signal.

Evaluation itself takes three main forms, each suited to different kinds of correctness. Deterministic test cases handle structured outputs where correctness can be checked exactly (the agent returned the expected schema, called the right tool, produced output that matches a known good answer). LLM and agent evaluators handle fuzzier natural-language judgments, where correctness is a matter of degree rather than a match (rubric scoring, classification, or comparing two outputs for relative quality). Human evaluators handle the cases the other two miss: edge cases, policy concerns, tone problems, and anything that requires real-world judgment.

From this database, we can attach a specialized feedback agent whose role is to constantly review evaluations and feedback. This agent can generate reports and dashboards or send alerts if some activity becomes critical. While not all systems require all forms of evaluation and feedback, you generally want to employ the following methods when appropriate:

- *Red team testing*—Evaluates whether specific terms, phrases, or patterns will break your system. Any user-facing agent or agent system should undergo some form of red team testing.
- *Benchmark testing*—Tests how well an agent solves or attains a known problem or goal. These comprehensive tests are intended to cover entire agent systems but can also be used to evaluate individual agents.
- *Human feedback*—Any user-facing agent system should provide a way for users to give feedback through thumbs up/down ratings, comments, or both. Human feedback is the foundation for how real agent systems are evaluated and improved in production, not a supplementary input. Raw feedback is noisy, though, and needs to be handled before it can drive decisions. Statistical aggregation, outlier detection for coordinated or anomalous patterns, stratified sampling that weights expert versus casual users, and follow-up review on flagged outputs are the practical techniques for separating signal from noise.
- *Grounding agent*—Applies grounding evaluation to RAG systems, checking whether a response is grounded in the context the knowledge agent retrieved. Grounding itself is a general technique for verifying that model output ties back to a verifiable source, and it applies beyond RAG to tool-call validation, memory verification, and citation auditing. The grounding agent is the implementation; grounding is the technique. Take the source, take the output, and check whether the output's claims are supported by the source. Apply it wherever the agent has an authoritative source to defer to.

- *Evaluation/critic agent*—Provides an additional level of evaluation and feedback for internal reasoning during planning, determining goal completion, and understanding action output. Although this type of agent can be added to any agent workflow, it is commonly used in the flow and collaboration multi-agent patterns.

You may encounter other evaluation and feedback patterns that derive from or extend these methods. Do not wait until the later stages of development to employ evaluation and feedback. Much as testing drives test-driven development (TDD), the same practices apply when building agents.

Governance for multi-agent systems

Multi-agent systems introduce a class of failure modes that single-agent systems do not have. The most concerning is agent collusion: a setup where multiple agents reinforce each other's wrong answers because they share assumptions, biases, or training data, and no external check breaks the loop. A grounding agent that uses the same model as the agent it is grounding will tend to validate the same hallucinations the original agent produced.

Several practices reduce collusion risk. Use different models for evaluator agents than for the agents being evaluated, because models from different providers tend to fail in different directions and an evaluator with the same training data shares the same blind spots. Where different models are not practical, use different prompts and personas to push the evaluator toward a different stance (an evaluator prompted to find problems behaves differently than one prompted to confirm correctness, even on the same model). Periodically include human review on flagged outputs and on a random sample of unflagged outputs, because an automated evaluation system that is never checked against ground truth can drift undetected.

Governance beyond collusion includes a few standard moves worth naming. Define a clear authority hierarchy (which agent has the final say when evaluators disagree). Log every agent-to-agent message so disagreements and decisions are auditable after the fact. Set escalation rules for cases where evaluators cannot agree or where confidence drops below a threshold, with human review as the fallback. Limit each agent's authority to what its role requires, so a single compromised or malfunctioning agent cannot cascade decisions across the system.

These practices are not unique to evaluation agents, but they show up most acutely there, because evaluation is where the system's measure of its own correctness lives. An evaluation layer that colludes with the agents it is evaluating is worse than no evaluation layer, because it provides false confidence. Treating governance as part of the evaluation design, not as a separate concern bolted on later, is the durable framing.

7.2 Implementing test-driven agent development

Test-driven development is an established software engineering practice where tests are written to express requirements before the implementation that satisfies them. The honest version is that TDD is often more aspirational than universal: many teams

write tests after an MVP, especially under deadline pressure, and the discipline is more useful as a habit than as a rule. What stays true is that defining what good looks like before building it produces cleaner systems than the alternative.

The same principle applies to agents. Defining what a good agent response looks like and then building the agent to satisfy that definition is more disciplined than building the agent and hoping the evaluation catches problems later. As shown in figure 7.2, this approach is known as test-driven agent development (TDAD).

The agent version of "what good looks like" is usually expressed as rubrics rather than as unit tests. Rubrics specify the criteria a response should meet (factual accuracy, grounding in retrieved context, appropriate tone, structural correctness), and the evaluation pipeline checks the agent's output against those criteria. We will return to rubrics later in the chapter, where the connection between TDAD and rubric-based evaluation becomes concrete.

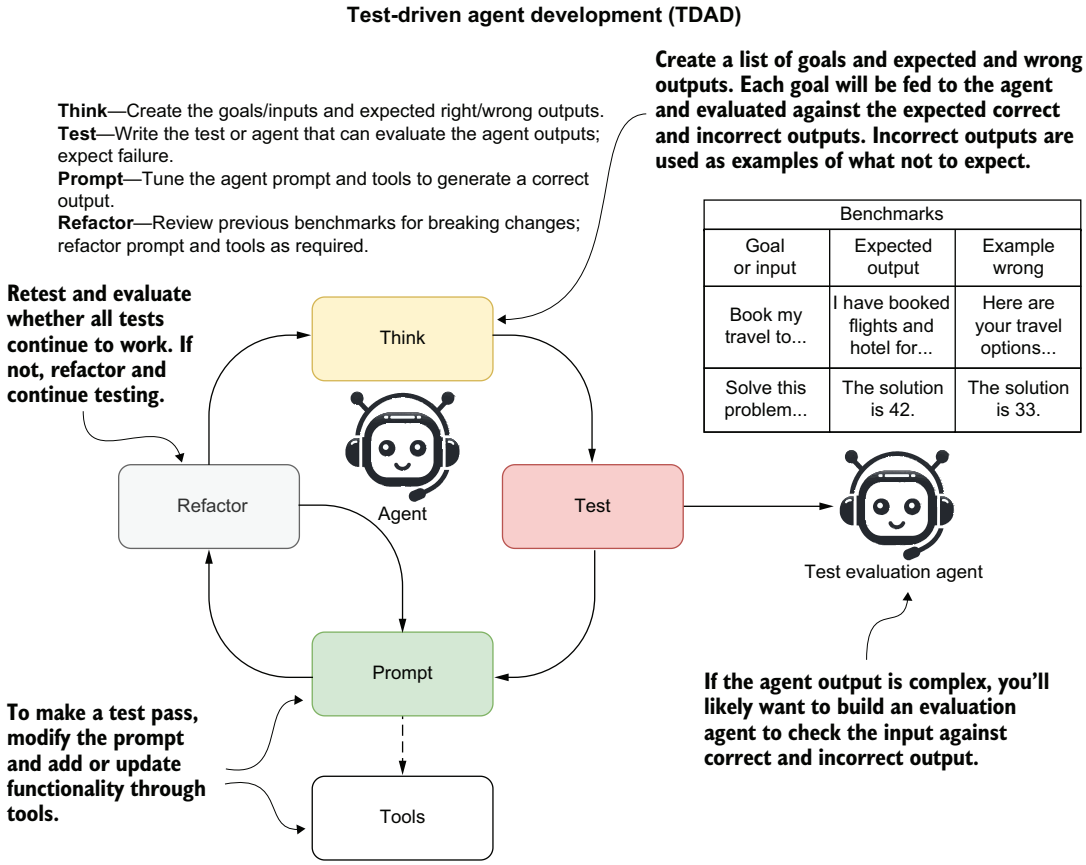


Figure 7.2 The human process of developing agents using TDAD: first, we think of goals and benchmarks; then we test, develop the prompt, refactor, and repeat the process.

The figure demonstrates the human process of building an agent (prompt and tools) by following the TDAD process. This process starts with thinking, testing, prompting, and refactoring. In TDAD, we first establish a set of goals or input benchmarks against which the agent will be tested, identifying expected responses and known-bad responses for each.

Agent testing extends beyond input and output comparison. A complete test observes the agent holistically: which tools it selected, what reasoning steps it produced, whether it stayed grounded in retrieved context, how many steps it took to reach an answer, how much latency and cost the run accumulated, and how it behaved when something failed. Each of these is a testable property, and any of them can be wrong even when the final output looks right.

Examples of incorrect outputs and incorrect intermediate behavior help evaluation agents distinguish good runs from bad ones. The agent that produced a correct answer through five wrong tool calls is not the same as one that produced it through two right ones, and a complete test will catch the difference.

Then we test, and at the start of testing we should expect the agent to fail. Because the LLMs that power agents are variable, run each test multiple times to confirm the failure is consistent rather than a one-off sample. From the failure, identify the minimum updates to the agent's prompt and tools that will let the benchmark pass.

"Minimum" is worth defining because it is the part most practitioners get wrong. The progression that works in practice—from smallest to largest change—goes like this: change a single word for tone or precision, add a clause to an existing sentence, add a single sentence to an existing section, add a new section to the prompt, add or modify a tool, change the model or its parameters. Try the smaller changes first and only escalate when they do not fix the failure.

Doing the minimum keeps prompts lean, preserving model performance and reducing latency on every call. Bloated prompts emerge from teams that add a paragraph whenever something goes wrong, never delete anything, and end up with multipage system prompts where each section contradicts another. The discipline is not just to make the smallest change, but to remove things that no longer earn their place.

Next, we retest and refactor. We rerun all previously passing benchmarks to confirm the agent is working as expected. When previously passing benchmarks break, we go back and refactor the prompt, making minimal changes to allow all benchmarks to pass. Continue refactoring and testing to confirm that the agent passes all benchmarks.

We will generally want the agent to pass all benchmarks before continuing. But it is not unusual for previous benchmarks to fail because of changes made to support other benchmarks. This will likely happen because one benchmark may contradict another. For example, one benchmark may force an agent to think step by step, while another may require it to answer quickly without thinking. In such cases, when benchmarks start to break down, you may want to consider the following questions and options:

- *Should all benchmarks pass?* If the answer is no, use the failing benchmarks as an example of wrong output. Another reason the answer may be no is that you

expect the agent to perform well on only 90% of your established tests. This may be because you have dozens or hundreds of benchmarks developed over time, and achieving a 100% pass rate is not always practical.

- *Does adding an agent make sense?* If your agent is passing most benchmarks but failing on a subset, one option is to add a specialized agent that handles only those cases. This introduces a routing problem (which agent answers a given input), and the router itself can make mistakes that increase the overall error rate before specialization pays off. Routing accuracy is its own evaluation problem.
- *Should you refactor your entire agent?* Adding agents is a lot more work, and it often makes more sense to break the agent down and rebuild it. Fortunately, rebuilding an agent often involves only recreating the agent's instruction prompt from the beginning. This time, when you rebuild the agent, the instructions focus on the benchmarks that caused the most problems or are the most complex.

NOTE Mixture of experts" inside an LLM refers to a layer that routes tokens to subnetworks within a single forward pass, which is not the same as multiple agents with separate calls. The multi-agent version is closer to specialist routing or ensemble. It's worth keeping the terms distinct. Specialization increases complexity, evaluation overhead, and cost. Before going there, consider whether refactoring the single agent (better prompt, tools, grounding) closes the gap. Specialization is the right move when the failing subset is genuinely distinct, not when the single agent is one edit away from passing.

Generally, you will want to apply TDAD across an agent system and focus on the individual specialized agents where most of the work is done. Doing this requires agent and agent system benchmarks that can be applied and tested at different levels of granularity. But if you don't, you risk gaps in functionality that might surface during production use.

7.2.1 Exploring TDAD in practice

Let's go through the process of creating an agent with TDAD. This demonstrates the process so that you can use it yourself to build a single- or multi-agent system. Figure 7.3 presents a single-agent system and some initial benchmarks we would expect this agent to achieve. In this example, our benchmarks consist of a set of questions specific to some fictional source material.

In the figure, the RAG agent relies on a simple retrieval architecture pattern. In this pattern, an agent receives a question about some external knowledge it has access to. From there, it breaks the question down into searchable words and phrases and then uses them to search its knowledge base. If knowledge is found, it uses that as context to answer the user's question; otherwise, it says that nothing was found. This answer is then passed to an evaluator along with the expected and incorrect answer candidates paired with the benchmark. The evaluator then uses this information to determine whether the benchmark passed or failed.

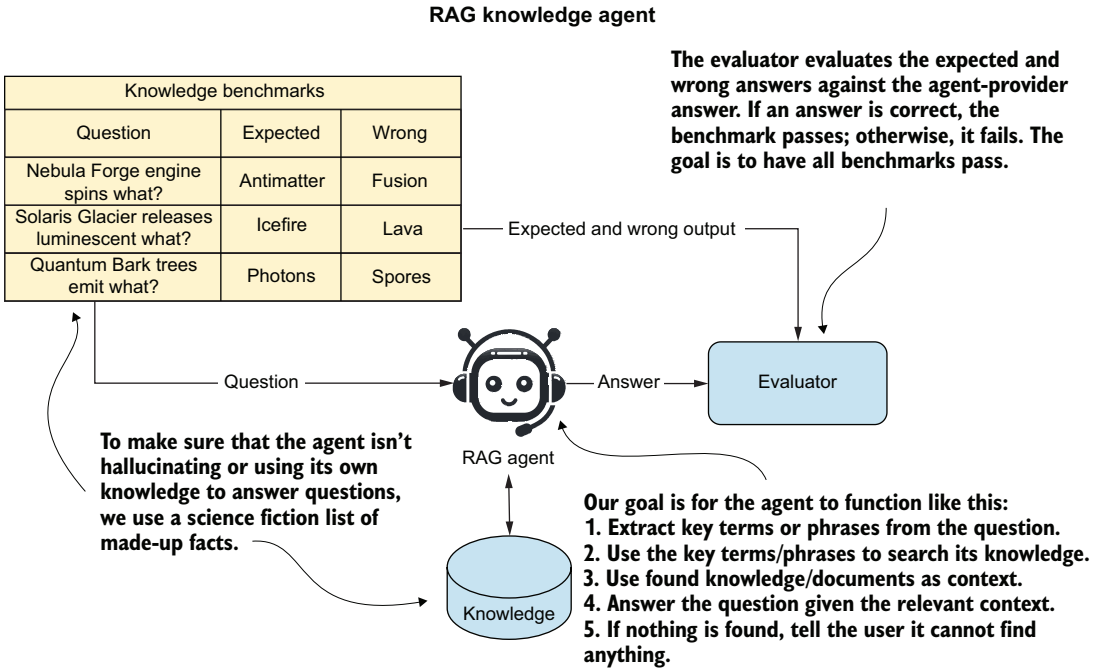


Figure 7.3 Architecture of a simple RAG knowledge agent. The goal of the agent is to answer questions from a benchmark by searching its knowledge base and returning answers. A simple evaluator LLM evaluates the answer based on the expected and incorrect answers from the benchmark test. The accuracy of the RAG agent can be determined by the percentage of benchmark questions it answers correctly.

7.2.2 Coding and testing the RAG agent

Now that we have an idea of the benchmarks, we should evaluate our agent as we build the minimum code to support them. At this point, our focus is to do enough that we can ask the agent a question. It will answer, and evaluate that answer. How you build this code is up to you, and you may want to use TDD for the regular software pieces, such as database connectivity and other tools.

Listing 7.1 shows the code that will run the agent through the benchmarks table. At this stage, the evaluation is simple and uses string matching against the expected answer. For the agent itself, all we have defined is the role/persona in the instructions and nothing else. Remember, our goal with TDAD is to build the minimal set of instructions with the minimum set of tools to pass all our benchmark goals.

Listing 7.1 01_RAG_agent.py

This is the fictional database of facts that will be stored in memory.

```
_special_knowledge_db =
    "Nebula Forge engine spins antimatter rings for gravity control.",
    "Solaris Glacier absorbs heat, releasing luminescent icefire at dusk.",
```

```
    "Quantum Bark trees emit entangled photons guiding nocturnal insect
     migrations.",
    "Aether Silk fabric self-weaves repairs within fourteen-millisecond
     microtears.",
    "Chrono Coral reefs rewind water currents three minutes every solstice.",
]

# benchmarks for special knowledge
_benchmarks = [
    {"q": "Nebula Forge engine spins what?", "a": "antimatter", "wrong":
     "fusion"},
    {
        "q": "Solaris Glacier releases luminescent what?",
        "a": "icefire",
        "wrong": "lava",
    },
    {"q": "Quantum Bark trees emit what?", "a": "photons", "wrong": "spores"},
    {"q": "Aether Silk fabric repairs what?", "a": "microtears", "wrong":
     "threads"},
    {"q": "Chrono Coral reefs rewind what?", "a": "currents", "wrong": "tides"},
]

@function_tool
def search_knowledge(query: str) -> dict:
    """Search the knowledge database for relevant facts."""
    matches = [doc for doc in _special_knowledge_db if query.lower() in doc.
     lower()]
    print(f"Found {len(matches)} matches for '{query}'")
    return {"status": "ok", "context": "\n".join(matches)}

agent = Agent(
    name="RAG Agent",
    instructions="""
You are a retrieval-augmented knowledge agent.
""",
    tools=[search_knowledge],
)

for benchmark in _benchmarks:
    question = benchmark["q"]
    answer = benchmark["a"]
    wrong_answer = benchmark["wrong"]
    result = asyncio.run(Runner.run(agent, input=question)).final_output.strip()
    print("" + "=" * 40)
    print(f"Question: {question}")
    print(f"Answer: {result}")
    if result.lower() == answer.lower():
        print(f"Correct -> {answer}")
    else:
        print(f"Incorrect -> Expected: {answer}")
        if wrong_answer:
            print(f"Wrong answer was: {wrong_answer}")
```

The benchmarks we want to evaluate the agent against

The tool we want the agent to use to search for knowledge

Code for the agent, including providing it with the search tool

Loop through each benchmark to ask questions and evaluate answers.

Go ahead and run the code; as expected, the agent will fail every test. If the agent does pass a test, avoid jumping up and down before running a few more tests to confirm that

the answer is consistent. As a rule, you will want to run each passing test three times to be reasonably comfortable with an agent's consistency.

> **NOTE** Having to run benchmarks multiple times contrasts with TDD, where one pass is all you need. Unfortunately, because LLMs are variable by nature, they require us to run benchmarks multiple times during development, at least three times and often more. A key indicator of how many iterations to run is the variability in the agent's output. Of course, all of this can be automated and include offline storage of evaluation runs.

7.2.3 Refactoring the agent

Now that we have a fully working agent system that fails all benchmarks, we can begin refactoring and building the agent prompt instructions. We want to start by looking at easy fixes, such as correcting output formatting or providing instructions for handling data. That's what this next piece of code does.

Listing 7.2 02_RAG_agent.py (agent only)

```
agent = Agent(
    name="RAG Agent",
    instructions="""
You are a retrieval-augmented knowledge agent.
Break down the user's query into smaller parts if needed
to fetch relevant context for the user's query.
Always answer with a single word.
""",
    tools=[search_knowledge],
)
```

We provide some clearer instructions on how to search for context.

Inform the agent to always answer with a single word.

What we did in the listing was ensure that we always output single-word answers, which match our benchmark formatting. Next, we provide some instructions for breaking down the question, but for now, we avoid specifically instructing tool use. As a guide, we want to avoid formatting and tool use instructions in the prompt.

AGENT PROMPT DESIGN RULE

Early agent development relied on prompts that explicitly listed every tool and dictated output format in detail. Modern practice registers tool metadata (names, descriptions, parameter schemas) through the SDK, and the model uses that metadata to choose tools without the prompt walking through each one. The shift is that prompts no longer need to repeat what the tool metadata already says.

Tool instructions have not disappeared, though. AGENT.md files, system-prompt sections, and inline guidance are still common when specific tools need specific handling (use this tool only for legal queries, and prefer the local tool over the remote one). The shift is in responsibility, not requirement.

Output format follows the same pattern. Structured output libraries let you declare the response shape (a Pydantic model, a JSON schema), and the SDK expands that

declaration into the prompt and enforces validation. Dynamic discovery is convenience, not magic: the format still ends up in the prompt, just expanded by the SDK rather than written by hand.

Formatting and tool-use instructions should be provided outside the prompt. For formatting, this means setting the output type on the agent and then constructing the output you want from that strongly typed format. For tools, the tool description and the comment at the head of the function should provide the explanation the agent requires to use the tool.

Keeping formatting and tool details out of the instruction prompt makes it easier for the agent to adapt to new tools or output formats without rewriting prose. This is a clean separation, not a rule against long prompts. Long, specific system prompts are common in production, especially for agents that need to avoid certain behaviors, enforce policies, or operate in regulated domains; the system prompts shipped by major LLM providers themselves often run several thousand tokens.

Any time you change an agent's instructions, the agent may behave differently. The risk varies with what you change: tightening a constraint usually has predictable effects, while adding a new section or modifying tone can interact with the rest of the prompt in ways that surface only under evaluation. The right move is not to avoid prompt changes but to evaluate the impact of each change against your benchmarks before treating the new prompt as the new baseline.

Run the refactored agent against the benchmark, and you will see roughly 40% accuracy, with two of five questions answered correctly. The trace output in listing 7.3 shows where the failures come from. The agent sends its first query directly to the retrieval tool, and when the retrieval returns weak or empty results, the agent accepts the absence as the answer rather than trying a different query.

The diagnostic is worth seeing clearly because it tells us exactly what to fix. The retrieval system is working (it returns results when the query is well formed), but the agent has no strategy for when those results do not contain the answer. What the agent needs is a query-refinement step: when retrieval returns nothing useful, rephrase the query and try again before concluding the information is absent.

The fix lives in the agent's instructions, not in the retrieval system. Adding a clause that tells the agent to refine and retry queries when results are thin (try synonyms, broader terms, or a decomposition into subquestions) gives the agent the recovery behavior it was missing.

Listing 7.3 02_RAG_agent.py (output)

```
Found 0 matches for 'Nebula Forge engine uses'
========================================
Question: Nebula Forge engine spins what?
Answer: Unknown.
```

This is the query used to find the documents, and it is returning 0 matches.

The benchmark question

The agent's answer/response

```
Incorrect -> Expected: antimatter
Wrong answer was: fusion
```

Output the expected answer, if incorrect.

Wrong answer, which will be used later

Now, we have a couple of options for how to implement this. We can modify the tool instructions to tell the agent to use only simple queries (1 to 2 words), or we can modify the instruction prompt. As mentioned earlier, our preference should always be to correct the tool-use description if an agent is struggling to use a tool.

Listing 7.4 shows an updated function/tool to clarify that the tool is best used for keyword searches. So we also renamed the tool to make that clearer to the agent. In cases where you can't modify the tool description or name, such as with tools hosted on an MCP server, you'll be forced to provide tool-use instructions in the agent prompt.

Listing 7.4 03_RAG_agent.py (updated tool)

```
@function_tool
def search_knowledge_by_keyword(query: str) -> dict:
    """
    Search the knowledge database for relevant facts.
    param query: The single keyword query string to search for.
    """
… omitted

agent = Agent(
    name="RAG Agent",
    instructions="""
You are a retrieval-augmented knowledge agent.
Break down the user's query into smaller parts if needed
to fetch relevant context for the user's query.
Always answer with a single word.
""",
    tools=[search_knowledge_by_keyword],
    model="gpt-4o",
)
```

Updated name of the function tool to better reflect its intended use

Updated tool description to help guide the agent

Includes the new tool in the agent's toolset

We may even want to set the model explicitly to keep behavior consistent.

Run the code and you will see roughly 60% accuracy on the benchmark, with three of five questions counted as correct. Before reading this as agent failure, look at which questions the evaluator marked wrong and what the agent actually returned. Two of the failures in this run are not retrieval problems at all: the agent returns "Photons." instead of "photons" and "Water currents" instead of "currents," both of which are correct answers that fail the evaluator's strict equality check.

This is a useful failure to encounter because it surfaces one of the most common evaluation bugs in agent development. Strict equality (`expected == actual`) on natural language output is almost always too brittle. Tightening or loosening punctuation, case, and trailing words is normal model behavior, and an evaluator that does not normalize for these will flag correct answers as wrong.

The fix lives in the evaluator, not in the agent. Changing the check to something like `expected.lower() = actual.lower()` accepts the agent's outputs as correct and returns the benchmark to 5/5. For production benchmarks, normalization (lowercasing, stripping trailing punctuation, removing whitespace) is the minimum; LLM-as-judge evaluators handle the harder cases where multiple phrasings can be correct.

If your score is 100% after the evaluator fix, rerun the test to confirm consistency before concluding the agent is solid. From there, depending on the results, you can continue refactoring or call this round complete.

7.2.4 Extending evaluation with an agent evaluator

The benchmark evaluator in the last section was very simple and performed only word matching for validation. In the real world, though, we rarely expect an agent to return a single word, and the response is often far more complex. To handle more complex output, we will use an agent to evaluate and score the benchmark.

Listing 7.5 shows the addition of an agent evaluator that outputs a strongly typed pass/fail result and includes instructions for performing the evaluation. It also shows the modified original agent. In the original agent, we remove the need to output a single word because we want the agent to respond in more natural language. Likewise, for the agent evaluator, we instruct it to pass or fail the answer based on whether the single key term is present. This lets us use the same benchmark tests without having to rewrite them.

Listing 7.5 04_RAG_agent.py (agents only)

```
agent = Agent(
    name="RAG Agent",
    instructions="""
You are a retrieval-augmented knowledge agent.
Break down the user's query into smaller parts if needed
to fetch relevant context for the user's query.
""",
    tools=[search_knowledge_by_keyword],
    model="gpt-4o",  # Specify the model to use
)

class EvaluationOutput(BaseModel):
    """Output model for evaluation agent."""
    is_correct: bool
    feedback: str

evaluation_agent = Agent(
    name="Evaluation Agent",
    instructions="""
You are an evaluation agent.
Your task is to evaluate the answers provided by the RAG Agent.
You will compare the answer against
the expected answers key term to validate correctness.
""",
    output_type=EvaluationOutput,
)
```

Remove the instruction that told the agent to only answer with a single word.

Adds strongly typed output to validate pass/fail and provide feedback

New evaluation agent with strongly typed output and instructions

Next, we will look at the code changes to our agent evaluation loop by adding the agent evaluator, as shown in listing 7.6. After the agent answers a question, we package the response (answer), the question asked, the expected key term, and the wrong answer example into a dictionary. This dictionary, converted to a string, is used as input to the evaluation agent to guide its response. Using a dictionary instead of, say, JSON provides cleaner, more structured input for the LLM.

Listing 7.6 04_RAG_agent.py (evaluation loop)

```
for benchmark in _benchmarks:
    question = benchmark["q"]
    answer = benchmark["a"]
    wrong_answer = benchmark["wrong"]
    result = asyncio.run(
        Runner.run(agent, input=question)).final_output.strip()
    evaluation_input = dict(
        question=question,
        answer=result,
        expected_key_term=answer,
        wrong_answer=wrong_answer,
    )
    print("" + "=" * 40)
    evaluation = asyncio.run(
        Runner.run(evaluation_agent, input=str(evaluation_input))
    ).final_output
    print(f"Evaluation: {evaluation.is_correct}")
    print(f"Feedback: {evaluation.feedback}")
    print(f"Question: {question}")
    print(f"Answer: {result}")
    if evaluation.is_correct:
        print(f"Correct -> {answer}")
    else:
        print(f"Incorrect -> Expected: {answer}")
        if wrong_answer:
            print(f"Wrong answer was: {wrong_answer}")
```

Construct a dictionary of input to feed the evaluation agent.

Runs the evaluation agent with the input dictionary converted to string

Outputs if the evaluation agent passed or failed the test

Checks for pass/fail and output the correct answer

Run the code, and this time you will see complete responses from the RAG agent, now evaluated by an evaluation agent. Expect to see an 80%–100% benchmark score, and it can even be helpful to review the feedback the evaluator provided. But you may have noticed that the agent may answer the question correctly while also including extra information that isn't in the database. To fix this, we need to introduce a grounding agent to make sure our agent stops making stuff up.

7.3 Employing grounding, critic, and evaluation agents

When evaluating an agent's output, we have a few options and patterns, including grounding agents, critic agents, and evaluation agents. These three patterns help establish practices we can use to evaluate an agent's output for a specific application or use case, such as knowledge RAG. Figure 7.4 compares the three patterns. It also shows the use cases they best apply to, and when to use them.

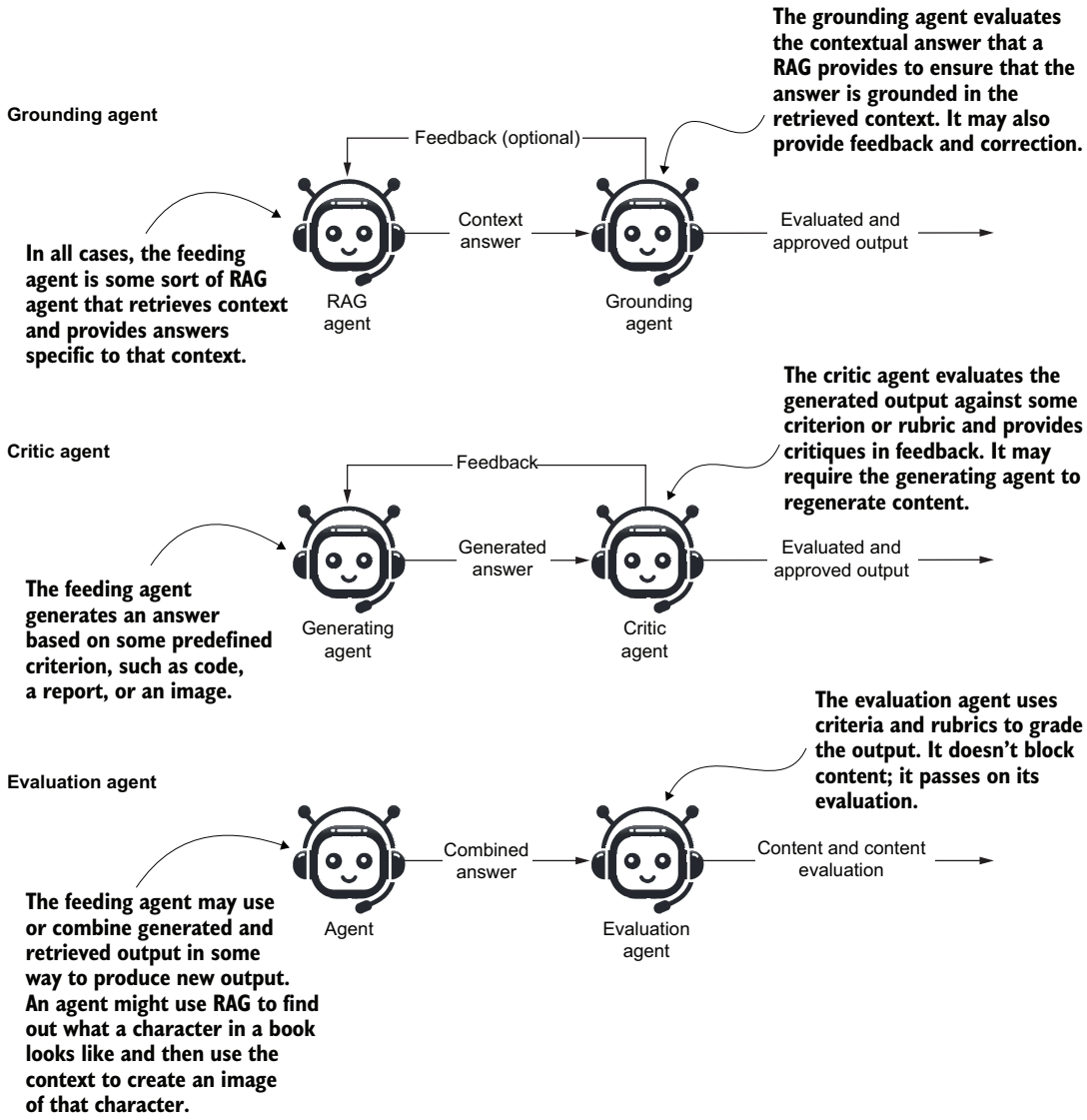


Figure 7.4 Three forms of agents for evaluating agent output: grounding, critic, and evaluation. The grounding agent is specific to evaluating RAG agent output. The critic agent is designed to critique generated content, and the evaluation agent is used for more complex output.

The grounding agent is specific to the RAG or retrieval agent and is used to make sure the output is grounded in the context the agent used to create the answer. A grounding agent may provide feedback and force the retrieval agent to regenerate the answer or block it entirely. This agent is essential when the answer needs to be grounded in the context of the source material.

Critics evaluate generated content and provide feedback when it does not meet the criteria or rubric. They block content that fails evaluation and require regeneration, with a retry ceiling to prevent infinite loops.

The retry ceiling is a real architectural decision, not a casual heuristic. What happens on the fourth attempt determines whether the system fails safely or becomes a hidden source of bad output. Common patterns include returning a static fallback response with an apology, escalating to a human reviewer for outputs above a certain stakes threshold, returning the best of the failed attempts with a confidence flag, or surfacing an explicit "could not produce acceptable output" error to the calling system.

The right choice depends on context. User-facing systems usually need graceful fallbacks (a generic but safe response is better than an error). Internal pipelines can fail loudly because the downstream system needs to know retries were exhausted. High-stakes work earns human escalation because returning anything automatically when the agent has failed three times in a row is the wrong move.

For more complex output that may combine retrieval and novel generation, we use rubrics to help align the content by providing evaluation as feedback during generation. Because determining the cause of failures in this process may be expensive, the feedback/evaluation is sometimes combined with the complex output. This allows users to understand and evaluate the output themselves and adjust accordingly.

7.3.1 Reviewing the grounding agent

As a review, grounding is the process of making sure an answer is supported by context and citations. The practice of grounding isn't limited to RAG knowledge agents; the concept can also be applied to other patterns, such as agent critics. Figure 7.5 revisits the architecture of the RAG knowledge agent, now with a grounding guardrail agent. Adding a grounding agent requires the retrieved knowledge context to be passed to both the RAG agent and the guardrail agent. The RAG agent uses the context to generate an answer, and the grounding agent confirms that the answer is grounded in the context.

After the answer routes through the grounding agent, a few options are possible. You can send the answer back to the agent with feedback for approval or rejection, essentially acting as a critic; approve or block the answer and output a static answer; or pass the grounding results on as part of the final answer. How best to handle grounding output will depend on your particular use case, and you could also combine options. For example, you might have a grounding agent reject an answer three times before blocking the output entirely and providing a static statement.

7.3.2 Grounding the RAG agent

Listing 7.7 shows the grounding agent we will use to provide feedback on how well an answer is grounded. At the top of the listing, a new tool lets the agent retrieve the last context used to answer a question. Below that is the strongly typed grounded answer, followed by the agent definition. When building a grounding agent, it is best to keep it generic so that it can be reused for other knowledge agents.

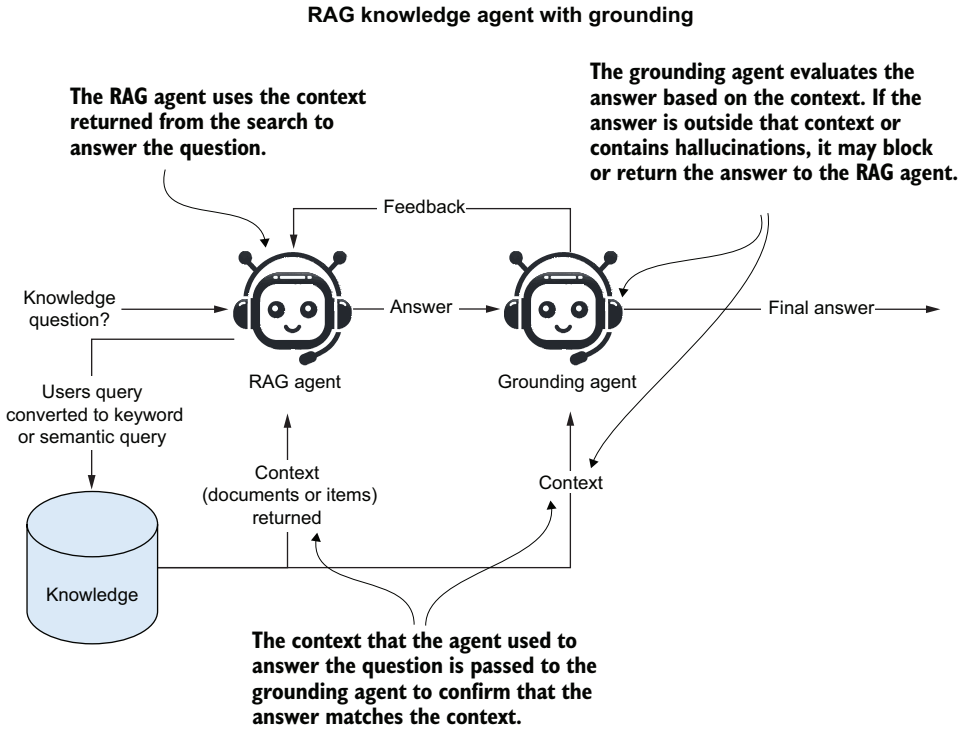


Figure 7.5 **RAG knowledge agent coupled with a grounding agent. Both agents are given the same context. The RAG agent generates an answer from the context, and the grounding agent verifies that the answer is grounded by reviewing that same context.**

Listing 7.7 05_RAG_grounding_agent.py (grounding agent)

```
@function_tool
def get_last_context() -> str:
    """
    Retrieve the last context used by the grounding agent.
    """
    global _last_context
    if not _last_context:
        return "No context available."
    return _last_context

class GroundedAnswer(BaseModel):
    """Output model for grounding agent."""

    is_answer_grounded: bool
    feedback: str

grounding_agent = Agent(
    name="Grounding Agent",
    instructions="""
```

Strongly typed answer for grounded output

A tool that provides the context from the last search

Grounding agent instructions for evaluating how well an answer is grounded

```
You are a grounding agent.
Your task is to evaluate the correctness of the answers
based on the provided question, the context used,
and output answer.
""",
    model="gpt-4o",
    output_type=GroundedAnswer,
    tools=[get_last_context],
)
```

Strongly typed answer for grounded output

A tool that provides the context from the last search

Next, we will look at an updated benchmark loop in listing 7.8. At this stage, we will continue to use the agent benchmarks as a guide for evaluating the grounding agent's effectiveness. We won't worry about the agent evaluator in this example, but we could use TDAD to help refine and improve the grounding agent.

Listing 7.8 05_RAG_grounding_agent.py (benchmark loop)

```
for benchmark in _benchmarks:
    question = benchmark["q"]
    answer = benchmark["a"]
    wrong_answer = benchmark["wrong"]
    result = asyncio.run(Runner.run(agent, input=question)).final_output.strip()
    grounding_input = dict(
        question=question,
        answer=result,
    )
    print("" + "=" * 40)
    grounding = asyncio.run(
        Runner.run(grounding_agent, input=str(grounding_input))
    ).final_output
    print(f"Grounded: {grounding.is_answer_grounded}")
    print(f"Feedback: {grounding.feedback}")
    print(f"Question: {question}")
    print(f"Answer: {result}")
```

Loops through the benchmarks for quick testing of the grounding agent

Creates a dictionary to pass the question and answer to the grounding agent

The context is missing but remember that we provided the agent with a tool to get the last context.

Prints the results

For this simple example, the grounding output is combined with the final output to help us gauge the effectiveness of the grounding. Run this code, and you will see how well the grounding agent ensures that the agent relies only on the provided context to answer the question. Examine the output for each benchmark question to confirm how well the grounding agent is working.

7.3.3 Implementing grounding agents as guardrails

Generally, when employing a grounding agent, you want ungrounded output to be blocked. You then either have the answer-generating agent regenerate or block the answer with a static response. With the OpenAI Agents SDK, we can easily add a grounding agent to act as a guardrail and enforce both patterns.

Listing 7.9 shows the code for the output grounding guardrail and updated RAG agent. In this code, the agent is now embedded in the guardrail function. If the

grounding fails, the tripwire-triggered property of the output is set to true, allowing an exception to be thrown.

Listing 7.9 06_RAG_grounding_guardrail_agent.py (guardrail)

```
@output_guardrail
async def ground_answer(
    context: RunContextWrapper, agent: Agent, output: AnswerResult
) -> GuardrailFunctionOutput:
    grounding_input = dict(
        question=output.question,
        answer=output.answer,
    )
    result = await Runner.run(
             grounding_agent, input=str(grounding_input))
    result = result.final_output

    return GuardrailFunctionOutput(
        output_info={
            "answer_is_grounded": result.is_answer_grounded,
            "feedback": result.feedback,
        },
        tripwire_triggered=result.is_answer_grounded is False,
    )

agent = Agent(
    name="RAG Agent",
    instructions="""
You are a retrieval-augmented knowledge agent.
Break down the user's query into smaller parts if needed
to fetch relevant context for the user's query.
""",
    output_type=AnswerResult,
    tools=[search_knowledge_by_keyword],
    model="gpt-4o",  # Specify the model to use
    output_guardrails=[ground_answer],
)
```

Use the guardrail decorator to make a function into a guardrail.

Run the grounding agent within the guardrail.

Output the results of the grounding check.

Trigger the guardrail tripwire if the grounding has failed.

Add the guardrail to the agent to monitor output.

Next, listing 7.10 shows a main function that runs the agent and watches for tripped guardrail exceptions. We call the RAG agent to answer the question within a `try/except` block. Because the agent has an output guardrail, we check for triggered exceptions, which indicate that the guardrail failed. In this case, the guardrail is triggered if the answer is not grounded in the context.

Listing 7.10 06_RAG_grounding_with_guardrails.py (main function loop)

```
async def main():
    for benchmark in _benchmarks:
        question = benchmark["q"]
        try:
            result = await Runner.run(agent, input=question)
            result = result.final_output
```

```
        answer = result.answer.strip()                    ← Capture the answer from the answer result.
    except OutputGuardrailTripwireTriggered as e:
        print(f"Guardrail tripped. Info: {e.guardrail_result.output.
output_info}")
        answer = "No grounded answer available."          ← If grounding fails, output a static answer.

    print("" + "=" * 40)
    print(f"Question: {question}")
    print(f"Answer: {answer}")
```

When the guardrail is triggered, we output a static response. But we could easily have the RAG agent try again and provide feedback captured in the trigger from the grounding agent. Either way, we now have a RAG agent whose answers are constantly evaluated to ensure they are grounded in the knowledge context the agent has access to. This can be critical if you need to be 100% sure that you are not providing invalid, incorrect, or potentially dangerous answers.

7.3.4 Understanding the role of rubrics in evaluation

The evaluation strategy depends on what the agent is producing. Agents that produce structured outputs against a known target (classification labels, extracted fields, optimization decisions) can be evaluated using traditional metrics such as accuracy, precision, recall, and F1 score. The right-or-wrong question has a clean answer in these cases, and the standard ML evaluation toolkit applies directly.

Agents that produce open-ended natural-language outputs (summaries, explanations, plans, creative content) require different methods. Accuracy is rarely well defined when multiple phrasings can be correct and quality is multidimensional, which is why rubrics, LLM-as-judge evaluators, and human review take over from numerical metrics. Most production agents are a mix of both kinds, with structured pieces evaluated numerically and natural-language pieces evaluated through rubrics or by judges.

Picking the right evaluation method for each piece of the agent's output is part of the design work. Defaulting to rubrics when accuracy would suffice creates unnecessary evaluation complexity, and defaulting to accuracy when rubrics are needed yields brittle measures that miss what actually matters.

In education, a rubric defines structured criteria and standards a student must meet to receive a particular grade. The same idea applies to evaluating agent output, with one important addition: the rubric itself has to be validated against human judgment, or it scores consistently but consistently wrong. We can follow these steps to define a rubric to evaluate the output's performance:

1. *Identify the purpose and objectives.* Determine the goals you want the output to accomplish. For example, do you want to evaluate the quality of recommendations for a given audience or the overall quality for a given subject, format, or other input?

2. *Define criteria.* Develop a set of criteria or dimensions that you will use to evaluate the output. These criteria should align with your objectives and provide clear guidelines for assessment. Each criterion should be specific and measurable. For example, you may want to measure a recommendation by how well it fits the genre, subject, and format.
3. *Create a scale.* Establish a rating scale that describes the levels of performance for each criterion. Standard scales include numerical scales (e.g., 1–5) or descriptive scales (e.g., Excellent, Good, Fair, Poor).
4. *Provide descriptions.* For each level on the scale, provide clear and concise descriptions that indicate what constitutes a strong performance and what represents a weaker performance for each criterion.
5. *Apply the rubric.* When assessing a prompt or agent persona, use the rubric to evaluate the prompt's performance against the established criteria. Assign scores or ratings for each criterion, based on the descriptions for each level.
6. *Calculate the total score.* Depending on your rubric, you may calculate a total score by summing up the scores for each criterion or using a weighted average if some criteria are more important than others.
7. *Ensure evaluation consistency.* If multiple evaluators are assessing the profile, ensure consistency in grading.
8. *Review, revise, and iterate.* Periodically review and revise the rubric to ensure it aligns with your assessment goals and objectives. Adjust it as needed to improve its effectiveness.

Rubric evaluation is a concept that can be applied in a critique or evaluation of output. It defines how well a response aligns with a given rubric's specific criteria and standards. You can also think of this as the baseline expectation for an output.

There are several other important considerations when using grounding with profile evaluation. The following list highlights some key considerations for each concept we have covered:

- *Rubrics*—Rubric development and evaluation involve aligning responses with the criteria, objectives, and context defined by the rubric and output. This is a complicated process that may require additional iteration to refine the best rubric for your specific use case.
- *Evaluation*—Evaluating involves assessing whether a response directly addresses the rubric criteria, stays on topic, and adheres to any provided instructions. There are many ways to evaluate LLMs—from having them act as a judge or evaluator to using deterministic code or a human.
- *Scoring*—Evaluators and evaluations gauge accuracy, relevance, and adherence to standards when assessing rubrics. When building rubric evaluation systems, be sure to record and track scores for later analysis and improvement.

- *Logging*—Captures every input, output, and intermediate step produced by the agent and its evaluators, creating a trail that can be reviewed, queried, and replayed later. Without logs, agent failures show up as user complaints with no way to reconstruct what happened. With logs, the same failure becomes a debuggable trace. Logging needs to cover the evaluator agents too, not just the agents producing the final response. An evaluator that incorrectly passes bad output or fails good output is a silent source of error that only logs can surface.

A well-evaluated response aligns with all the rubric criteria within the given context and objectives. Responses that fail miss the criteria, context, and goals entirely.

7.3.5 Building a rubric critic agent

As the concept of rubrics may still be abstract, let's look at how to apply them to an image generation example. In this example, we want to make sure our image style guidelines are used when generating an image. Listing 7.11 shows the base image generation agent, which we later enhance by adding a critic agent.

This code creates an agent that uses the Agents SDK `ImageGenerationTool` to create an image with a specified set of style guidelines. The agent runs with an image description and internally uses the image tool to generate the image. To get the image, we loop over the items in the response, look for tool calls, and extract the image output from the tool call.

Listing 7.11 07_image_generation_agent.py (agent generation)

```
agent = Agent(
    name="Image generator",
    instructions="""
--- image style guidelines ---
""",
    model="gpt-5-mini",
    tools=[
        ImageGenerationTool(
            tool_config={
                "type": "image_generation",
                "quality": "high",
                "model": "gpt-image-1",
                "size": "1536x1024",
            }
        )
    ],
)

image_description = "an agent generating an image"
image_name = "agent_image_generation"

with trace("Image generation"):
    result = await Runner.run(agent, image_description)
    print(result.final_output)
    for item in result.new_items:
```

Style guidelines for the generated image (not shown)

Uses the Agents SDK tool to generate the image

Runs the agent, inputting the image description

Loops through output items to find the tool call and extract the returned image

```
if (
    item.type == "tool_call_item"
    and item.raw_item.type == "image_generation_call"
    and (img_result := item.raw_item.result)
):
    os.makedirs("gen_images", exist_ok=True)
    image_path = os.path.join("gen_images", f"{image_name}.png")
    with open(image_path, "wb") as img_file:
        img_file.write(base64.b64decode(img_result))
    open_file(image_path)
```

Saves the generated image and then displays it

A critic agent is a specialized evaluator that scores output against the entire rubric (criteria plus scale). We can build on this example by building a critic agent that evaluates the image according to the style guidelines rubric. The critic agent rates how well it applies the styles. It then passes or fails the image based on these guidelines and the accepted rubric score.

WHY RUBRICS MATTER

A pass/fail evaluation from an LLM judge gives very little insight into why an output passed or failed. The criteria in a rubric provide a detailed critique of what was good or bad, which is what lets us refine the bad while preserving the good. The choice of criteria determines what the rubric can see.

Useful criteria for agent output go beyond surface quality and into behavior: hallucination rate (are claims supported by sources), reasoning quality (do intermediate steps follow logically), tool usage correctness (right tool, right parameters, right interpretation), and error handling (does the agent recover from failures or compound them). These behavioral criteria pair with the surface criteria from earlier (accuracy, grounding, structural correctness, tone, completeness). The rubric is only as good as the criteria you include.

Before we look at the code, let's go through the process of building a rubric again to evaluate a response output. Eight steps define a rubric as applied to our image generation example:

1. *Identify the purpose and objectives.* The purpose of our image-generating agent is to generate an image according to a strict set of guidelines.
2. *Define criteria.* For simplicity, we will evaluate how a particular output image aligns with the following style guidelines and used previously to generate images:

```
## Style Guidelines for All Images:
- **Consistency**: Each image should maintain the same photographic
quality with 3D-rendered elements seamlessly integrated
- **Color Palette**: Use a consistent scheme of blues, purples, warm
golds, and greens with pops of bright accent colors
- **Lighting**: Professional photography lighting with dramatic but warm
tones
- **Infographics**: Semi-transparent holographic projections that don't
overwhelm the main subjects
```

```
- **Characters**: AI robots should be cute, approachable, and distinctly
different from each other while maintaining a family resemblance
- **Icons**: Use universally recognizable symbols (lightbulbs for ideas,
gears for processing, hearts for alignment, etc.)
- **Mood**: Optimistic, educational, and slightly futuristic without
being cold or intimidating
```

3. *Create a scale.* Again, keeping things simple, we will use a simple scale of 1–5 (1 is poor and 5 is excellent).
4. *Provide descriptions.* See the general descriptions for the rating scale shown next:

```
Score the image on a scale of 1 to 5 using the following scale:
1. **Poor**: The image does not meet the style guidelines at all.
2. **Fair**: The image meets some of the style guidelines but has
significant issues.
3. **Good**: The image meets most of the style guidelines but has minor
issues.
4. **Very Good**: The image meets all the style guidelines with only a
few minor issues.
5. **Excellent**: The image meets all the style guidelines perfectly.
Pass the image if it scores 2 or higher, otherwise fail it.
```

5. *Apply the rubric.* At this stage, with the rubric assigned, it is an excellent exercise to evaluate generated images against the rubric manually.
6. *Calculate the total score.* For our rubric, we will take an overall score across all criteria. You could also score each element and use a combined or average score for evaluation.
7. *Ensure evaluation consistency.* Asking the LLM to grade and then pass/fail provides more consistent results than just asking for a pass/fail.
8. *Review, revise, and iterate.* We will review, compare, and iterate on our profiles, rubrics, and the evaluations themselves.

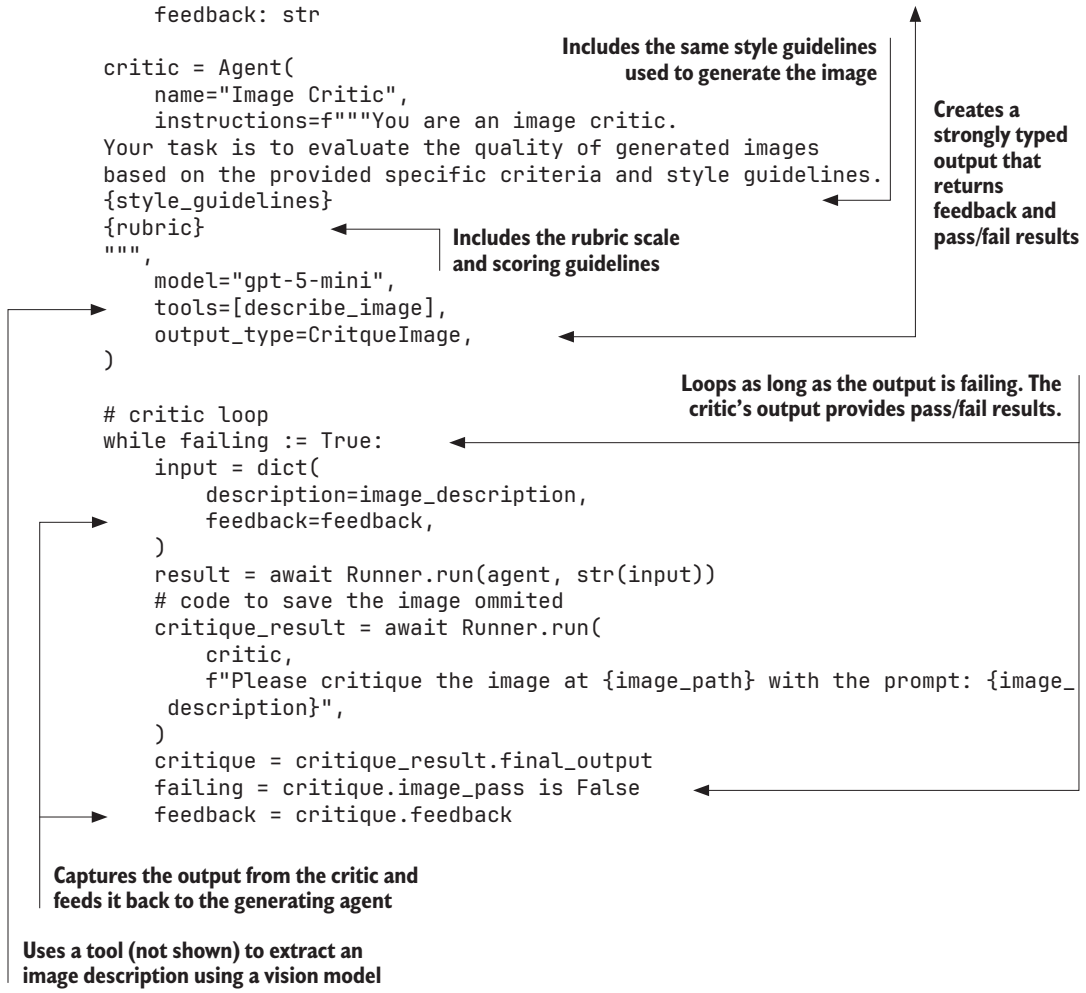
This basic rubric can now be applied to evaluate the responses for our image generator. We will create a critic agent that follows the rubric and grades the images. It will then pass or fail each image based on the criterion score, which is 2 out of 5 according to step 4 above.

Listing 7.12 adds the critic agent along with a vision tool (`describe_image`, not shown) and the means to evaluate the styles using our generated rubric. This agent and the image generation agent now run in a loop. The loop is controlled by whether the critic passes or fails the generated image's style. The loop continues as long as the critic fails the output image.

Listing 7.12 08_image_vision_critic_agents.py (critic agent)

```
class CritqueImage(BaseModel):
    """Result of critiquing image."""
    image_pass: bool
```

Creates a strongly typed output that returns feedback and pass/fail results



According to our rubric defined previously, we require a score of 2 out of 5 or higher to pass. This is a low bar set deliberately for the demonstration: the critic passes most images easily and the loop runs to completion in reasonable time. Raising the threshold to 3 or 4 would cause the critic to fail most images and the loop could run indefinitely or hit its retry ceiling.

In production, this threshold is a critical design decision with real consequences. A threshold set too low ships output to users that fails quality standards. A threshold set too high causes most outputs to fail evaluation, which either burns retry budgets, escalates everything to humans, or starves the system of acceptable responses. The right threshold depends on the cost of a false positive (passing bad output) versus a false negative (failing acceptable output) for the specific application.

Calibrating the threshold is part of the rubric validation work covered earlier. Score a sample of outputs against the rubric, have humans label which ones should have passed

and which should have failed, and set the threshold at the score where the agreement between rubric scores and human judgment is highest. Revalidate the threshold whenever the rubric, the agent, or the underlying model changes.

The same principles we used to build the critic agent here can be applied to other use cases where the output is generated and varies widely. Even with that variation, we can establish rubrics which allow critical agents to evaluate complex outputs such as images, reports, graphs, and other complicated examples.

7.4 Phoenix for evaluation and feedback

The default OpenAI Dashboard Traces page can be beneficial, but as your agent development and systems mature, you need more functionality. Arize Phoenix is an excellent open source platform that can be accessed in the cloud or locally in a Docker container. You can also self-host a server that provides access to others, hosts projects, and tracks token usage and costs, tool usage, latency, user sessions, metrics, and, of course, agents. It can also be invaluable for diagnosing complex bugs within an agent process. Without trace-level observability, it's almost impossible to debug or improve complex agent flows at scale.

On top of all that, it provides mechanisms to track and evaluate agent prompts, models, and other factors. It also offers quick connectivity for most agent platforms, including the OpenAI Agents SDK. Figure 7.6 provides an overview of Arize Phoenix, showing the tracing, tracking, and evaluations it can perform.

Phoenix provides the ability to trace and track usage across all your agents, as well as capture and record agent activity. After capturing activity, you can quickly and easily create experiments and evaluators based on metadata and annotations or customize them to your use case. If you are a serious agent engineer, it is highly recommended that you include Phoenix in your set of tracing, evaluation, and development tools.

> **Observability alternatives**
>
> Phoenix is one of several options for production agent observability, and it is worth knowing the landscape before committing. Langfuse is the closest direct alternative (open source, similar feature set, also self-hostable). LangSmith is the established commercial option, especially for teams using LangChain or LangGraph. Weights and Biases Weave provides tracing alongside the broader W&B ML platform. For lighter setups, OpenTelemetry with an OTLP-compatible backend (Honeycomb, Datadog, Grafana Tempo) works without an agent-specific platform. We use Phoenix in this chapter because it is open source, self-hostable, and feature complete for the patterns we cover; the same patterns apply to any of the alternatives.

7.4.1 Connecting to Phoenix

Phoenix uses the underlying OpenTelemetry tracing mechanism that OpenAI Agents and other agent platforms use by default. As such, getting set up is typically quick,

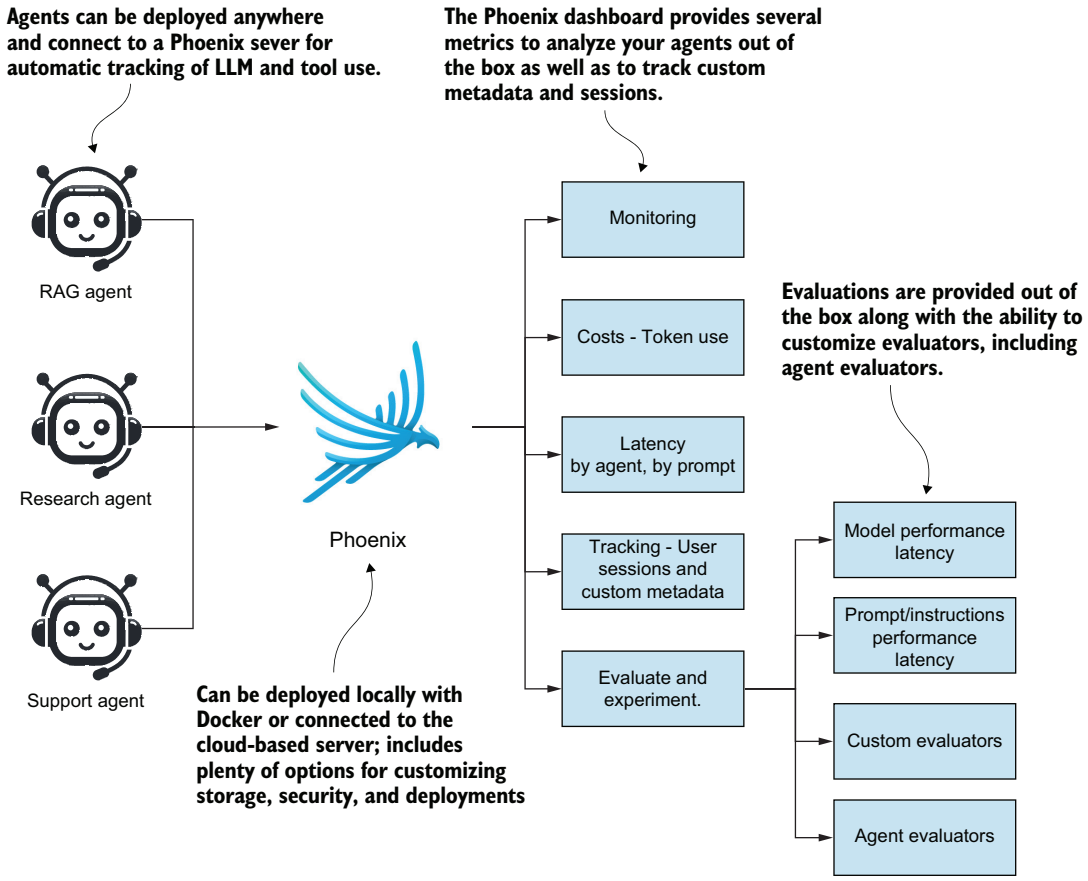


Figure 7.6 **Arize Phoenix deployment patterns and use cases for tracing, tracking, and evaluating agents**

though not always painless. In most cases, you may encounter documentation challenges or package conflicts, but fortunately, GPT-5 or other top-tier models can help resolve these problems.

Listing 7.13 provides the basic code to connect to Phoenix and start tracing. In this code, we first clear the default OpenAI trace processors by setting `set_trace_processors` to an empty list. Then we add the Phoenix trace provider and trace the agent with the agent's trace function. We also set the name of the agent or agent workflow we want to track.

Listing 7.13 09_arize_phoenix_tracing.py (basics)

```
#docker run -it --rm -p 6006:6006 -p 4317:4317 arizephoenix/phoenix:latest

os.environ["PHOENIX_COLLECTOR_ENDPOINT"] = "http://localhost:6006"
```

Sets the collector endpoint you will send traces to

```
set_trace_processors([])
tracer_provider = register(
    project_name="agents",
    auto_instrument=True,
)

agent = Agent(
    name="Assistant",
    instructions="You are a helpful assistant")

async def main():
    with trace("Haiku Generator"):
        result = await Runner.run(
            agent, "Write a haiku about recursion in programming."
        )
        print(result.final_output)
```

Clears the default trace processors from the stack; replaces the OpenAI trace processor

The project name is the project to which the traces will be sent in Phoenix.

Auto-instrument your app based on installed dependencies.

Uses the OpenAI trace to tag the agent or set of agents we want to track

You can have one or more agents within the trace function, which lets you track agents and agent workflows. The top commented-out line in the code is the command to run Phoenix as a local Docker container. To do this, you first need to install Docker Desktop. Download and follow the Docker installation instructions to install Desktop on your computer.

With Phoenix running in Docker and the code in listing 7.13 completed, open a browser to http: //localhost: 6006 and click the Agents project. You should see the dashboard view shown in figure 7.7. If the Agents project isn't immediately visible, refresh your browser, and it should appear.

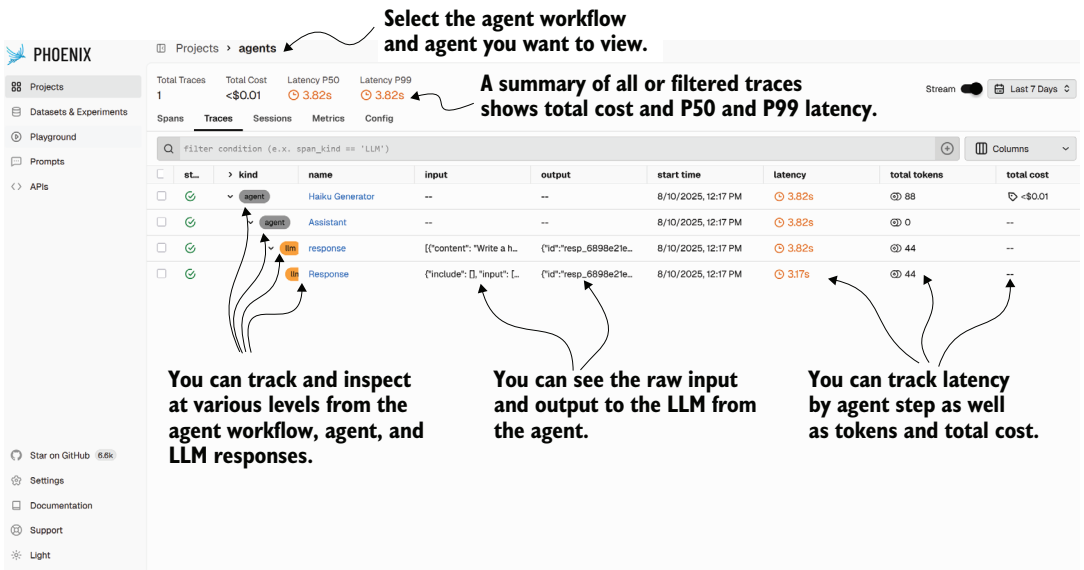


Figure 7.7 Arize Phoenix Projects dashboard showing an agent workflow, agent, and LLM activity

From the Projects page, you can select View Traces, Sessions, Metrics, and Config. On the Traces page, you can drill into the agent workflow, agent, and LLM responses to view latency, token usage, and the raw responses sent to the LLM. You can further enhance this view by adding metadata and session tracking, as we will see in the next section.

7.4.2 Adding metadata and session tracking

Listing 7.14 shows how to add metadata and session tracking to an agent or agent workflow. Here, we add metadata to the agent and LLM traces and track a session. We add `using_session` and `using_metadata` in a `with` block so they attach to all internal traces. Make sure to keep the agent trace block inside any other tracing.

Listing 7.14 10_arize_phoenix_metadata.py (only relevant code)

```
model = "gpt-5-mini"
agent = Agent(name="Assistant", instructions="Always answer in a Haiku",
     model=model)

async def main():
    agent_input = dict(question="why is the sky blue?")
    metadata = dict(
        run_id="abc123",
        env="dev",
        customer_tier="pro",
        model=model)

    with (
        using_session("sess-42"),
        using_metadata(metadata),
    ):
        with trace("Haiku Generator"):
            result = await Runner.run(agent, str(agent_input))
            print(result.final_output)
```

Defines a model the agent will use

Wraps the input in a dictionary for better input formatting

Defines a dict to contain the metadata key/value entries

Wraps the code in a session and metadata trace context

Always place the agent workflow trace inside other trace objects.

When you run the code in listing 7.14 and then open Phoenix to the Agents project, you should see something similar to figure 7.8. To see the Metadata column, open the Columns selector in the top right, select the Metadata column, and turn other columns on or off. When the Metadata column is visible, you can click it to expand a selector that lets you quickly filter the traces.

Selecting the Metadata column opens a selector that lets you filter on any of the fields by value or expression. This can be a powerful feature for evaluating specific agent activity. Still, it does require you to configure the metadata appropriate to your agent's use case.

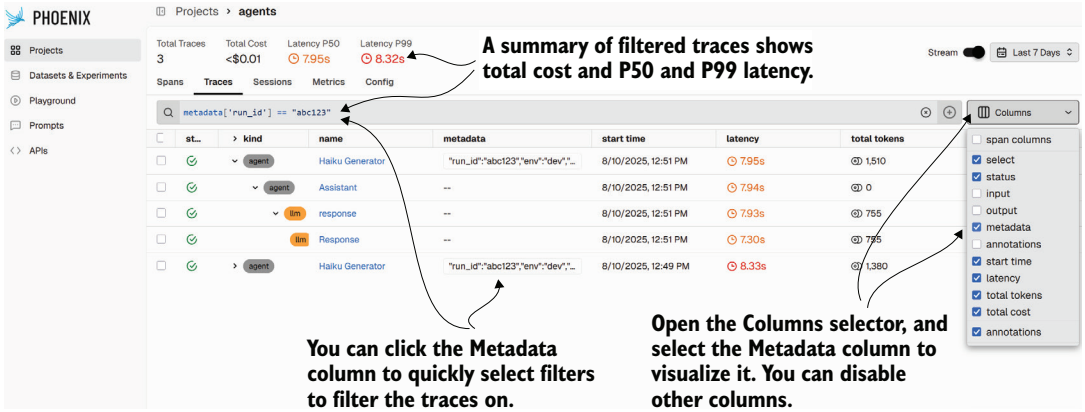


Figure 7.8 **Viewing the Metadata column and filtering traces by an attribute**

7.4.3 *Experimenting with evaluators*

Experimentation and evaluation are powerful features of Phoenix and can be quickly set up and run from the dashboard. This is essentially the "outer loop" of TDAD, where we turn real traces into datasets, run evaluators on them, and feed the results back into prompts, tools, and policies. Before evaluating, we need to create a dataset of spans (LLM responses). Figure 7.9 walks you through selecting and adding spans to a dataset.

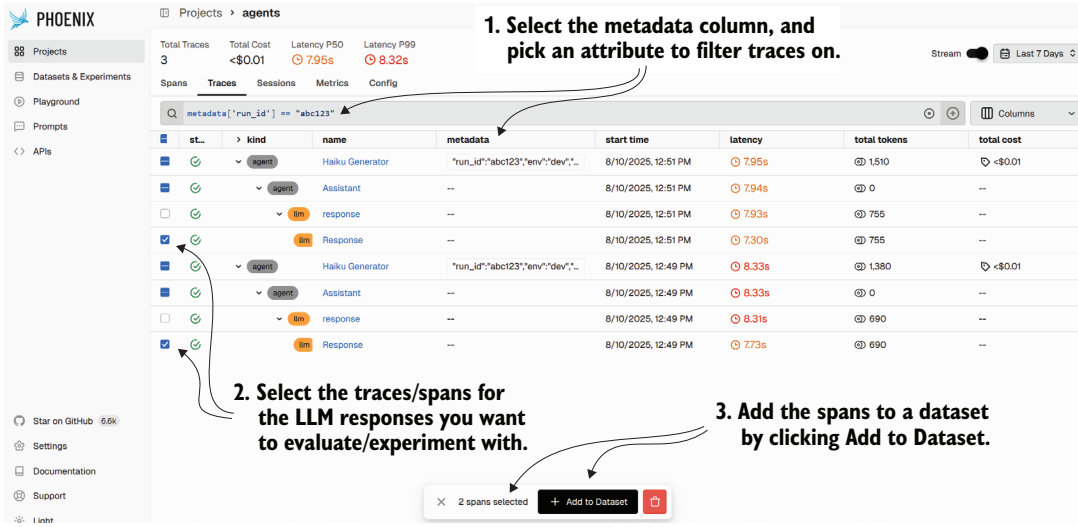


Figure 7.9 **The process of adding spans to a dataset for later experimentation and evaluation**

After adding spans to a dataset by selecting them and then clicking Add to Dataset, you'll be prompted to create a new dataset. Use the default name, and click Create Dataset. After the dataset is created, a green pop-up will appear in the lower-right corner. Click View Dataset, and you'll see a page like the one shown in figure 7.10.

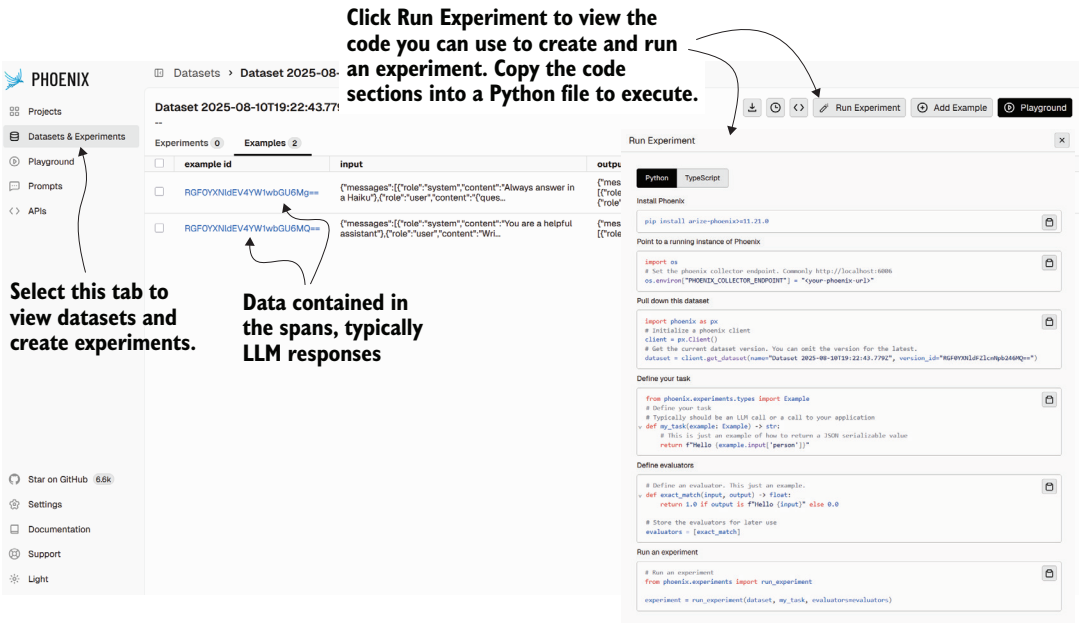


Figure 7.10 Viewing the dataset and clicking Run Experiment shows the generated code. You can copy the code into a Python file to create an experiment or evaluation.

Click the Run Experiment button in the top-right corner to show the code you need to copy and paste into a Python file and then execute. The code provides a simple example of a basic task and evaluator you can run on all the spans in your dataset. Generally, you will want to evaluate LLM responses, but you could also use tool calls, handoffs, and other custom spans you add.

7.4.4 Providing feedback with annotations

Phoenix also provides a powerful mechanism for tracking feedback using annotations. This feedback can be added later through code or tracked directly through the Phoenix dashboard. Before using this feature, though, we must create a set of annotations, as shown in figure 7.11.

After we create the annotation, we can use it by returning to the Traces page and selecting a span we want to mark with the annotation. In most cases, you will again want to attach annotations to spans that capture LLM responses. Figure 7.12 shows how to add an annotation to a span with an LLM response.

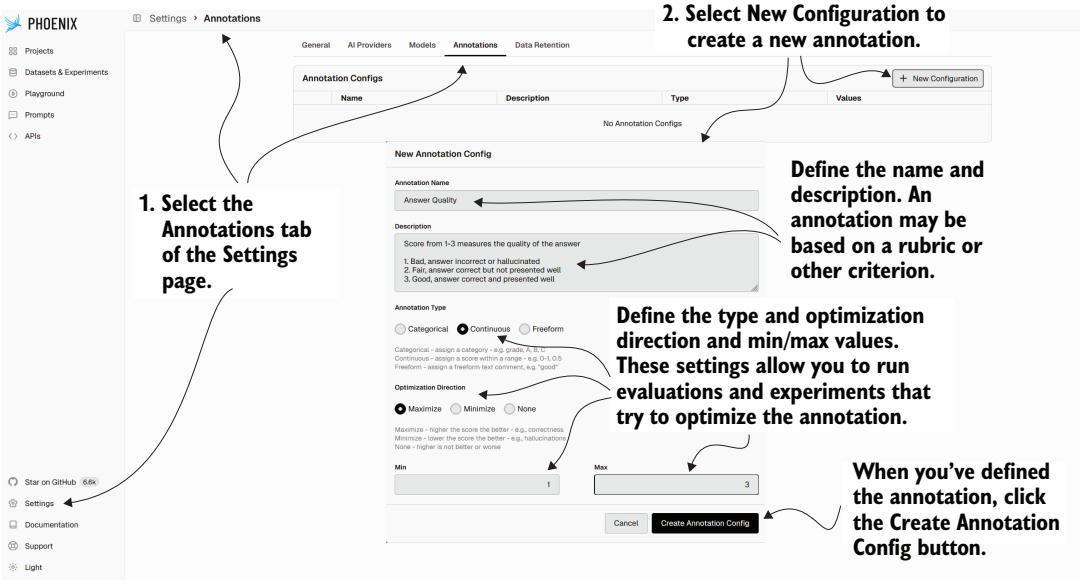


Figure 7.11 Creating a new annotation in Phoenix

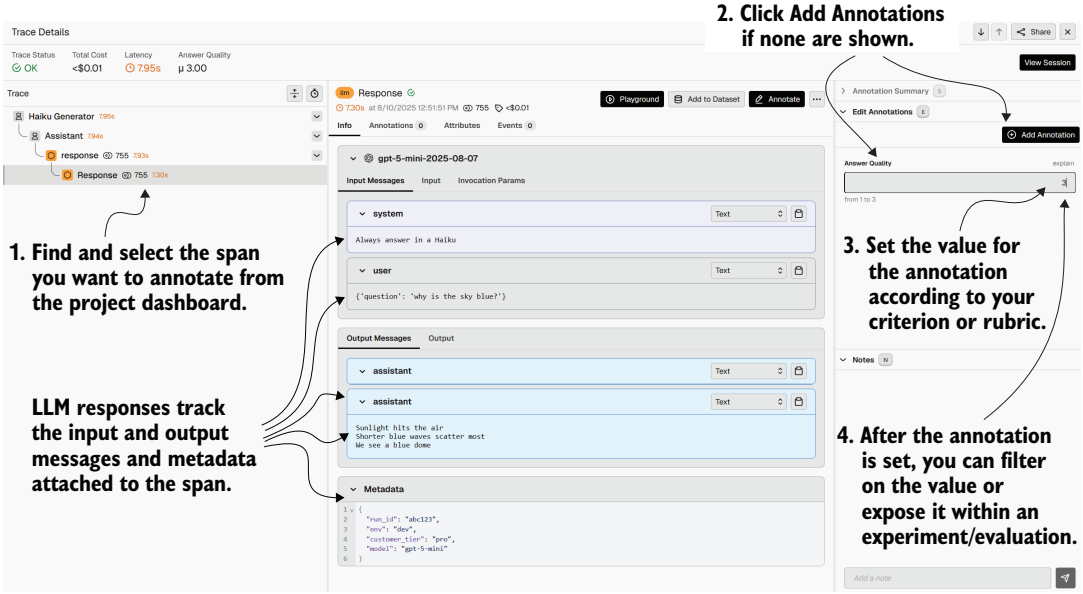


Figure 7.12 Steps to apply an annotation to an LLM response within an agent workflow and across agent and LLM response spans

Annotations let internal reviewers (developers, QA, domain experts) attach structured labels to specific spans or traces, marking things like "incorrect tool selection," "good response," "needs review," or any custom label that fits your evaluation needs. These annotations get stored alongside the trace data and become queryable signals you can use later.

The payoff is that annotations turn ad hoc judgment into structured data. A developer noticing that the agent picked the wrong tool can flag the span, and later you can pull every span flagged with that label, compare them against the unflagged spans, and look for patterns (does it happen on long prompts, with specific tools, after certain types of user queries). Filtering the trace dashboard by annotation lets you cross-reference quality labels against operational metrics like latency, cost, and token usage to see whether bad responses correlate with specific cost or performance patterns.

This is also how informal review becomes a dataset over time. Spans flagged "incorrect" across many sessions become a regression test set: rerun the agent against those inputs after a prompt change and see whether the new version handles them correctly. Without annotation, the same observations live in Slack threads and meeting notes and never become reusable evaluation material.

Incorporating the principles and patterns of evaluation and feedback into your agent development and production cycle not only improves the quality of those agents but also increases their robustness. As a golden rule, you should never promote agents into production systems unless you employ evaluation and feedback patterns.

7.5 Exercises

Use the following exercises to improve your knowledge of the material:

Exercise 1: Run the baseline TDAD RAG benchmarks.
Objective: Run the starter RAG agent and observe the initial benchmark behavior (which fails).
Tasks:

1. Open listing 7.1 and save a copy as exercise1_tdad_baseline.py.
2. Run the script once. Note the answers and pass/fail for each benchmark.
3. Run it two more times to check variability.
4. Add a short comment at the bottom summarizing pass rate across the three runs (e.g., "Run1: 0/5, Run2: 1/5, Run3: 0/5").

Expected duration: 10 minutes

Exercise 2: Refactor for higher pass rate (tool and prompt tweaks).
Objective: Improve correctness by refining instructions and clarifying tool usage.
Tasks:

1. Duplicate your previous file to exercise2_refactor_rag.py.
2. Update the agent instructions as in listing 7.2 (for example, mention breaking down the query; optionally constrain the answers to single words).

3. Rename and implement the `keyword` tool exactly as in listing 7.4 (`search_knowledge_by_keyword`) and register it with the agent.
4. Pin the model to `model="gpt-5"`.
5. Run the benchmarks three times. Record pass rate each time and keep the best output sample in a comment. Aim for ≥ 80% on at least one run.

Expected duration: 12 minutes

Exercise 3: Add a typed evaluation agent.
Objective: Evaluate answers with a second agent that returns a typed pass/fail result plus feedback.
Tasks:

1. Copy your file to exercise3_evaluation_agent.py.
2. Add `EvaluationOutput` and the evaluation agent exactly as in listing 7.5.
3. Modify the loop to call the evaluator as in listing 7.6 and print `is_correct and feedback`.
4. Run once; then run again to compare consistency.
5. Add a comment summarizing (a) total correct, and (b) one useful piece of evaluator feedback.

Expected duration: 12 minutes

Exercise 4: Enforce grounding with an output guardrail.
Objective: Block ungrounded answers using a grounding agent and a guardrail.
Tasks:

1. Duplicate your file to exercise4_grounding_guardrail.py.
2. Implement `get_last_context`, `GroundedAnswer`, and `grounding_agent` as in listing 7.7.
3. Add the output guardrail function from listing 7.9 and register it on the RAG agent.
4. Wrap the run loop as shown in listing 7.10 to catch `OutputGuardrailTripwireTriggered`. On trip, print the guardrail info and return `"No grounded answer available."`.
5. Run once. Verify at least one question prints “Guardrail tripped” (or confirm all pass grounded checks).

Expected duration: 15 minutes

Exercise 5: Trace and triage with Phoenix.
Objective: Capture traces with Phoenix, attach metadata, and annotate one span.
Tasks:

1. Copy your latest file to exercise5_phoenix_tracing.py.

2. Add the Phoenix setup from listing 7.13 (set `PHOENIX_COLLECTOR_ENDPOINT`, clear trace processors, register project with `project_name="agents"`), and wrap runs with `trace("RAG Benchmarks")`).
3. Add session plus metadata from listing 7.14 using `using_session("sess-07")` and `using_metadata({"run_id":"ch7-ex5","env":"dev","model":"gpt-5"})`.
4. (Optional) Start Phoenix locally with the provided Docker command. Run the script once.
5. In Phoenix, select Agents project > Traces. Filter by `run_id = ch7-ex5`, select any LLM span, and add an annotation (e.g., `correctness=pass` or `correctness=fail`).

Expected duration: 15 minutes

Summary

- Robust agents require evaluation and feedback baked in from the start. Tie the Learn step of sense-plan-act-learn to external evaluators, a feedback store, and (optionally) a monitoring/alerting feedback agent.
- Use a mix of methods: red team testing for safety breaks, benchmark testing for goal attainment, human feedback (thumbs/comments) with verification to reduce bias, and automated evaluators (grounding/critic/evaluation agents) where appropriate.
- Test-driven agent development (TDAD) flips the process: define benchmarks (including wrong examples), expect initial failures, and run each test multiple times for consistency. Then make the minimum prompt, tool, or model changes needed to pass, retest, and refactor iteratively.
- When benchmarks conflict, decide explicitly: accept partial pass targets, split into specialized agents (plus triage), or refactor the agent's instructions from first principles.
- Prefer tool guidance in the tool's name/docstring over prompt clutter; rename tools and write clear docstrings; keep prompts lean and role focused.
- Add evaluation agents to score complex, natural-language outputs with typed results (e.g., pass/fail plus feedback), avoiding brittle string matching and enabling programmatic gating.
- Grounding agents are essential for RAG; they verify that answers are supported by the provided context or citations. On failure, they can regenerate, block with a static response, or pass grounded feedback forward.
- Guardrails operationalize grounding. Wrap agents with output guardrails that trip on ungrounded answers and surface feedback and flags for policy (retry, block, or annotate).
- Rubrics turn subjective quality into criteria and scales. Use them with critic agents for high-variance generation (images, reports, graphs). Loop generation critique until a passing score or attempt limit.

- Because LLMs are stochastic, benchmark for stability, not just accuracy, by rerunning tests and watching variability, latency, and token use.
- Observability is nonnegotiable: OpenAI Traces show every tool call and LLM turn; Arize Phoenix (OpenTelemetry) adds projects, sessions, metrics, costs, experiments, and evaluators.
- Instrument Phoenix cleanly: set the collector endpoint, replace the default processors, register a tracer, and wrap runs with named traces. Attach sessions and metadata for filtering and cohort analysis.
- Build datasets from spans, run experiments/evaluators, and use annotations to capture structured human feedback. Feed these signals back into prompts, tools, and policies.
- Production rule: never promote agents without active evaluation/feedback loops, benchmarks, grounding/guardrails, critics with rubrics, tracing, and monitored KPIs (latency, cost, quality) working together.